Understanding LLM Evaluation: A Practical Guide to Ten Group Topics

A combined study guide based on the ten group presentations for the “LLM Performance Evaluation” group-discussion activity

Item	Details
Course	Advanced Programming (Advanced Programming for Data Science)
Institution	Institute of Technology of Cambodia (ITC)
Department	Department of Applied Mathematics and Statistics
Lecturer	Mr. VANN Visal
Activity	Group Discussion Assignment: *Evaluation Methods for Large Language Models (LLMs)*, Chapter 3 activity (course file `Activity_12A-Nov-Ch3-ADV_Programming_Friday.pdf`)
Academic year	2025–2026 (as printed on Group 8’s title slide)
Compiled by	**[PLACEHOLDER: name(s) of the person or group who compiled this combined report]**
Student ID(s)	**[PLACEHOLDER: student ID(s) of the compiler(s)]**
Submission date	**[PLACEHOLDER: submission date]**
Source groups	Groups 1–10 (see Appendix D)

Overview. Large language models (LLMs) write answers, translations, summaries and transcripts. To decide whether those outputs are good, we need *evaluation methods* (systematic ways of measuring quality). Ten student groups each studied one method. This guide turns their presentations into one connected textbook. Every chapter explains one method from the beginning: what question it answers, its formula, worked calculations, Python code, and its limits. The guide corrects a small number of technical errors found in the slides, keeps teaching examples separate from real model experiments, and ends with advice on choosing and combining methods.

Table of Contents

How to Use This Report 5

Quick Guide: Where to Find Each Topic 6

Introduction: Why and How We Evaluate LLMs 7

- Why LLM evaluation is needed 7
- Reference answers and model predictions 7
- Classification versus generated-text evaluation 7
- Tokens: the units we count 8
- Automatic scores versus human review 8
- Why one score cannot measure every aspect of quality 8
- Short glossary of essential terms 8

Chapter 1: Exact Match (Group 1) 10

- 1.A Learning goals 10
- 1.B Plain-English introduction 10
- 1.C Key terms 10
- 1.D Formula and reasoning 10
- 1.E Worked examples 11
- 1.F Python implementation 12
- 1.G How to interpret results 13
- 1.H Strengths and limitations 14
- 1.I When to use it 14
- 1.J Common misunderstandings 14
- 1.K Quick recap and practice 14
- 1.L Sources 15

Chapter 2: Accuracy (Group 2) 16

- 2.A Learning goals 16
- 2.B Plain-English introduction 16
- 2.C Key terms 16
- 2.D Formula and reasoning 16
- 2.E Worked examples 17
- 2.F Python implementation 18
- 2.G How to interpret results 19
- 2.H Strengths and limitations 19
- 2.I When to use it 19
- 2.J Common misunderstandings 19
- 2.K Quick recap and practice 20
- 2.L Sources 21

Chapter 3: Precision, Recall, and F1-score (Group 3) 22

- 3.A Learning goals 22
- 3.B Plain-English introduction 22
- 3.C Key terms 22
- 3.D Formula and reasoning 23
- 3.E Worked examples 24
- 3.F Python implementation 25
- 3.G How to interpret results 27
- 3.H Strengths and limitations 27
- 3.I When to use it 27
- 3.J Common misunderstandings 27
- 3.K Quick recap and practice 28
- 3.L Sources 28

Chapter 4: BLEU (Group 4) 29

- 4.A Learning goals 29
- 4.B Plain-English introduction 29
- 4.C Key terms 29
- 4.D Formula and reasoning 30
- 4.E Worked examples 31
- 4.F Python implementation 32
- 4.G How to interpret results 34

4.H Strengths and limitations 35
4.I When to use it 35
4.J Common misunderstandings 35
4.K Quick recap and practice 35
4.L Sources 36

Chapter 5: ROUGE (Group 5) 37
5.A Learning goals 37
5.B Plain-English introduction 37
5.C Key terms 37
5.D Formula and reasoning 37
5.E Worked examples 39
5.F Python implementation 40
5.G How to interpret results 42
5.H Strengths and limitations 43
5.I When to use it 43
5.J Common misunderstandings 43
5.K Quick recap and practice 43
5.L Sources 44

Chapter 6: METEOR (Group 6) 45
6.A Learning goals 45
6.B Plain-English introduction 45
6.C Key terms 45
6.D Formula and reasoning 45
6.E Worked examples 47
6.F Python implementation 48
6.G How to interpret results 49
6.H Strengths and limitations 49
6.I When to use it 50
6.J Common misunderstandings 50
6.K Quick recap and practice 50
6.L Sources 51

Chapter 7: BERTScore (Group 7) 52
7.A Learning goals 52
7.B Plain-English introduction 52
7.C Key terms 52
7.D Formula and reasoning 53
7.E Worked examples 54
7.F Python implementation 55
7.G How to interpret results 57
7.H Strengths and limitations 57
7.I When to use it 57
7.J Common misunderstandings 57
7.K Quick recap and practice 57
7.L Sources 59

Chapter 8: Word Error Rate — WER (Group 8) 60
8.A Learning goals 60
8.B Plain-English introduction 60
8.C Key terms 60
8.D Formula and reasoning 61
8.E Worked examples 62
8.F Python implementation 63
8.G How to interpret results 65
8.H Strengths and limitations 65
8.I When to use it 65
8.J Common misunderstandings 65
8.K Quick recap and practice 65
8.L Sources 67

Chapter 9: Character Error Rate — CER (Group 9) 68
9.A Learning goals 68
9.B Plain-English introduction 68
9.C Key terms 68
9.D Formula and reasoning 69

9.E Worked examples 70
9.F Python implementation 71
9.G How to interpret results 73
9.H Strengths and limitations 73
9.I When to use it 74
9.J Common misunderstandings 74
9.K Quick recap and practice 74
9.L Sources 76

Chapter 10: Comparative Evaluation and Choosing Methods (Group 10) 77
10.1 Learning goals 77
10.2 Plain-English introduction 77
10.3 Four families of methods 77
10.4 Comprehensive comparison table 78
10.5 Practical selection examples 79
10.6 Why methods can disagree 80
10.7 Combining automatic metrics with human checks 80
10.8 Why the groups' experiments cannot rank models 81
10.9 Python implementation 81
10.10 Common misunderstandings 83
10.11 Quick recap and practice 83
10.12 Sources 83

Conclusion: Main Lessons 84

Appendix A: Formula Reference Sheet 85

Appendix B: Glossary 86

Appendix C: Answers to Review Questions 88

Appendix D: Source-to-Chapter Credits 89

Appendix E: Experiments, Missing Information and Corrections 91
E.1 Evidence categories used in this guide 91
E.2 Reported experiments: what is documented and what is missing 91
E.3 Corrections and clarifications to the slides 92

Appendix F: References 93

How to Use This Report

If you are new to the topic, read the Introduction first. It explains the basic ideas that every chapter uses: references, predictions, classification, tokens, and the difference between automatic scores and human review.

If you need one method, go straight to its chapter. Each chapter can be studied on its own. When a chapter needs an idea from another chapter, it gives a short explanation and a link.

Every method chapter uses the same twelve parts:

Part	What you will find
A. Learning goals	What you should understand by the end
B. Plain-English introduction	The question the method answers; what it measures and does not measure
C. Key terms	Words you need, each with a small example
D. Formula and reasoning	Every symbol explained, with the range and the "better" direction
E. Worked examples	At least two step-by-step calculations
F. Python implementation	Runnable code, its inputs, and its real output
G. Interpreting results	What a high or low score does and does not show
H. Strengths and limitations	Each limitation linked to an example
I. When to use it	Suitable tasks and the extra checks they need
J. Common misunderstandings	"Misunderstanding → correction" pairs
K. Recap and practice	A summary and three questions with answers
L. Sources	The papers, documentation and slides behind the chapter

Labels used for evidence. This guide separates three kinds of material, because mixing them is a common source of wrong conclusions:

Label	Meaning
Teaching example	A small input made up to explain an idea. It is *not* a model output.
Code verification	A teaching example run through Python code to check the arithmetic.
Reported experiment	Outputs that a group says it collected from real models. We report what the group documented and say what is missing.

About the code. All Python examples in Chapters 1–10 were executed for this guide on 13 November 2026 in an isolated virtual environment (Python 3.13.13 on Linux; scikit-learn 1.9.1, sacreBLEU 2.6.0, NLTK 3.10.4, rouge-score 0.1.2, jiwer 4.0.0, bert-score 0.3.13, PyTorch 2.15.0 CPU, Transformers 5.23.0, regex 2026.11.9). The printed outputs shown in this guide were copied from those runs. The scripts are stored in `Combined_LLM_Evaluation_Report_build/code/` and their outputs in `Combined_LLM_Evaluation_Report_build/outputs/`. Library versions change; newer versions may print slightly different numbers or messages.

Ratios and percentages. Most scores in this guide can be written as a ratio (for example 0.8) or a percentage (80%). They mean the same thing: multiply a ratio by 100 to get a percentage. A stored value of 0.4 is 40%, not 0.4%. BLEU is often printed on a 0–100 scale; that is the same idea.

Quick Guide: Where to Find Each Topic

If you want to...	Read	Group
check whether a short answer is exactly right	Chapter 1: Exact Match	1
count the share of correct labels or choices	Chapter 2: Accuracy	2
study false alarms and missed cases	Chapter 3: Precision, Recall, F1	3
score a translation against references	Chapter 4: BLEU	4
score a summary by content coverage	Chapter 5: ROUGE	5
give credit for word forms and synonyms	Chapter 6: METEOR	6
compare meaning with neural embeddings	Chapter 7: BERTScore	7
measure speech-transcript word errors	Chapter 8: WER	8
measure character errors (OCR, spelling, Khmer)	Chapter 9: CER	9
choose and combine methods for a task	Chapter 10: Comparison	10
look up a formula quickly	Appendix A	—
look up a word	Appendix B	—

Introduction: Why and How We Evaluate LLMs

Why LLM evaluation is needed

A large language model (LLM) is a computer program trained on very large amounts of text so that it can continue or answer text. GPT, Claude, Gemini, Llama and Qwen are examples named in the teacher's assignment. LLM outputs often *look* fluent, but fluent text can still be wrong, incomplete, unsafe or badly formatted. Looking at a few answers by eye is useful, but it is slow, it depends on who is looking, and it cannot be repeated exactly on thousands of answers.

Evaluation means measuring output quality in a planned, repeatable way. It helps us to:

- compare two models on the **same** test questions;
- see whether a change to a prompt or a model made things better or worse;
- find the *kinds* of mistakes a model makes; and
- report results that another person can check.

Reference answers and model predictions

Most methods in this guide compare two pieces of text:

- the **reference** (also called the *ground truth* or *gold answer*): the answer we accept as correct, usually written or checked by a person; and
- the **prediction** (also called the *hypothesis*, *candidate* or *model output*): what the model produced.

> **Teaching example I.1.** Question: "What is the capital of Cambodia?" Reference: `Phnom Penh`. Prediction: `The answer is Phnom Penh.` A person sees that the prediction is correct. A strict text comparison sees two different strings. Much of this guide is about this gap between *what a person means* and *what a computer can count*.

A score is only as good as its reference. If the reference is wrong, a correct prediction is punished. If only one reference is given but many answers are acceptable, valid alternatives are punished. Some methods therefore allow **several references** for one question.

Classification versus generated-text evaluation

Two broad kinds of tasks appear in this guide.

Classification means choosing one item from a fixed list of *labels* (allowed categories). Examples: deciding whether a review is Positive or Negative; choosing A, B, C or D in a multiple-choice question. Because the list is fixed, each prediction is simply right or wrong. Chapters 2 and 3 (Accuracy; Precision, Recall and F1) are mainly about classification.

Generated-text evaluation applies when the model writes free text: a translation, a summary, a transcript or an explanation. Many different texts can be correct, so "right or wrong" is often too simple. Methods for generated text (Chapters 4–9) instead measure *how close* the prediction is to the reference, using shared words, shared word sequences, edits, or meaning-based similarity.

Exact Match (Chapter 1) sits between the two: it treats short generated answers as either identical to a reference or not.

Tokens: the units we count

Most text metrics first cut text into **tokens** (small units that the method counts). Depending on the method, a token can be a word, a punctuation mark, a character, or a piece of a word.

> **Teaching example I.2.** The sentence `The cat sleeps.` can be cut into word tokens `The | cat | sleeps | .` (4 tokens), or into character tokens `T | h | e | ␣ | c | …` (15 characters, where ␣ marks a space). Neural models such as BERT use *subword* tokens; for example, a rare word may become two or three pieces.

The way text is cut into tokens is called **tokenization**. Different tokenization rules can change a score, even for the same texts. Languages written without spaces between words, such as Khmer, need special care: splitting on spaces does not give words (see Chapter 9 and Chapter 10).

Automatic scores versus human review

An **automatic metric** is a formula or program that turns a prediction and a reference into a number. It is fast, cheap and repeatable. **Human review** means people read the outputs and judge them, often with a checklist (correctness, completeness, fluency, safety). Human review notices meaning, facts and usefulness, but it is slower, costs more and can vary between reviewers.

Automatic metrics were designed to *approximate* some parts of human judgment on particular tasks. The original BLEU paper, for example, presented BLEU as an "understudy" (a stand-in) for skilled human judges of translation, not a replacement for them (Papineni et al., 2002). Group 4's slides make the same point: BLEU is "one automatic metric—not a replacement for human judgment."

Why one score cannot measure every aspect of quality

Quality has many parts: Is the answer correct? Complete? Fluent? Faithful to the source? Safe? In the requested format? Each metric looks at one or two of these parts through a narrow window.

> **Teaching example I.3 (code verification, Chapter 10).** Reference: `The medicine is safe for children.` Prediction: `The medicine is not safe for children.` The meaning is reversed, which could be dangerous. Yet in this guide's runs the prediction received ROUGE-L F1 0.92, METEOR 0.97 and BERTScore F1 0.94 (all close to their maximum of 1), and a word error rate of only 16.7%. A correct paraphrase, `Children can take this medicine safely.`, scored much *lower* on all of these. The full table is in Section 10.6.

The lesson is simple: **a metric measures what its formula counts, and nothing else.** Choose metrics that match what matters for the task, use more than one when needed, and add human or factual checks when mistakes are costly.

Short glossary of essential terms

A longer glossary is in Appendix B.

Term	Basic meaning	Tiny example
Reference	The accepted correct answer	`Paris`
Prediction	The model's answer	`paris`
Label	One allowed category in classification	`Positive`

Term	Basic meaning	Tiny example
Token	A unit of text that a metric counts	`cat` in `the cat`
n-gram	A run of *n* neighbouring tokens	2-gram: `the cat`
Probability	A number from 0 to 1 that says how likely something is	a fair coin lands heads with probability 0.5
Logarithm (log)	The power to which a base must be raised; logs turn products into sums	$\ln(a \times b) = \ln a + \ln b$
Embedding	A list of numbers that represents a word or text so that similar meanings get similar numbers	“happy” and “joyful” get nearby vectors
Edit distance	The smallest number of insertions, deletions and substitutions to turn one text into another	`cat` → `cut` needs 1
Dynamic programming	Solving a large problem by saving and reusing answers to smaller sub-problems	the edit-distance table in Chapter 9
Precision	Of what the system *produced*, how much was right	8 correct out of 10 predicted
Recall	Of what *should* be found, how much was found	8 found out of 20 that exist
Corpus	A collection of texts, such as a whole test set	500 test sentences

Chapter 1: Exact Match (Group 1)

1.A Learning goals

By the end of this chapter you should be able to:

- explain what Exact Match (EM) checks and why it gives only 0 or 1 per question;
- calculate dataset EM by hand;
- explain the difference between *raw* (strict) EM and EM after *text cleanup* (normalization);
- handle questions that have several acceptable references; and
- explain why a correct answer with different wording can still score 0.

1.B Plain-English introduction

Question answered: "Is the model's answer *exactly* the same text as an accepted answer?"

Exact Match compares the prediction with the reference as text. If the two strings are identical, the question scores 1. Otherwise it scores 0. The dataset score is the share of questions that scored 1.

What it measures: exact textual agreement for short answers, such as a name, a number or a date. It is widely used in extractive question answering, where the answer is a short span copied from a passage. The SQuAD reading-comprehension dataset, for example, reports EM together with a word-overlap F1 score (Rajpurkar et al., 2016).

What it does not measure: meaning, partial correctness, or quality of explanation. `Four` and `4` mean the same, but they are different strings, so strict EM gives 0.

1.C Key terms

- **String:** a sequence of characters stored by a computer, such as `"Phnom Penh"`.
- **Raw (strict) comparison:** compares strings as they are. `Paris` and `paris` are different because of the capital letter.
- **Normalization (text cleanup):** a fixed set of changes applied to *both* texts before comparison, such as making all letters lowercase. After lowercasing, `Paris` and `paris` both become `paris`.
- **Whitespace:** spaces, tabs and line breaks. `Phnom Penh` (two spaces) differs from `Phnom Penh` in a raw comparison.
- **Indicator function** $\mathbb{1}[\cdot]$: a function that returns 1 when the statement inside the brackets is true and 0 when it is false. Example: $\mathbb{1}[2 = 2] = 1$ and $\mathbb{1}[2 = 3] = 0$.

1.D Formula and reasoning

Suppose a test set has N questions. For question i, let P_i be the prediction and R_i the reference. Group 1 wrote the dataset score as:

$$\mathrm{EM} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}\left[\, P_i = R_i \,\right]$$

Reading the formula.

- $\mathbb{1}[P_i = R_i]$ is 1 when the two strings are identical, else 0. This is the per-question score.
- $\sum_{i=1}^{N} \mathbb{1}[P_i = R_i]$ ("sum for i from 1 to N") adds the per-question scores. The sum is the **number of exact matches**.

- Dividing by N turns the count into a **proportion** (a share of the total). So the numerator is "exact matches" and the denominator is "all questions".

With text cleanup. Let $\mathrm{norm}(\cdot)$ be a normalization function. Then the per-question score is $\mathbb{1}[\mathrm{norm}(P_i) = \mathrm{norm}(R_i)]$. The SQuAD v1.1 evaluation script, which many projects copy, lowercases text, removes punctuation, removes the English articles *a*, *an* and *the*, and collapses repeated whitespace (SQuAD evaluation code, as reproduced in Hugging Face *evaluate*). These rules are a *choice*, not part of EM itself, and they are designed for English.

With several acceptable references. If question i has a set of references $\{R_{i,1}, R_{i,2}, \ldots\}$, the usual rule is to take the best match: the question scores 1 if the prediction equals *any* reference:

$$\mathrm{EM}_i = \max_k \mathbb{1}\left[\mathrm{norm}(P_i) = \mathrm{norm}(R_{i,k})\right]$$

Range and direction. Each question scores 0 or 1, so dataset EM lies between 0 and 1 (0% to 100%). **Higher is better.**

Edge cases and implementation choices.

- *Empty test set* ($N = 0$): the formula divides by zero. Group 1's code correctly raises an error.
- *Unequal list lengths*: predictions and references must be paired one-to-one; Group 1's code checks this too.
- *Which cleanup rules?* Lowercasing, punctuation removal, article removal and whitespace handling must be stated, because each one can change the score.
- *Non-English text:* Python's `string.punctuation` contains only ASCII punctuation (basic English keyboard symbols). The Khmer full stop `។` is not in it, so the SQuAD-style rule does not remove it. Article removal is meaningless for Khmer.

1.E Worked examples

Example 1.1 — Five questions (from Group 1's slides; illustrative, not real model outputs)

Table 1.1 — Group 1's five-question EM example.

#	Question	Reference	Prediction	Identical?	Score
1	Capital of France?	Paris	Paris	yes	1
2	2 + 2?	4	4	yes	1
3	Largest planet?	Jupiter	Jupiter	yes	1
4	Chemical formula of water?	H2O	Water	no	0
5	Capital of Cambodia?	Phnom Penh	Phnom Penh	yes	1

Step 1: count the matches: $1 + 1 + 1 + 0 + 1 = 4$. Step 2: divide by the number of questions: $4/5 = 0.80$. Step 3: as a percentage, $0.80 \times 100 = 80\%$.

Question 4 shows a real weakness: "Water" names the substance correctly but does not give the requested *formula*, and in any case it is not the same string as "H2O".

Example 1.2 — Raw versus normalized EM (teaching example)

Table 1.2 — Effect of SQuAD-style normalization (verified with the code in 1.F).

Reference	Prediction	Raw EM	Normalized EM	What changed
Paris	paris	0	1	lowercasing
Phnom Penh	Phnom Penh!	0	1	punctuation removed
The United States	United States	0	1	article “the” removed
Phnom Penh	The answer is Phnom Penh	0	0	extra words remain

The first three rows come from Group 1’s slides. The fourth row exposes a limitation: cleanup removes small surface differences, but it does not remove whole extra phrases. The answer is correct in meaning but still scores 0. This is why projects that use EM usually tell the model to “answer with only the name”, or extract the answer before scoring.

Example 1.3 — Several acceptable references (teaching example)

Question: “What is 2 + 2?” Accepted references: `4` and `four`. Prediction: `four`. Comparing with `4` gives 0, comparing with `four` gives 1. The maximum is 1, so the question scores 1. With only the single reference `4`, the same correct answer would score 0. Group 1’s experiment slide shows exactly this effect: “Model B” wrote `Four` instead of `4` and lost the point.

1.F Python implementation

Package needed: none (standard library only). **Input format:** two lists of equal length; each reference item may be one string or a list of acceptable strings.

```
import re
import string


def normalize_answer(text):
    """SQuAD v1.1-style cleanup: lowercase, remove punctuation,
    remove English articles (a, an, the), and tidy whitespace."""
    text = text.lower()
    # string.punctuation holds ASCII punctuation only, e.g. "!" and ".".
    text = "".join(ch for ch in text if ch not in set(string.punctuation))
    text = re.sub(r"\b(a|an|the)\b", " ", text)
    return " ".join(text.split())  # split() + join() collapses all spaces
```

The order of steps follows the SQuAD script: lowercase first, then punctuation, then articles, then whitespace. `\b` in the regular expression means “word boundary”, so `the` is removed but `there` is not.

```
def exact_match(prediction, references, normalize=False):
    """Return 1 if the prediction equals ANY acceptable reference, else 0."""
    if isinstance(references, str):  # allow one reference or a list
        references = [references]
    if normalize:
        prediction = normalize_answer(prediction)
        references = [normalize_answer(ref) for ref in references]
    return int(any(prediction == ref for ref in references))


def dataset_em(predictions, references, normalize=False):
    """Average EM over a dataset. references[i] may be a string or a list."""
    if len(predictions) != len(references):
        raise ValueError("predictions and references must have equal length")
```

```
    if not predictions:
        raise ValueError("the dataset must not be empty")
    scores = [exact_match(p, r, normalize) for p, r in zip(predictions, references)]
    return sum(scores) / len(scores)
```

`any(...)` returns `True` if at least one reference matches; `int(True)` is 1. `dataset_em` keeps Group 1's two safety checks (equal lengths, non-empty data).

```
refs = ["Paris", "4", "Jupiter", "H2O", "Phnom Penh"]
preds = ["Paris", "4", "Jupiter", "Water", "Phnom Penh"]
print("Example 1 strict EM:", dataset_em(preds, refs))
print("Multiple references:", exact_match("four", ["4", "four"]))
print(repr(normalize_answer("ភ្នំពេញ។")))

pairs = [
    ("Paris", "paris"),
    ("Phnom Penh", "Phnom Penh!"),
    ("The United States", "United States"),
    ("Phnom Penh", "The answer is Phnom Penh"),
]
for ref, pred in pairs:
    print(
        f"{pred!r:28} raw={exact_match(pred, ref)} "
        f"normalized={exact_match(pred, ref, normalize=True)}"
    )
```

Executed output (file `code/ch01_exact_match.py`):

```
Example 1 strict EM: 0.8
Multiple references: 1
'ភ្នំពេញ។'
'paris'                      raw=0 normalized=1
'Phnom Penh!'                raw=0 normalized=1
'United States'              raw=0 normalized=1
'The answer is Phnom Penh'   raw=0 normalized=0
```

The last line confirms that the Khmer full stop survives English-style cleanup. **Configuration choices to report:** raw or normalized; the exact cleanup steps; how multiple references are handled; and how the answer is extracted from a longer model response.

1.G How to interpret results

- **High EM** (for example 90%) supports the claim that the model usually produces the *exact expected form* of the answer under the stated cleanup rules.
- **Low EM** may mean wrong answers, *or* correct answers in a different format ("Four", "The answer is..."). Look at the failed items before concluding that the model knows less.
- EM does **not** show *how wrong* an answer is. `Phnom Pen` (one missing letter) and `Bangkok` both score 0. For partial credit, see token-level F1 (Chapter 3) or CER (Chapter 9).
- EM scores are comparable only when the test questions, references and cleanup rules are the same.

1.H Strengths and limitations

Strengths	Limitations (with examples)
Simple and objective: every answer is 0 or 1	Penalizes correct alternative wording: `Four` vs `4` (Example 1.3)
Reproducible when the rules are stated	Sensitive to format: `Phnom Penh!` scores 0 without cleanup (Example 1.2)
Very fast, even on large datasets	No partial credit: a near miss equals a wild guess
Good for short, factual answers with one form	Unsuitable for open-ended text such as summaries or explanations
	English-oriented cleanup rules do not suit Khmer (1.F output)

1.I When to use it

Use EM for short answers that have one expected form or a small list of accepted forms: extractive question answering, arithmetic answers, entity names, codes, dates, and multiple-choice letters (where EM on the letter is the same as accuracy). Combine it with token-level F1 for partial credit, and with human review when correct answers can be phrased in many ways. Ask the model for a fixed answer format so that EM measures knowledge rather than formatting habits.

1.J Common misunderstandings

- *"EM checks whether the answer is correct."* → **Correction:** EM checks whether the *text* matches a reference. A correct answer in different words scores 0, and a wrong reference makes a correct answer fail.
- *"Normalization makes EM understand meaning."* → **Correction:** normalization removes only small surface differences such as capital letters and punctuation. `Four` and `4` remain different.
- *"EM and accuracy are the same thing."* → **Correction:** they share the same counting formula (correct ÷ total), but EM's judging rule is string equality, while accuracy uses whatever correctness rule the task defines (Chapter 2).

1.K Quick recap and practice

Recap. EM gives 1 when the prediction exactly equals an accepted reference and 0 otherwise; the dataset score is the average. Normalization rules and multiple references change the result, so they must be reported. EM is strict, simple and best for short answers.

Practice questions

1. A model answers 20 questions and 15 answers exactly match. What is the EM score as a ratio and as a percentage?
2. Reference `The Mekong`, prediction `mekong`. What are raw EM and SQuAD-style normalized EM?
3. Why can a model with lower EM still be the more knowledgeable model?

Answers

1. $15/20 = 0.75 = 75\%$.
2. Raw EM = 0 (different strings). Normalized: `The Mekong` → lowercase `the mekong` → remove article → `mekong`; prediction → `mekong`. Normalized EM = 1.
3. Because it may give correct answers in a different format (extra words, number words, synonyms). EM counts format agreement, so the failed items must be inspected.

1.L Sources

- Group 1 presentation: `Group1_Exact_Match_EM_Evaluation_for_Large_Language_Models_no_gamma.pptx` (definition, formula, five-question example, normalization table, Python code; the experiment slide is labelled “Illustrative predictions, not results from actual API calls”).
- Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. *EMNLP*. https://aclanthology.org/D16-1264/
- SQuAD v1.1 scoring code (normalization and EM), reproduced in Hugging Face *evaluate*: https://github.com/huggingface/evaluate/blob/main/metrics/squad/compute_score.py
- Python documentation, `string.punctuation`: https://docs.python.org/3/library/string.html

Chapter 2: Accuracy (Group 2)

2.A Learning goals

By the end of this chapter you should be able to:

- calculate accuracy as correct predictions divided by all predictions;
- write accuracy using a binary confusion matrix (TP, TN, FP, FN);
- explain why accuracy can mislead on imbalanced data and compare it with a majority-class baseline;
- explain why different errors can have different costs; and
- explain that "correct" is defined by the task's judging rule, not necessarily by identical wording.

2.B Plain-English introduction

Question answered: "Out of all the items, what share did the model get right?" Group 2 put it as: "Out of all the answers the LLM gave, how many were … correct?"

What it measures: the overall proportion of correct decisions. It fits classification tasks (choosing a label) and multiple-choice questions.

What it does not measure: *which kind* of mistakes are made, how serious they are, or how well the model handles rare classes. Two models can have the same accuracy while making very different mistakes.

2.C Key terms

- **Class / label:** one allowed category, such as `Positive` or `Negative`.
- **Binary classification:** a task with exactly two classes.
- **Positive class:** the class we choose to treat as "the thing we are looking for" (for example, `Positive` sentiment, or "spam"). The choice is ours and must be stated.
- **Confusion matrix:** a table that counts, for every pair (actual class, predicted class), how many items fell there. For two classes it has four cells:
  - **TP (true positive):** actually positive, predicted positive.
  - **TN (true negative):** actually negative, predicted negative.
  - **FP (false positive):** actually negative, predicted positive (a false alarm).
  - **FN (false negative):** actually positive, predicted negative (a missed case).
- **Class imbalance:** one class is much more common than another, for example 95 negative items and 5 positive items.
- **Majority-class baseline:** the accuracy of a "model" that always predicts the most common class. Any useful model should beat it.

2.D Formula and reasoning

General form. Let N be the number of items, $\hat{y}_i$ the prediction for item i, and y_i the correct label:

$$\text{Accuracy} = \frac{\text{number of correct predictions}}{\text{total number of predictions}} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}\left[\hat{y}_i \text{ is judged correct for } y_i\right]$$

The numerator counts correct predictions; the denominator counts all predictions. The result is a proportion.

Binary confusion-matrix form (as on Group 2's theory slide):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

The numerator holds the two kinds of *correct* decisions (the diagonal of the confusion matrix). The denominator holds all four cells, which together equal N.

Range and direction. From 0 to 1 (0% to 100%). **Higher is better.**

"Judged correct" depends on the task. For a fixed label set, correct usually means "same label". For free-text answers, the task may define correctness by a rule (for example, a numeric answer within a tolerance, or a human judge accepting a paraphrase). Accuracy therefore does **not** inherently require identical wording; that requirement belongs to Exact Match (Chapter 1). Group 10 made the same distinction: "EM is string-based; accuracy relies on the task's definition of a correct prediction."

Edge cases. If $N = 0$ accuracy is undefined. Items for which the model gives no usable answer (for example, it refuses or writes an unreadable choice) need a stated rule; counting them as wrong is common, but the rule must be reported.

2.E Worked examples

Example 2.1 — 100 questions (from Group 2's theory slide; illustrative)

An LLM answers 100 questions; 90 are correct and 10 incorrect.

$$\text{Accuracy} = \frac{90}{100} = 0.90 = 90\%$$

Example 2.2 — Sentiment and multiple choice (from Group 2's practical-examples slide)

Table 2.1 — Group 2's sentiment example.

Review	True label	Model guess	Result
"Best movie I've seen all year!"	Positive	Positive	correct
"Boring plot and terrible acting."	Negative	Negative	correct
"I fell asleep after ten minutes."	Negative	Negative	correct
"Not bad, actually pretty fun."	Positive	Negative	wrong

Accuracy $= 3/4 = 0.75 = 75\%$. Group 2 noted this beats the 50% that random guessing would give on average with two balanced classes. Their multiple-choice example (four questions, one wrong: "Author of *1984*?" answered Huxley instead of Orwell) also gives $3/4 = 75\%$, compared with 25% for random guessing among four options.

Example 2.3 — When accuracy misleads (teaching example; code verification)

A test set has 100 messages: 95 normal (negative) and 5 harmful (positive). A lazy model always predicts "normal".

Table 2.2 — Confusion matrix of the "always negative" model.

	Predicted positive	Predicted negative
Actually positive (5)	TP = 0	FN = 5
Actually negative (95)	FP = 0	TN = 95

$$\text{Accuracy} = \frac{0 + 95}{0 + 95 + 0 + 5} = \frac{95}{100} = 0.95 = 95\%$$

The model looks excellent, yet it never finds a single harmful message. Its accuracy equals the **majority-class baseline** (95%), so it has learned nothing useful. *Balanced accuracy* (the average of the recall for each class; Chapter 3 explains recall) exposes this: $(0/5 + 95/95)/2 = 0.5$, the same as guessing.

Error costs. Even when two models have equal accuracy, their mistakes can matter differently. Missing a harmful message (FN) may be far worse than flagging a normal one (FP). Accuracy treats every mistake as costing the same.

2.F Python implementation

Packages: `scikit-learn` (`pip install scikit-learn`). **Input format:** two lists of equal length containing labels of the same type (all strings or all integers).

Group 2's slide showed two snippets. The second used names `y_true` and `y_pred` that were never defined (it defined `refs` and `preds`), so it would stop with a `NameError`; its comment `# Output: 66.6%` also differs from what Python prints. The corrected and executed version is:

```
from sklearn.metrics import accuracy_score, balanced_accuracy_score


def calculate_accuracy(predictions, references):
    """Correct predictions divided by all predictions."""
    if len(predictions) != len(references) or not references:
        raise ValueError("need two non-empty lists of equal length")
    correct = sum(p == r for p, r in zip(predictions, references))
    return correct / len(references)


refs = ["Positive", "Negative", "Positive"]
preds = ["Positive", "Positive", "Positive"]
print("Own function :", calculate_accuracy(preds, refs))
print("scikit-learn :", accuracy_score(refs, preds))  # (y_true, y_pred)
print(f"As a percentage: {accuracy_score(refs, preds):.1%}")

# Recount of Group 2's reported experiment table (Example 2.4).
gold = ["Positive", "Negative", "Positive", "Positive", "Negative"]
gemini = ["Positive", "Negative", "Positive", "Positive", "Positive"]
llama = ["Positive", "Negative", "Negative", "Positive", "Negative"]
print(
    "Group 2 table - Gemini 1.5:",
    accuracy_score(gold, gemini),
    "Llama 3:",
    accuracy_score(gold, llama),
)
```

In `sum(p == r ...)`, each comparison gives `True` (counted as 1) or `False` (0). Note the argument order of `accuracy_score(y_true, y_pred)`: true labels first.

```
y_true = [0] * 95 + [1] * 5  # 95 negative, 5 positive
always_negative = [0] * 100  # majority-class baseline
print("Majority baseline accuracy:", accuracy_score(y_true, always_negative))
print("Balanced accuracy        :", balanced_accuracy_score(y_true, always_negative))
```

Executed output (file `code/ch02_accuracy.py`):

```
Own function : 0.6666666666666666
scikit-learn : 0.6666666666666666
As a percentage: 66.7%
Group 2 table - Gemini 1.5: 0.8 Llama 3: 0.8
Majority baseline accuracy: 0.95
Balanced accuracy        : 0.5
```

$2/3 = 0.6667$, which rounds to 66.7% (not 66.6%). **Configuration choices to report:** the label set, how unusable answers are counted, how free-text answers are mapped to labels, and the class balance of the test set.

2.G How to interpret results

- Always compare accuracy with the **majority-class baseline** and, for multiple choice, with random guessing ($1/k$ for k options).
- **High accuracy** supports the claim that the model is usually right *on this test set*. It does not show that it handles the rare class, and it does not show *why* it is right.
- **Equal accuracy** does not mean equal behaviour: look at the confusion matrix (Example 2.4 below).
- With small test sets, one item changes accuracy a lot: with 5 items, each item is worth 20 percentage points.

2.H Strengths and limitations

Strengths	Limitations (with examples)
Very easy to explain and compute	Misleading under class imbalance: 95% for a useless model (Example 2.3)
One number summarizing overall correctness	Hides the *type* of error: FP and FN are mixed together (Example 2.4)
Works for any number of classes	Treats all errors as equally costly
Natural for multiple-choice benchmarks	Needs a clear correctness rule for free-text answers

2.I When to use it

Use accuracy for classification and multiple-choice tasks with **reasonably balanced classes** and **similar error costs**. Report it together with the confusion matrix, and add Precision, Recall and F1 (Chapter 3) or balanced accuracy when classes are imbalanced or when one kind of mistake is more serious.

2.J Common misunderstandings

- *"Accuracy means the answer text was perfectly identical."* → **Correction:** accuracy counts predictions judged correct by the task's rule. Identical text is the rule of Exact Match, not of accuracy in general.
- *"95% accuracy is always good."* → **Correction:** compare with the baseline. On a 95/5 data set, 95% can be achieved by always predicting the majority class.
- *"Two models with the same accuracy are equally good."* → **Correction:** they may make different mistakes (see the reported experiment below) with different costs.

Example 2.4 — Reported experiment by Group 2

Group 2's slides report an experiment in which two models labelled five movie reviews.

Table 2.3 — Group 2's reported results (recounted; the totals are arithmetically correct).

Review	Actual (as labelled by Group 2)	Gemini 1.5	Llama 3
"Loved it!"	Positive	Positive ✓	Positive ✓
"Terrible acting."	Negative	Negative ✓	Negative ✓
"It was okay."	Positive	Positive ✓	Negative ✗
"A masterpiece."	Positive	Positive ✓	Positive ✓
"Boring plot."	Negative	Positive ✗	Negative ✓
Accuracy		**4/5 = 80%**	**4/5 = 80%**

What is documented: model names "Gemini 1.5" and "Llama 3", five reviews, gold labels, predictions and accuracy. **What is missing:** the exact model variants and sizes (for example, which Gemini 1.5 or Llama 3 version), the prompt, the date, the decoding settings, and how free-text responses were mapped to labels. This guide did not re-run the experiment.

Interpretation. Both models score 80%, but their errors differ: Gemini 1.5 produced a false positive (called a negative review positive), while Llama 3 produced a false negative on "It was okay." That item is also debatable: many people would call "It was okay." neutral rather than positive, which shows how much the *reference labels* matter. With five items, the tie says very little about which model is better in general.

2.K Quick recap and practice

Recap. Accuracy = correct ÷ total = (TP + TN) ÷ (TP + TN + FP + FN) for two classes. It is simple and useful for balanced classification, but it hides error types and can look high on imbalanced data. Always compare with a baseline.

Practice questions

1. TP = 30, TN = 50, FP = 10, FN = 10. What is the accuracy?
2. A test set has 80% "No" items. Model X has 82% accuracy. Is that impressive?
3. Can a free-text answer be counted correct in an accuracy calculation even if it is not identical to the reference?

Answers

1. $(30 + 50)/(30 + 50 + 10 + 10) = 80/100 = 0.80 = 80\%$.
2. Not very: always answering "No" already gives 80%. Model X is only 2 percentage points above the majority baseline.
3. Yes, if the task's judging rule accepts it (for example, a human judge or a rule that extracts and checks the final answer). Accuracy needs a correctness rule, not identical wording.

2.L Sources

- Group 2 presentation: `Group2_Presentation (1).pdf` (definition, confusion-matrix formula, sentiment and multiple-choice examples, Python code, Gemini 1.5 / Llama 3 experiment).
- scikit-learn User Guide, “Accuracy score” and “Balanced accuracy score”: https://scikit-learn.org/stable/modules/model_evaluation.html
- scikit-learn API, `accuracy_score`: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*, Section 8.3 (why accuracy is a poor measure when classes are skewed). https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-of-unranked-retrieval-sets-1.html

Chapter 3: Precision, Recall, and F1-score (Group 3)

3.A Learning goals

By the end of this chapter you should be able to:

- choose and state a positive class, and fill in TP, TN, FP and FN;
- calculate Precision, Recall and F1 and explain what each one answers;
- explain the harmonic mean and why F1 is not an ordinary average;
- handle undefined cases (division by zero) with a stated rule;
- explain binary, macro, micro and weighted averaging for more than two classes; and
- avoid the mistake of reading higher Precision as “fewer false positives in absolute number”.

3.B Plain-English introduction

Accuracy (Chapter 2) gives one overall share of correct decisions. It does not say *what kind* of mistakes happen. Precision and Recall split the picture into two questions about one chosen class, the **positive class**:

- **Precision:** “When the model says *positive*, how often is it right?” It is about the trustworthiness of positive predictions (avoiding false alarms).
- **Recall:** “Of all the items that really are positive, how many did the model find?” It is about coverage (avoiding missed cases).
- **F1-score:** one number that is high only when *both* Precision and Recall are high.

What they measure: error trade-offs for one class (or averaged over classes). **What they do not measure:** language quality, reasoning, or factual reliability. Group 3 put it well: “A high F1-score does not mean an LLM is generally intelligent or factually reliable.”

3.C Key terms

Group 3’s running example classifies customer reviews; **Positive** sentiment is the designated positive class.

Table 3.1 — The binary confusion matrix (layout from Group 3’s slide 5).

	Predicted Positive	**Predicted Negative**
Actually Positive	**TP** (true positive): correctly found	**FN** (false negative): missed positive
Actually Negative	**FP** (false positive): false alarm	**TN** (true negative): correctly rejected

Small example: the review “Great product!” is actually Positive. If the model says Positive, that is a TP; if it says Negative, that is an FN.

- **Positive class:** the class the scores are about. It is chosen by the analyst. In spam filtering, “spam” is usually positive; in disease screening, “has the disease” is usually positive. Changing the positive class changes Precision, Recall and F1.
- **Support:** the number of items that truly belong to a class.
- **Harmonic mean:** a kind of average of rates that is pulled strongly towards the smaller value (explained in 3.D).

3.D Formula and reasoning

Precision

$$P = \frac{TP}{TP + FP}$$

The denominator $TP + FP$ is **every item the model predicted positive**. The numerator is the part of those that were truly positive. So Precision is the share of positive predictions that were correct. Only FP (not FN) can lower it.

Recall (also called *sensitivity* or *true positive rate*)

$$R = \frac{TP}{TP + FN}$$

The denominator $TP + FN$ is **every item that is truly positive** (the support). The numerator is the part the model found. Only FN (not FP) can lower it.

F1-score

$$F_1 = \frac{2PR}{P + R} = \frac{2\,TP}{2\,TP + FP + FN}$$

The second form follows by substituting the definitions of P and R; it is convenient because it avoids rounding. TN does not appear anywhere in Precision, Recall or F1.

Why the harmonic mean? The harmonic mean of two numbers a and b is $\frac{2}{1/a + 1/b} = \frac{2ab}{a+b}$. Taking reciprocals ($1/a$) makes a *small* value produce a *large* reciprocal, which dominates the sum. So the result stays close to the smaller of the two numbers. With $P = 0.80$ and $R = 0.40$:

- ordinary (arithmetic) mean: $(0.80 + 0.40)/2 = 0.60$;
- harmonic mean (F1): $2 \times 0.80 \times 0.40 / 1.20 = 0.533$.

A model cannot hide poor Recall behind excellent Precision. In the extreme, a model that predicts "positive" for everything has $R = 1$ but low P; its arithmetic mean could look acceptable, but its F1 stays low. (A weighted version, $F_\beta = (1 + \beta^2)PR/(\beta^2 P + R)$, gives Recall β times as much importance; F_1 is the case $\beta = 1$.)

Range and direction. Each score lies between 0 and 1. **Higher is better.**

Undefined cases. If the model never predicts positive, then $TP + FP = 0$ and Precision is $0/0$, which is undefined. If the data contain no positive items, Recall is $0/0$. Libraries need a rule. scikit-learn uses the parameter `zero_division`: by default it returns 0 and shows a warning, or you can set it to 0, 1 or `np.nan` explicitly (scikit-learn `precision_recall_fscore_support`). This guide uses `zero_division=0` and states it.

More than two classes: averaging. With classes such as Positive, Negative and Neutral, we compute P, R and F1 *for each class* (treating that class as positive and all others as negative) and then combine them:

Table 3.2 — Averaging options (scikit-learn names).

Averaging	How it combines	Effect
binary	report only the chosen positive class	for two-class tasks
macro	plain mean of the per-class scores	every class counts equally, so rare classes matter a lot
micro	add up TP, FP, FN over all classes first, then compute	every *item* counts equally; for single-label multiclass tasks micro-P = micro-R = micro-F1 = accuracy
weighted	mean of per-class scores weighted by support	large classes dominate

Token-level F1 for short answers. Question-answering benchmarks such as SQuAD also compute F1 over *words*: Precision = shared words ÷ words in the prediction, Recall = shared words ÷ words in the reference (Rajpurkar et al., 2016). This gives partial credit where Exact Match gives 0.

3.E Worked examples

Example 3.1 — Group 3's Precision, Recall and F1 slides

Group 3 used TP = 8, FP = 2, FN = 12.

- Precision $= 8/(8+2) = 8/10 = 0.80$: 80% of positive predictions were correct.
- Recall $= 8/(8+12) = 8/20 = 0.40$: only 40% of the actual positives were found.
- F1 $= 2 \times 0.80 \times 0.40/(0.80+0.40) = 0.64/1.20 = 0.533$. Check with the count form: $2 \times 8/(2 \times 8 + 2 + 12) = 16/30 = 0.533$.

The low Recall pulls F1 down, exactly as Group 3 explained.

Example 3.2 — Spam and medical screening (Group 3's slide 10; illustrative)

Table 3.3 — Group 3's two applications (all values re-checked).

Case	TP	FP	FN	TN	Precision	Recall	F1
Spam detection	20	5	10	65	20/25 = 0.800	20/30 = 0.667	40/55 = 0.727
Medical screening	90	20	10	80	90/110 = 0.818	90/100 = 0.900	180/210 = 0.857

In spam filtering a false positive (a real email sent to the spam folder) annoys users, so Precision matters. In screening, a false negative (a sick person missed) is dangerous, so Recall matters more.

Example 3.3 — Higher Precision does not always mean fewer false positives (teaching example)

Model	TP	FP	Precision
A	8	2	8/10 = 0.80
B	90	10	90/100 = 0.90

Model B has the *higher* Precision but *more* false positives (10 versus 2), because it makes many more positive predictions. Precision is a **ratio**: it tells you what *share* of positive predictions are false alarms (20% for A, 10% for B), not how many false alarms there are. Group 3's sentence "High Precision

means fewer false positive predictions" is true only in this relative sense, or when the number of positive predictions is fixed. The same caution applies to Recall and false negatives.

Example 3.4 — Three classes and four averages (teaching example; code verification)

Ten reviews have gold labels: 6 pos, 3 neg, 1 neu. The model predicts:

Item	1–5	6	7–8	9	10
Gold	pos	pos	neg	neg	neu
Predicted	pos	neg	neg	pos	pos

Per-class counts (each class in turn treated as positive):

Table 3.4 — Per-class scores for Example 3.4.

Class	TP	FP	FN	Precision	Recall	F1	Support
pos	5	2 (items 9, 10)	1 (item 6)	5/7 = 0.7143	5/6 = 0.8333	10/13 = 0.7692	6
neg	2	1 (item 6)	1 (item 9)	2/3 = 0.6667	2/3 = 0.6667	0.6667	3
neu	0	0	1 (item 10)	0/0 → 0 (rule)	0/1 = 0	0	1

- **Macro F1** $= (0.7692 + 0.6667 + 0)/3 = 0.4786$.
- **Micro F1:** total TP $= 5 + 2 + 0 = 7$, total FP $= 3$, total FN $= 3$, so micro-P $=$ micro-R $= 7/10 = 0.70$ and micro-F1 $= 0.70$, equal to accuracy (7 of 10 correct).
- **Weighted F1** $= (0.7692 \times 6 + 0.6667 \times 3 + 0 \times 1)/10 = 0.6615$.

The rare class *neu* is completely missed. Macro F1 (0.48) reveals this; micro F1 (0.70) hides it. Neutral's Precision is $0/0$, so the result depends on the stated `zero_division` rule.

3.F Python implementation

Packages: `scikit-learn`. **Input format:** two lists of labels, `y_true` first and `y_pred` second. For binary tasks scikit-learn assumes the positive class is `1` unless you set `pos_label`.

```
import re
from collections import Counter

from sklearn.metrics import (
    confusion_matrix,
    f1_score,
    precision_recall_fscore_support,
    precision_score,
    recall_score,
)

# Example from Group 3's code slide: 1 = Positive (the positive class).
y_true = [1, 1, 1, 1, 0, 0, 0, 0, 0, 0]
y_pred = [1, 1, 1, 0, 1, 0, 0, 0, 0, 0]
print("Precision:", precision_score(y_true, y_pred))  # pos_label=1 by default
print("Recall   :", recall_score(y_true, y_pred))
print("F1       :", f1_score(y_true, y_pred))
# labels=[0, 1] fixes the row/column order: rows = actual, columns = predicted.
print(confusion_matrix(y_true, y_pred, labels=[0, 1]))
```

Here TP = 3 (items 1–3), FN = 1 (item 4), FP = 1 (item 5), TN = 5, so all three scores are $3/4 = 0.75$. In scikit-learn's matrix the **first row is the negative class**, so it prints `[[TN FP] [FN TP]]` = `[[5 1] [1 3]]`. This order differs from Table 3.1, which is a common source of confusion.

```
# Undefined case: the model never predicts Positive, so TP + FP = 0.
never_positive = [0] * 10
print(
    "Precision, zero_division=0 :",
    precision_score(y_true, never_positive, zero_division=0),
)
```

```
# Multiclass teaching example (Example 3.4).
gold = ["pos", "pos", "pos", "pos", "pos", "pos", "neg", "neg", "neg", "neu"]
pred = ["pos", "pos", "pos", "pos", "pos", "neg", "neg", "neg", "pos", "pos"]
labels = ["pos", "neg", "neu"]
p, r, f, support = precision_recall_fscore_support(
    gold, pred, labels=labels, zero_division=0
)
for name, *vals in zip(labels, p, r, f, support):
    print(f"{name}: P={vals[0]:.4f} R={vals[1]:.4f} F1={vals[2]:.4f} support={vals[3]}")
for avg in ["macro", "micro", "weighted"]:
    print(
        avg, round(f1_score(gold, pred, labels=labels, average=avg, zero_division=0), 4)
    )
```

```
def token_f1(prediction, reference):
    """SQuAD-style word-overlap F1 (lowercased, punctuation removed)."""
    pred = re.findall(r"\w+", prediction.lower())
    ref = re.findall(r"\w+", reference.lower())
    common = sum((Counter(pred) & Counter(ref)).values())  # shared words
    if common == 0:
        return 0.0
    precision, recall = common / len(pred), common / len(ref)
    return 2 * precision * recall / (precision + recall)


print("Token F1:", round(token_f1("Mekong River delta", "Mekong River"), 4))
```

`Counter(pred) & Counter(ref)` keeps each word the smaller number of times it appears in the two lists, so repeated words are not over-counted. This simplified version removes punctuation but not articles.

Executed output (file `code/ch03_precision_recall_f1.py`):

```
Precision: 0.75
Recall   : 0.75
F1       : 0.75
[[5 1]
 [1 3]]
Precision, zero_division=0 : 0.0
pos: P=0.7143 R=0.8333 F1=0.7692 support=6
neg: P=0.6667 R=0.6667 F1=0.6667 support=3
neu: P=0.0000 R=0.0000 F1=0.0000 support=1
macro 0.4786
micro 0.7
weighted 0.6615
Token F1: 0.8
```

Token F1: shared words = 2 (`mekong`, `river`); Precision $= 2/3$, Recall $= 2/2$; F1 $= 2(2/3)(1)/(2/3 + 1) = 0.8$. **Configuration choices to report:** the positive class (`pos_label`), the averaging method, the `zero_division` rule, and the label list.

3.G How to interpret results

- **High Precision, low Recall:** the model is cautious; what it flags is usually right, but it misses many cases.
- **High Recall, low Precision:** the model flags generously; it finds most cases but raises many false alarms.
- **High F1:** both are reasonably high. F1 ignores TN, so it says nothing about how well negatives are handled.
- Always say **which class** is positive and **which average** was used; a "macro F1 of 0.48" and a "micro F1 of 0.70" can describe the same predictions (Example 3.4).

3.H Strengths and limitations

Strengths	Limitations (with examples)
Separate false alarms (Precision) from misses (Recall)	Depend on the chosen positive class
F1 summarizes both and resists imbalance better than accuracy	F1 ignores TN entirely (formula in 3.D)
Extend to many classes with averaging	Averaging choice can hide or reveal rare classes (Example 3.4)
Token-level F1 gives partial credit for short answers	Ratios, not counts: higher Precision can coexist with more FPs (Example 3.3)
	F1 weights P and R equally even when error costs differ (Example 3.2)
	Need reliable reference labels, like every metric here

3.I When to use it

Use Precision, Recall and F1 for classification tasks with imbalanced classes or unequal error costs: harmful-content detection, spam filtering, information extraction, and checking whether an LLM correctly flags items. Use token-level F1 together with EM for short-answer question answering. Report the confusion matrix too, and choose the metric to emphasize (Precision, Recall, or F_β) based on which mistake costs more.

3.J Common misunderstandings

- *"High Precision means fewer false positives."* → **Correction:** it means a smaller *proportion* of positive predictions are false. A model making many more positive predictions can have higher Precision and more false positives (Example 3.3).
- *"F1 is the average of Precision and Recall."* → **Correction:** it is the *harmonic* mean, which stays near the smaller value: P = 0.8, R = 0.4 gives F1 = 0.533, not 0.6.
- *"Micro and macro F1 are basically the same."* → **Correction:** micro F1 counts items, macro F1 counts classes; they can differ sharply when a rare class is handled badly.
- *"Precision is undefined, so the model scored 0."* → **Correction:** 0 is a *convention* chosen by the software or analyst; report the rule.

3.K Quick recap and practice

Recap. Precision = TP/(TP+FP) asks "are the positive predictions trustworthy?"; Recall = TP/(TP+FN) asks "are the positives found?"; F1 = 2PR/(P+R) balances them and is pulled towards the smaller value. For many classes, state the averaging method.

Practice questions

1. TP = 40, FP = 10, FN = 20, TN = 30 (Group 10's example). Compute P, R and F1.
2. A model predicts "positive" for every item in a data set with 10 positives and 90 negatives. What are its Precision and Recall?
3. Why can micro-F1 be much higher than macro-F1?

Answers

1. $P = 40/50 = 0.800$; $R = 40/60 = 0.667$; $F_1 = 80/(80 + 10 + 20) = 80/110 = 0.727$.
2. TP = 10, FP = 90, FN = 0: Precision $= 10/100 = 0.10$, Recall $= 10/10 = 1.00$ (F1 $= 20/110 = 0.18$).
3. Micro-F1 is dominated by common classes because every item counts once; if a rare class is handled badly, macro-F1 (which gives each class equal weight) drops much more.

Reported experiment. Group 3's code slide states that "the same functions are used to evaluate both LLMs", but the supplied PDF contains **no experiment results**: no model names, prompts, data or scores. The slide numbering jumps from 11 to 14, suggesting slides were removed or not exported. No results are reported here.

3.L Sources

- Group 3 presentation: `Group3_Precision_Recall_F1_Research_Presentation.pptx.pdf` (confusion matrix, formulas, examples, applications, scikit-learn code).
- scikit-learn User Guide, "Precision, recall and F-measures" and multiclass averaging: https://scikit-learn.org/stable/modules/model_evaluation.html#precision-recall-f-measure-metrics
- scikit-learn API, `precision_recall_fscore_support` (`average`, `pos_label`, `zero_division`): https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_recall_fscore_support.html
- Manning, Raghavan & Schütze (2008), *Introduction to Information Retrieval*, Section 8.3 (Precision, Recall, F measure and harmonic mean): https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-of-unranked-retrieval-sets-1.html
- Rajpurkar et al. (2016), SQuAD (token-level F1 for QA): https://aclanthology.org/D16-1264/

Chapter 4: BLEU (Group 4)

4.A Learning goals

By the end of this chapter you should be able to:

- list the n-grams of a sentence and compute *clipped* n-gram precision;
- compute the brevity penalty and combine everything into BLEU-2 and BLEU-4;
- explain why a logarithm and an exponential appear in the formula;
- explain what happens with zero matches and short sentences, and what smoothing does;
- distinguish corpus BLEU from the average of sentence BLEU scores; and
- report BLEU reproducibly (tokenization, case, references, software signature).

4.B Plain-English introduction

BLEU (Bilingual Evaluation Understudy) was proposed by Papineni et al. (2002) to evaluate machine translation automatically. **Question answered:** "How many of the short word sequences in the model output also appear in the reference translation(s), and is the output long enough?"

What it measures: surface overlap of words and short word sequences (n-grams) with one or more references, with a penalty for outputs that are too short. Longer matching sequences reward matching local word order.

What it does not measure: meaning, factual correctness, grammar as such, or usefulness. A correct paraphrase with different words can score low, and a fluent sentence with one critical wrong word can score high. Group 4's summary is accurate: "BLEU evaluates text overlap—not understanding."

4.C Key terms

- **Candidate:** the model output being scored (also called *hypothesis*).
- **n-gram:** a run of n consecutive tokens. For `the cat is sleeping` (Group 4's slide 4):

Table 4.1 — n-grams of a four-word sentence. A sentence of L tokens has $L - n + 1$ n-grams of length n.

n	n-grams	count
1 (unigram)	the · cat · is · sleeping	4
2 (bigram)	the cat · cat is · is sleeping	3
3 (trigram)	the cat is · cat is sleeping	2
4 (4-gram)	the cat is sleeping	1

- **Precision (here):** the share of the candidate's n-grams that also appear in the reference. BLEU is precision-based; it does not directly measure Recall (how much of the reference is covered). The brevity penalty partly takes Recall's place.
- **Clipping:** a candidate n-gram is credited at most as many times as it appears in a reference, so repeating a correct word cannot inflate the score.
- **Geometric mean:** the k-th root of the product of k numbers, for example $\sqrt{a \times b}$ for two numbers. It becomes 0 if any number is 0.
- **Logarithm:** $\log x$ answers "to what power must $e \approx 2.718$ be raised to get x?" Key facts: $\log(a \times b) = \log a + \log b$; $\exp(\log x) = x$; $\log x$ is negative when $0 < x < 1$; and $\log 0$ is undefined (it goes to $-\infty$).

4.D Formula and reasoning

Step 1 — clipped (modified) n-gram precision. For each n-gram g in the candidate, $\mathrm{Count}(g)$ is how often it appears in the candidate and $\mathrm{MaxRef}(g)$ is the largest number of times it appears in any single reference.

$$p_n = \frac{\sum_{g \in \text{candidate } n\text{-grams}} \min\left(\mathrm{Count}(g),\ \mathrm{MaxRef}(g)\right)}{\sum_{g \in \text{candidate } n\text{-grams}} \mathrm{Count}(g)}$$

The numerator counts *credited* matches; the denominator counts *all* candidate n-grams. For a whole test set (corpus), both sums run over all candidate sentences before dividing.

Step 2 — brevity penalty (BP). Precision alone would reward very short outputs (a one-word output can have perfect precision). Let c be the candidate length and r the *effective reference length* (for each sentence, the reference length closest to the candidate length; summed over the corpus):

$$\mathrm{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{1-r/c} & \text{if } c \le r \end{cases}$$

If the candidate is shorter than the reference, $r/c > 1$, the exponent is negative and BP is below 1. There is no extra penalty for being *longer*, because longer outputs are already punished by lower precision. When $c = r$, $e^0 = 1$. (NLTK and sacreBLEU both choose the shorter reference when two are equally close.)

Step 3 — combine. With weights w_n that add up to 1 (usually $w_n = 1/N$):

$$\mathrm{BLEU} = \mathrm{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

Why log and exp? The quantity inside is a weighted *geometric mean*:

$$\exp\left(\sum_n w_n \log p_n\right) = \prod_n p_n^{w_n} \qquad \text{e.g. } \exp\left(\tfrac{1}{4}\sum_{n=1}^{4} \log p_n\right) = \sqrt[4]{p_1 p_2 p_3 p_4}$$

Taking logs turns the product into a sum (easy to weight), and $\exp$ turns it back. Papineni et al. chose the geometric mean because n-gram precisions fall roughly exponentially as n grows, and an ordinary average would be dominated by p_1. The price is harshness: if any $p_n = 0$, then $\log p_n$ is undefined and BLEU is 0.

BLEU-2 versus BLEU-4. "BLEU-N" means the maximum n-gram order is N. Standard BLEU is **BLEU-4** ($N = 4$, $w_n = 1/4$), as in the original paper and in sacreBLEU's default. BLEU-2 uses $N = 2$, $w_n = 1/2$; it is easier to calculate by hand and less likely to be 0 for short sentences, but it is not the standard score.

Range and direction. 0 to 1, often printed as 0 to 100 (sacreBLEU prints 0–100). **Higher means more n-gram overlap with the references.** It is *not* a linear scale of human quality: going from 20 to 25 is not "25% better translation", and even good human translations rarely score 100 because they word things differently from the references (Papineni et al., 2002).

Edge cases and smoothing.

- *Zero matches at some order.* A short sentence often has no matching 3-gram or 4-gram, which makes unsmoothed BLEU exactly 0 even when many words match.
- *Sentence too short.* A 3-word candidate has no 4-grams at all.
- *Smoothing* adds a small value to zero counts so that sentence-level BLEU is not 0. Several methods exist (Chen & Cherry, 2014), and they give different numbers (Example 4.3). sacreBLEU's `sentence_bleu` uses the `exp` method by default and also uses *effective order* (ignoring n-gram orders

longer than the sentence).

- *Corpus BLEU versus averaged sentence BLEU.* BLEU was designed at corpus level: add all clipped counts, all n-gram totals and all lengths over the whole test set, then compute one score. Averaging separate sentence scores is a different statistic and gives a different number (Example 4.4). sacreBLEU's own documentation warns that sentence-level BLEU is "not its intended use".

Implementation choices that change the score (Post, 2018): the tokenizer (sacreBLEU default `13a`; there are also `intl`, `char`, `zh`, `ja-mecab`, `ko-mecab`, `flores101`, `flores200` and others), case sensitivity (default: case-sensitive), the number of references, smoothing, and the software version. Post found that commonly used configurations can differ by as much as 1.8 BLEU points on the same outputs, which is as large as many reported improvements, and introduced a *signature* string that records all of these settings.

4.E Worked examples

Example 4.1 — BLEU-2 and BLEU-4 step by step (Group 4's slide 6, extended)

Reference: `the cat is on the mat` (6 tokens). Candidate: `the cat is on mat` (5 tokens).

Table 4.2 — n-gram matches for Example 4.1.

n	Candidate n-grams	Matched in reference?	p_n
1	the, cat, is, on, mat	all 5	5/5 = 1.000
2	the cat, cat is, is on, on mat	first 3 (`on mat` is not in the reference)	3/4 = 0.750
3	the cat is, cat is on, is on mat	first 2	2/3 = 0.667
4	the cat is on, cat is on mat	first 1	1/2 = 0.500

Brevity penalty: $c = 5$, $r = 6$, $c \le r$, so $\mathrm{BP} = e^{1 - 6/5} = e^{-0.2} = 0.8187$.

BLEU-2 (Group 4's calculation):

$$\text{BLEU-2} = 0.8187 \times \exp\left(\tfrac{1}{2}\log 1 + \tfrac{1}{2}\log 0.75\right) = 0.8187 \times \sqrt{1 \times 0.75} = 0.8187 \times 0.8660 = 0.7090$$

BLEU-4:

$$\sqrt[4]{1 \times 0.75 \times 0.6667 \times 0.5} = \sqrt[4]{0.25} = 0.7071, \qquad \text{BLEU-4} = 0.8187 \times 0.7071 = 0.5789$$

So the same pair scores 70.9 as BLEU-2 but 57.9 as standard BLEU-4. Both numbers were confirmed by the code in 4.F (sacreBLEU prints `57.89`). Always say which one you report.

Example 4.2 — Why clipping is needed (teaching example, after Papineni et al.)

Reference: `the cat is on the mat`. Candidate: `the the the the the the the` (7 tokens).

Without clipping, every `the` appears in the reference, so unigram precision would be $7/7 = 1$, a perfect score for nonsense. With clipping, `the` appears at most **2** times in the reference, so only 2 of the 7 are credited: $p_1 = 2/7 = 0.2857$.

Example 4.3 — A paraphrase and the smoothing problem (Group 4's slide 7 idea, computed)

Reference: `the cat is sleeping`. Candidate: `the feline is asleep` (same meaning).

- Unigrams: `the` and `is` match: $p_1 = 2/4 = 0.5$.
- Bigrams: `the feline`, `feline is`, `is asleep` — none match: $p_2 = 0/3$.
- Since $p_2 = 0$, unsmoothed BLEU-4 is **0**.

Group 4 described this example only qualitatively. Computed with three common tools (verified in 4.F):

Table 4.3 — The same sentence pair under different smoothing rules.

Tool and setting	Score
Unsmoothed BLEU-4 (own code; NLTK without smoothing, which also warns)	0
NLTK `SmoothingFunction().method1` (adds 0.1 to zero counts)	0.0955
sacreBLEU `sentence_bleu` (default `exp` smoothing)	19.0 (= 0.190)

This exposes two limitations at once: BLEU does not recognize a correct paraphrase, and sentence-level numbers depend heavily on the smoothing rule. Scores from different tools must not be compared.

Example 4.4 — Corpus BLEU versus averaged sentence BLEU (teaching example)

#	Candidate	Reference	Sentence BLEU (sacreBLEU)
1	the cat is on the mat	the cat is on the mat	100.00
2	a dog runs	the dog is running in the park	7.25

Average of sentence scores: $(100.00 + 7.25)/2 = 53.63$.

Corpus BLEU pools the counts first: unigrams $(6+1)/(6+3) = 7/9$; bigrams $(5+0)/(5+2) = 5/7$; trigrams $(4+0)/(4+1) = 4/5$; 4-grams $(3+0)/(3+0) = 3/3$. Lengths: $c = 6+3 = 9$, $r = 6+7 = 13$, so $\mathrm{BP} = e^{1-13/9} = 0.6412$.

$$\mathrm{BLEU} = 0.6412 \times \sqrt[4]{0.7778 \times 0.7143 \times 0.8 \times 1} = 0.6412 \times 0.8165 = 0.5235 \ (52.35)$$

The two summaries differ (52.35 versus 53.63) because pooling lets long sentences and the overall length ratio count properly. In real test sets with hundreds of sentences, the gap can be much larger.

4.F Python implementation

Packages: `sacrebleu` and `nltk` (`pip install sacrebleu nltk`). **Input format:** sacreBLEU takes *untokenized strings* and tokenizes them itself; `corpus_score` takes a list of candidate strings and a *list of reference lists* (one inner list per reference set). NLTK takes *lists of tokens*.

First, a transparent version written from the formula (for learning; use sacreBLEU for reporting):

```
import math
from collections import Counter

import sacrebleu
from nltk.translate.bleu_score import SmoothingFunction, sentence_bleu


def ngrams(tokens, n):
    """All runs of n consecutive tokens, counted."""
    return Counter(tuple(tokens[i : i + n]) for i in range(len(tokens) - n + 1))


def simple_bleu(candidate, references, max_n=4):
    """Sentence BLEU with uniform weights and no smoothing (Papineni et al.)."""
    cand = candidate.split()
    refs = [r.split() for r in references]
    log_sum = 0.0
    for n in range(1, max_n + 1):
```

```
        cand_counts = ngrams(cand, n)
        # Clipping: a candidate n-gram earns credit at most as many times
        # as it appears in any single reference.
        max_ref = Counter()
        for ref in refs:
            for gram, count in ngrams(ref, n).items():
                max_ref[gram] = max(max_ref[gram], count)
        clipped = sum(min(c, max_ref[g]) for g, c in cand_counts.items())
        total = max(sum(cand_counts.values()), 1)
        print(f"  p{n} = {clipped}/{total}")
        if clipped == 0:
            return 0.0  # log(0) is undefined, so unsmoothed BLEU is 0
        log_sum += (1 / max_n) * math.log(clipped / total)
    c = len(cand)
    # Effective reference length: the reference length closest to c.
    r = min((abs(len(ref) - c), len(ref)) for ref in refs)[1]
    bp = 1.0 if c > r else math.exp(1 - r / c)
    print(f"  c={c}, r={r}, BP={bp:.4f}")
    return bp * math.exp(log_sum)
```

`ngrams` uses Python's `Counter` to count each n-gram tuple. `max_ref` stores, for each n-gram, the highest count in any single reference; this is the clipping limit. `log_sum` accumulates $\sum w_n \log p_n$, and the last line applies BP and $\exp$.

```
ref = "the cat is on the mat"
cand = "the cat is on mat"
print("BLEU-2 (own):", round(simple_bleu(cand, [ref], max_n=2), 4))
print("BLEU-4 (own):", round(simple_bleu(cand, [ref], max_n=4), 4))
print("BLEU-4 (sacreBLEU):", sacrebleu.sentence_bleu(cand, [ref]))

# Clipping example: a repeated word.
print(
    "Clipped unigram precision:",
    sentence_bleu([ref.split()], ["the"] * 7, weights=(1,)),
)
```

```
# A paraphrase with no matching bigram: three tools, three answers.
para = "the feline is asleep"
print("Own, unsmoothed :", simple_bleu(para, ["the cat is sleeping"]))
print(
    "NLTK method1    :",
    round(
        sentence_bleu(
            [["the", "cat", "is", "sleeping"]],
            para.split(),
            smoothing_function=SmoothingFunction().method1,
        ),
        4,
    ),
)
print(
    "sacreBLEU (exp) :",
    round(sacrebleu.sentence_bleu(para, ["the cat is sleeping"]).score, 2),
)

# Corpus BLEU versus the average of sentence BLEU scores.
hyps = ["the cat is on the mat", "a dog runs"]
refs = ["the cat is on the mat", "the dog is running in the park"]
bleu = sacrebleu.BLEU()
corpus = bleu.corpus_score(hyps, [refs])  # [refs] = one reference set
sentence_scores = [sacrebleu.sentence_bleu(h, [r]).score for h, r in zip(hyps, refs)]
print("Corpus BLEU:", round(corpus.score, 2))
print(
    "Sentence BLEU:",
```

```
    [round(s, 2) for s in sentence_scores],
    "average:",
    round(sum(sentence_scores) / 2, 2),
)
print("Signature:", bleu.get_signature())
```

For several references per sentence, pass several reference *sets*: `bleu.corpus_score(hyps, [refs_set1, refs_set2])`, where `refs_set1[i]` and `refs_set2[i]` are two references for `hyps[i]`.

Executed output (file `code/ch04_bleu.py`):

```
  p1 = 5/5
  p2 = 3/4
  c=5, r=6, BP=0.8187
BLEU-2 (own): 0.709
  p1 = 5/5
  p2 = 3/4
  p3 = 2/3
  p4 = 1/2
  c=5, r=6, BP=0.8187
BLEU-4 (own): 0.5789
BLEU-4 (sacreBLEU): BLEU = 57.89 100.0/75.0/66.7/50.0 (BP = 0.819 ratio = 0.833 hyp_len = 5 ref_len =
6)
Clipped unigram precision: 0.2857142857142857
  p1 = 2/4
  p2 = 0/3
Own, unsmoothed : 0.0
NLTK method1    : 0.0955
sacreBLEU (exp) : 19.0
Corpus BLEU: 52.35
Sentence BLEU: [100.0, 7.25] average: 53.63
Signature: nrefs:1|case:mixed|eff:no|tok:13a|smooth:exp|version:2.6.0
```

sacreBLEU's summary line shows the four precisions (100.0/75.0/66.7/50.0), BP, the length ratio $c/r = 0.833$, and both lengths, exactly matching Table 4.2. The **signature** records the settings: one reference, case-sensitive (`case:mixed`), no effective order, `13a` tokenizer, `exp` smoothing, version 2.6.0. Report it with every BLEU score.

4.G How to interpret results

- **Higher BLEU** on the *same test set with the same settings* means more n-gram overlap with the references. Across a large test set, higher BLEU has historically tended to agree with human rankings of translation systems, but the link is not guaranteed (Callison-Burch et al., 2006; Reiter, 2018).
- **A single sentence's BLEU** is unstable and depends on smoothing (Table 4.3).
- **BLEU does not establish** factual correctness, absence of harmful content, or fluency. One wrong number or a missing "not" changes only a few n-grams.
- **Never compare** BLEU across papers, groups or tools unless the signature (tokenizer, case, references, smoothing, version) and the test set match.

4.H Strengths and limitations

Strengths	Limitations (with examples)
Fast, cheap, automatic, language-independent arithmetic	Misses paraphrases and synonyms (Example 4.3)
Rewards local word order via longer n-grams	Sentence-level scores unstable; 0 without smoothing (Example 4.3)
Clipping stops repetition tricks (Example 4.2)	Sensitive to tokenization, case and number of references (Post, 2018)
Widely used, so results can be related to past work *when settings match*	Corpus and averaged sentence scores differ (Example 4.4)
Multiple references can capture some valid variation	Does not check meaning or facts; not a linear quality scale

4.I When to use it

Use corpus-level BLEU-4 to compare machine-translation systems (or other generation systems with close reference wording) on the **same test set**, with sacreBLEU and a reported signature. Add a meaning-based metric such as BERTScore (Chapter 7) or METEOR (Chapter 6), and human review of samples, before drawing conclusions about quality. For languages without spaces such as Khmer, choose and report a suitable tokenizer (for example sacreBLEU's SentencePiece-based `flores200`, or character-level scoring), because splitting on spaces does not give words.

4.J Common misunderstandings

- *"BLEU measures translation accuracy."* → **Correction:** it measures n-gram overlap with references. A correct paraphrase can score near 0 (Example 4.3).
- *"BLEU 40 is twice as good as BLEU 20."* → **Correction:** BLEU is not a linear scale of human quality.
- *"The average of sentence BLEU equals corpus BLEU."* → **Correction:** corpus BLEU pools counts first; the results differ (Example 4.4).
- *"BLEU is BLEU."* → **Correction:** BLEU-2 and BLEU-4, different tokenizers, and different smoothing rules give different numbers for the same texts (Examples 4.1 and 4.3).

4.K Quick recap and practice

Recap. BLEU multiplies a brevity penalty by the geometric mean of clipped n-gram precisions (usually $n = 1$–4). Logs and exponentials implement the geometric mean. It is a corpus-level overlap measure; sentence scores need smoothing; settings must be reported.

Practice questions

1. Candidate `a a a b` (4 tokens); reference `a b c d`. What is the clipped unigram precision?
2. Candidate length 8, reference length 10. What is BP?
3. Why is standard BLEU-4 often 0 for very short sentences without smoothing?

Answers

1. `a` appears 3 times in the candidate but once in the reference, so it gets 1 credit; `b` gets 1. $p_1 = 2/4 = 0.5$.
2. $c \le r$, so $\mathrm{BP} = e^{1 - 10/8} = e^{-0.25} = 0.7788$.
3. Short sentences often have no matching 3-gram or 4-gram (or none at all), so some $p_n = 0$, and the geometric mean of numbers containing 0 is 0.

Reported experiment. The supplied Group 4 slides contain no Python code and **no experiment with real LLM outputs**; slide 7 is labelled “qualitative illustrations, not computed BLEU scores.” Example 4.3 above computes one of those illustrations as a teaching example.

4.L Sources

- Group 4 presentation: `Group4_BLEU_Score_10_Slides (1).pptx` (motivation, n-grams, method, BLEU-2 worked example, interpretation, limitations; speaker notes state that standard BLEU uses $n = 1$–4 at corpus level).
- Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. *ACL*. https://aclanthology.org/P02-1040/
- Post, M. (2018). A Call for Clarity in Reporting BLEU Scores. *WMT*. https://aclanthology.org/W18-6319/
- Chen, B., & Cherry, C. (2014). A Systematic Comparison of Smoothing Techniques for Sentence-Level BLEU. *WMT*. https://aclanthology.org/W14-3346/
- Callison-Burch, C., Osborne, M., & Koehn, P. (2006). Re-evaluating the Role of BLEU in Machine Translation Research. *EACL*. https://aclanthology.org/E06-1032/
- Reiter, E. (2018). A Structured Review of the Validity of BLEU. *Computational Linguistics*, 44(3). https://aclanthology.org/J18-3002/
- sacreBLEU source and documentation: https://github.com/mjpost/sacrebleu
- NLTK `bleu_score` documentation: https://www.nltk.org/api/nltk.translate.bleu_score.html

Chapter 5: ROUGE (Group 5)

5.A Learning goals

By the end of this chapter you should be able to:

- explain why ROUGE was designed around Recall (coverage of the reference);
- compute ROUGE-1 and ROUGE-2 Precision, Recall and F1;
- find a longest common subsequence and compute ROUGE-L;
- explain how ROUGE-Lsum treats sentence boundaries differently from ROUGE-L;
- explain ROUGE-W, ROUGE-S and ROUGE-SU with small examples; and
- explain why ROUGE depends on references, settings and language-specific text preparation.

5.B Plain-English introduction

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is a family of measures introduced by Lin (2004) for evaluating summaries. **Question answered:** "How much of the reference summary's wording does the generated summary contain?"

Where BLEU (Chapter 4) starts from the candidate and asks how much of it is supported by the reference (Precision), ROUGE was originally built around **Recall**: how much of the reference is covered. Modern software usually reports Precision, Recall and F1 for every ROUGE variant.

What it measures: overlap of words, word pairs and in-order word sequences with one or more reference summaries. **What it does not measure:** factual correctness, logical consistency, readability, or whether different wording means the same thing (Group 5's limitation table lists all of these).

5.C Key terms

- **Reference summary:** a human-written summary used as the target.
- **Consecutive sequence (n-gram):** neighbouring words with no gaps: in `the cat sat`, `cat sat` is a 2-gram.
- **Subsequence:** words that keep their order but may have gaps between them: in `the cat sat on the mat`, `cat … mat` is a subsequence (length 2), but `mat … cat` is not.
- **Longest common subsequence (LCS):** the longest subsequence that appears in both texts. It rewards correct order without requiring the words to be adjacent.
- **Skip-bigram:** any ordered pair of words from a sentence, with any gap allowed: `the cat sat` has three: (the, cat), (the, sat), (cat, sat). A sentence of m distinct words has $\binom{m}{2} = m(m-1)/2$ skip-bigrams.
- **Stemming:** cutting words to a rough root form so that `running` and `runs` both become `run`. The common English tool is the Porter stemmer.

5.D Formula and reasoning

All variants share one pattern. Let X be the reference with m units and Y the candidate with n units (the *unit* changes by variant):

$$R = \frac{\text{matched units}}{\text{units in the reference}}, \qquad P = \frac{\text{matched units}}{\text{units in the candidate}}, \qquad F = \frac{(1+\beta^2)\,R\,P}{R + \beta^2 P}$$

With $\beta = 1$, F is the ordinary F1 $= 2PR/(P+R)$ (Chapter 3 explains the harmonic mean). Lin's paper allowed a large β to favour Recall; Google's widely used `rouge-score` Python package reports F1 ($\beta = 1$). Each score lies between 0 and 1, and **higher is better**.

ROUGE-N (consecutive n-grams). The unit is the n-gram; matches are clipped as in BLEU (a reference n-gram can be matched at most as many times as it occurs):

$$\text{ROUGE-N recall} = \frac{\sum_{g \in \text{ref } n\text{-grams}} \min\left(\text{Count}_{\text{cand}}(g), \text{Count}_{\text{ref}}(g)\right)}{\sum_{g \in \text{ref } n\text{-grams}} \text{Count}_{\text{ref}}(g)}$$

ROUGE-1 uses single words (content coverage); ROUGE-2 uses word pairs (some local fluency and order).

ROUGE-L (longest common subsequence). The number of matched units is $\text{LCS}(X, Y)$:

$$R_{\text{lcs}} = \frac{\text{LCS}(X,Y)}{m}, \qquad P_{\text{lcs}} = \frac{\text{LCS}(X,Y)}{n}$$

LCS is found with **dynamic programming** (filling a table of answers for every pair of prefixes and reusing them; Chapter 9 shows such a table for edit distance). ROUGE-L rewards correct order but ignores whether matches are adjacent.

ROUGE-Lsum (summary-level LCS). A summary often has several sentences. Lin defines a *union LCS*: for each reference sentence r_i, take the LCS of r_i with *each* candidate sentence, and count the **union** of the reference words matched by any of them. Then

$$R_{\text{lcs}} = \frac{\sum_i \left|\text{LCS}_{\cup}(r_i, C)\right|}{m}, \qquad P_{\text{lcs}} = \frac{\sum_i \left|\text{LCS}_{\cup}(r_i, C)\right|}{n}$$

where m and n are total words in the reference and candidate. This is different from joining the whole summary into one long sequence (plain ROUGE-L): it does not punish putting the sentences in a different order. In `rouge-score`, ROUGE-Lsum expects **sentences separated by newline characters** (`"\n"`); without newlines it behaves like ROUGE-L on one sentence. The option `split_summaries=True` splits sentences automatically.

ROUGE-W (weighted LCS). Two candidates can have the same LCS length while one matches in one continuous run and the other in scattered pieces. ROUGE-W gives a run of k consecutive matches a credit $f(k)$ that grows faster than k, such as $f(k) = k^2$. Lin's weighted LCS adds $f(k+1) - f(k)$ for each new consecutive match, then converts back with the inverse function:

$$R_{\text{wlcs}} = f^{-1}\left(\frac{\text{WLCS}(X,Y)}{f(m)}\right), \qquad P_{\text{wlcs}} = f^{-1}\left(\frac{\text{WLCS}(X,Y)}{f(n)}\right)$$

With $f(k) = k^2$, $f^{-1}(x) = \sqrt{x}$. (Lin's experiments also used $f(k) = k^{1.2}$; the weighting is a setting.)

ROUGE-S (skip-bigrams). The unit is the skip-bigram. With $\text{SKIP2}(X, Y)$ the number of shared skip-bigrams,

$$R_{\text{skip2}} = \frac{\text{SKIP2}(X,Y)}{\binom{m}{2}}, \qquad P_{\text{skip2}} = \frac{\text{SKIP2}(X,Y)}{\binom{n}{2}}$$

It rewards correct *order* of word pairs anywhere in the sentence. A maximum gap d_{skip} can be set (Lin reports ROUGE-S4, ROUGE-S9 and so on); an unlimited gap can reward far-apart, unrelated pairs.

ROUGE-SU (skip-bigrams plus unigrams). ROUGE-S gives 0 to a candidate that shares words but no ordered pair (for example, a reversed sentence). ROUGE-SU adds single words as units. Lin notes it can be obtained by adding a begin-of-sentence marker to both texts before counting skip-bigrams.

Settings that change scores: tokenization, lowercasing, stemming, stop-word removal, sentence splitting (for Lsum), β, the ROUGE-W weight, the skip distance, multiple-reference handling, and confidence-interval resampling. The original Perl package (ROUGE-1.5.5) and Python reimplementations differ in some defaults, so report the package and options.

5.E Worked examples

Example 5.1 — ROUGE-1 and ROUGE-2 (Group 5's slide, computed)

Reference: `the weather is cold outside` (5 words). Candidate: `the weather is cold` (4 words).

Table 5.1 — Example 5.1. F1 for ROUGE-1: $2(0.8)(1)/(1.8) = 0.889$.

Variant	Reference units	Candidate units	Matches	Recall	Precision	F1
ROUGE-1	5 words	4 words	4 (all but `outside`)	4/5 = 0.800	4/4 = 1.000	0.889
ROUGE-2	4 bigrams	3 bigrams	3 (`the weather`, `weather is`, `is cold`)	3/4 = 0.750	3/3 = 1.000	0.857

Group 5 gave this example without the numbers; the code in 5.F confirms them. Precision is perfect (nothing extra was written) but Recall is lower because `outside` is missing.

Example 5.2 — ROUGE-L with a gap (Group 5's slide)

Reference: `the cat sat on the mat`. Candidate: `the cat rested on the mat`. The LCS is `the cat · on the mat` (length 5): the words are in the same order, and `sat`/`rested` is skipped.

$$R = 5/6 = 0.833, \quad P = 5/6 = 0.833, \quad F_1 = 0.833$$

Compare ROUGE-2 for the same pair: shared bigrams are `the cat`, `on the`, `the mat` (3 of 5), so ROUGE-2 = 0.600. ROUGE-L is more forgiving because it allows gaps.

Example 5.3 — ROUGE-L versus ROUGE-Lsum (teaching example; code verification)

Reference (one sentence): `heavy rain closed the schools today` (6 words). Candidate (two sentences, separated by a newline): `the schools closed today` / `heavy rain fell` (7 words).

- **ROUGE-L** treats the candidate as one sequence: `the schools closed today heavy rain fell`. The longest in-order match with the reference is `the schools today` (3 words), because `heavy rain` comes *after* those words in the candidate but *before* them in the reference. $R = 3/6 = 0.500$, $P = 3/7 = 0.429$, $F_1 = 0.462$.
- **ROUGE-Lsum** takes the LCS of the reference with each candidate sentence separately: sentence 1 gives {the, schools, today}; sentence 2 gives {heavy, rain}. The union has 5 words. $R = 5/6 = 0.833$, $P = 5/7 = 0.714$, $F_1 = 0.769$.

The candidate covers most of the reference but in a different sentence order. Lsum does not penalize the order of sentences; plain ROUGE-L does. Which one is "right" depends on whether sentence order matters for the task — another reason to report the variant.

Example 5.4 — ROUGE-W rewards continuous runs (Group 5's slide; numbers from Lin, 2004)

Reference: `A B C D E F G` (7 tokens). Candidate 1: `A B C D H I K`. Candidate 2: `A H B K C I D`. Both have LCS = 4 (`A B C D`), so ROUGE-L cannot separate them. With $f(k) = k^2$:

- Candidate 1 matches in **one run of 4**: $\text{WLCS} = f(4) = 16$; $R_{\text{wlcs}} = \sqrt{16/49} = 4/7 = 0.571$.
- Candidate 2 matches in **four runs of 1**: $\text{WLCS} = 4 \times f(1) = 4$; $R_{\text{wlcs}} = \sqrt{4/49} = 2/7 = 0.286$.

Group 5 described this as “LCS finds what matched; W cares how it matched”, which is a good summary. (Both candidates have 7 tokens, so their Precision equals their Recall.)

Example 5.5 — ROUGE-S and ROUGE-SU (Group 5’s slides, verified)

ROUGE-S. Reference `the cat sat` has skip-bigrams (the, cat), (the, sat), (cat, sat). Candidate `the cat slept` has (the, cat), (the, slept), (cat, slept). One shared pair: $R = P = F_1 = 1/3 = 0.333$.

ROUGE-SU. Reference `A B C`, candidate `C B A` (reversed). Skip-bigrams: reference AB, AC, BC; candidate CB, CA, BA; **none** shared, so ROUGE-S = 0. Adding unigrams: 3 shared unigrams + 0 shared pairs = 3 matches out of 3 + 3 = 6 units on each side, so $R = P = F_1 = 3/6 = 0.50$. ROUGE-SU gives some credit for shared words even when no order is preserved.

Example 5.6 — A limitation: Khmer text with the default tokenizer (code verification)

`rouge-score`’s default tokenizer lowercases text and **replaces every character that is not a–z or 0–9 with a space**. Khmer letters are removed, so even two *identical* Khmer sentences score 0 (see output in 5.F). For Khmer, first segment the text into words with a Khmer word-segmentation tool, then use a custom tokenizer that splits on the inserted spaces, and report this choice. The same applies to other scripts outside a–z.

A second limitation, meaning reversal, is shown in Section 10.6: adding “not” to a sentence still gives ROUGE-L F1 = 0.92.

5.F Python implementation

Package: `rouge-score` (`pip install rouge-score`; Python import name `rouge_score`). It implements ROUGE-N (`rouge1` … `rouge9`), `rougeL` and `rougeLsum`, **but not ROUGE-W, ROUGE-S or ROUGE-SU**, so small teaching functions for those are written below. **Input format:** plain strings; **the reference (target) comes first**: `scorer.score(target, prediction)`.

```
from itertools import combinations

from rouge_score import rouge_scorer


def fmt(score):
    return f"P={score.precision:.4f} R={score.recall:.4f} F1={score.fmeasure:.4f}"


scorer = rouge_scorer.RougeScorer(["rouge1", "rouge2", "rougeL"])
# RougeScorer.score(target, prediction): the REFERENCE comes first.
for ref, cand in [
    ("the weather is cold outside", "the weather is cold"),
    ("the cat sat on the mat", "the cat rested on the mat"),
]:
    for name, s in scorer.score(ref, cand).items():
        print(f"{cand!r:30} {name:7} {fmt(s)}")

# ROUGE-L versus ROUGE-Lsum: sentences are separated by newlines.
ref = "heavy rain closed the schools today"
cand = "the schools closed today\nheavy rain fell"
for name in ["rougeL", "rougeLsum"]:
    s = rouge_scorer.RougeScorer([name]).score(ref, cand)[name]
    print(f"{name:9} {fmt(s)}")
```

Each result is a named tuple with `precision`, `recall` and `fmeasure`. `RougeScorer(..., use_stemmer=True)` would apply the Porter stemmer (so `cats` matches `cat`).

```python
def skip_bigrams(tokens):
    """Ordered pairs (earlier word, later word), any gap allowed."""
    return set(combinations(tokens, 2))  # teaching version: no repeated words


def rouge_s(ref, cand, with_unigrams=False):
    r_units, c_units = skip_bigrams(ref.split()), skip_bigrams(cand.split())
    if with_unigrams:  # ROUGE-SU also counts single words
        r_units |= {(w,) for w in ref.split()}
        c_units |= {(w,) for w in cand.split()}
    hits = len(r_units & c_units)
    p, r = hits / len(c_units), hits / len(r_units)
    f = 0.0 if hits == 0 else 2 * p * r / (p + r)
    return round(p, 4), round(r, 4), round(f, 4)


print("ROUGE-S  the cat sat / the cat slept:", rouge_s("the cat sat", "the cat slept"))
print("ROUGE-SU A B C / C B A:", rouge_s("A B C", "C B A", with_unigrams=True))
```

`itertools.combinations(tokens, 2)` produces every ordered pair that keeps sentence order. Using `set` is a simplification that is correct only when no word repeats; a full implementation counts repeated pairs.

```python
def wlcs(x, y, f=lambda k: k**2):
    """Weighted LCS (Lin, 2004): consecutive matches earn extra credit."""
    m, n = len(x), len(y)
    c = [[0.0] * (n + 1) for _ in range(m + 1)]
    w = [[0] * (n + 1) for _ in range(m + 1)]  # current run length
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if x[i - 1] == y[j - 1]:
                k = w[i - 1][j - 1]
                c[i][j] = c[i - 1][j - 1] + f(k + 1) - f(k)
                w[i][j] = k + 1
            else:
                c[i][j] = max(c[i - 1][j], c[i][j - 1])
    return c[m][n]


ref = list("ABCDEFG")
for cand in ["A B C D H I K", "A H B K C I D"]:
    score = wlcs(ref, cand.split())
    recall = (score / len(ref) ** 2) ** 0.5  # inverse of f(k) = k^2
    print(f"ROUGE-W recall for {cand!r}: WLCS={score:.0f}, R={recall:.4f}")
```

`c[i][j]` stores the best weighted score for the first `i` reference tokens and first `j` candidate tokens (dynamic programming); `w[i][j]` remembers how long the current run of consecutive matches is, so the next match in the run earns $f(k+1) - f(k)$.

```python
# The default tokenizer keeps only a-z and 0-9, so Khmer text disappears.
khmer = "ខ្ញុំ ចូលចិត្ត អាន សៀវភៅ"
print(
    "Default tokenizer, identical Khmer texts:",
    fmt(rouge_scorer.RougeScorer(["rouge1"]).score(khmer, khmer)["rouge1"]),
)


class WhitespaceTokenizer:
    """Use only for text that is already segmented into words."""

    def tokenize(self, text):
        return text.split()
```

```
custom = rouge_scorer.RougeScorer(["rouge1"], tokenizer=WhitespaceTokenizer())
print(
    "Whitespace tokenizer, identical Khmer texts:",
    fmt(custom.score(khmer, khmer)["rouge1"]),
)
```

The Khmer test sentence (“I like reading books”) was written with spaces between words only for this demonstration; normal Khmer writing does not put spaces between words, so real text needs a segmentation step first.

Executed output (file `code/ch05_rouge.py`):

```
'the weather is cold'          rouge1  P=1.0000 R=0.8000 F1=0.8889
'the weather is cold'          rouge2  P=1.0000 R=0.7500 F1=0.8571
'the weather is cold'          rougeL  P=1.0000 R=0.8000 F1=0.8889
'the cat rested on the mat'    rouge1  P=0.8333 R=0.8333 F1=0.8333
'the cat rested on the mat'    rouge2  P=0.6000 R=0.6000 F1=0.6000
'the cat rested on the mat'    rougeL  P=0.8333 R=0.8333 F1=0.8333
rougeL    P=0.4286 R=0.5000 F1=0.4615
rougeLsum P=0.7143 R=0.8333 F1=0.7692
ROUGE-S  the cat sat / the cat slept: (0.3333, 0.3333, 0.3333)
ROUGE-SU A B C / C B A: (0.5, 0.5, 0.5)
ROUGE-W recall for 'A B C D H I K': WLCS=16, R=0.5714
ROUGE-W recall for 'A H B K C I D': WLCS=4, R=0.2857
Default tokenizer, identical Khmer texts: P=0.0000 R=0.0000 F1=0.0000
Whitespace tokenizer, identical Khmer texts: P=1.0000 R=1.0000 F1=1.0000
```

Every number matches the hand calculations in 5.E. **Configuration choices to report:** package and version, variants, stemming, tokenizer, sentence splitting for Lsum, β, and how multiple references are combined.

5.G How to interpret results

- **High ROUGE Recall** supports “the summary contains much of the reference wording”. It does not show that the summary is correct, concise or well written: a very long candidate can reach high Recall by including everything.
- **High Precision** supports “little of the candidate’s wording is absent from the reference”.
- **ROUGE-1 vs ROUGE-2/L:** ROUGE-1 reflects word coverage; ROUGE-2 and ROUGE-L add some order information.
- A **low** score can mean a poor summary or a good summary written in different words. A **high** score can hide a reversed or invented fact.
- Compare scores only with the same references, variant and settings.

5.H Strengths and limitations

Strengths	Limitations (with examples)
Recall view suits summaries, where coverage matters	Only surface overlap: paraphrases score low (Section 10.6, paraphrase row)
Several variants for different kinds of overlap	Cannot detect factual errors: a "not" barely changes ROUGE-L (Section 10.6)
LCS variants allow gaps (Example 5.2)	Depends on reference quality and number of references
ROUGE-W and ROUGE-S capture run continuity and pair order (Examples 5.4, 5.5)	Recall favours longer outputs
Fast, standard for summarization research	Settings and sentence splitting change scores (Example 5.3)
	Default English tokenization erases Khmer (Example 5.6)

5.I When to use it

Use ROUGE (typically ROUGE-1, ROUGE-2 and ROUGE-L or Lsum, reported as F1, with stated settings) to compare summarization systems on the same test set with good reference summaries. Add a meaning-based metric (BERTScore, Chapter 7), a factual-consistency check, and human review, because summaries can be fluent, overlapping and still wrong (Maynez et al., 2020). For Khmer and other non-Latin scripts, segment and tokenize appropriately first.

5.J Common misunderstandings

- *"ROUGE is only Recall."* → **Correction:** it is recall-oriented in origin, but modern tools report Precision, Recall and F1; state which you use.
- *"ROUGE-L requires the words to be next to each other."* → **Correction:** LCS allows gaps; it requires only the same order. Consecutive matching is ROUGE-N's job (and ROUGE-W rewards it).
- *"ROUGE-L and ROUGE-Lsum are the same."* → **Correction:** Lsum works sentence by sentence and needs sentence boundaries (newlines in `rouge-score`); results can differ a lot (Example 5.3).
- *"A ROUGE score works the same in every language."* → **Correction:** tokenization and stemming are language-specific; the default `rouge-score` tokenizer gives 0 for Khmer (Example 5.6).

5.K Quick recap and practice

Recap. ROUGE counts overlapping units between a candidate and reference summaries: n-grams (ROUGE-N), longest in-order subsequence (ROUGE-L, Lsum), weighted runs (ROUGE-W), and ordered pairs with gaps (ROUGE-S, SU). Report Precision/Recall/F1, the variant and the settings.

Practice questions

1. Reference `a b c d` (4 words), candidate `a b x` (3 words). Compute ROUGE-1 Recall, Precision and F1.
2. How many skip-bigrams does a sentence of 5 distinct words have?
3. Why might ROUGE-Lsum be higher than ROUGE-L for the same multi-sentence summary?

Answers

1. Matches: `a`, `b` = 2. $R = 2/4 = 0.5$, $P = 2/3 = 0.667$, $F_1 = 2(0.5)(0.667)/(1.167) = 0.571$.
2. $\binom{5}{2} = 5 \times 4/2 = 10$.
3. Lsum compares each reference sentence with each candidate sentence and takes the union of matches, so correct content in a different sentence order still counts; ROUGE-L uses one in-order sequence for the whole text.

Reported experiment. Group 5's contents slide lists "Implementation", but the supplied PDF contains **no Python implementation slide and no experiment with model outputs**. Numbers in this chapter are teaching examples and code verifications.

5.L Sources

- Group 5 presentation: `Group5_ROUGE SCORE Group5.pdf` (introduction, Precision/Recall/F1 table, six ROUGE variants with examples, limitation table).
- Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. *Text Summarization Branches Out*. https://aclanthology.org/W04-1013/
- Google Research `rouge-score` package (source, tokenizer, `rougeLsum` newline rule): https://github.com/google-research/google-research/tree/master/rouge
- `rouge-score` on PyPI: https://pypi.org/project/rouge-score/
- Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On Faithfulness and Factuality in Abstractive Summarization. *ACL*. https://aclanthology.org/2020.acl-main.173/

Chapter 6: METEOR (Group 6)

6.A Learning goals

By the end of this chapter you should be able to:

- state which version and implementation of METEOR a score comes from;
- explain METEOR's staged word matching (exact, stem, synonym);
- compute Precision, Recall, the Recall-weighted mean F_{mean}, chunks, the fragmentation penalty and the final score by hand;
- explain when stemming and synonym resources are used and what happens for languages without them; and
- recognize implementation-specific behaviour, including two issues found in the NLTK version used by Group 6.

6.B Plain-English introduction

METEOR (Metric for Evaluation of Translation with Explicit ORdering) was introduced by Banerjee and Lavie (2005) to fix some weaknesses of BLEU. **Question answered:** "How many words of the candidate can be matched to the reference — allowing different word forms and synonyms — and how well is their order preserved?"

Unlike BLEU (Chapter 4), METEOR uses **Recall** as well as Precision (with more weight on Recall), it can match `cats` with `cat` (stemming) and `pupil` with `student` (synonyms), and it measures word order with an explicit **fragmentation penalty**.

What it measures: word-level alignment between candidate and reference, with flexible matching and an order penalty. **What it does not measure:** whether the content is true, whether the overall meaning is the same (a single "not" is one extra word), or anything for which it has no language resources. Group 6's slide says it well: METEOR "cannot prove the answer is true".

6.C Key terms

- **Alignment:** a list of word pairs (candidate word ↔ reference word), each word used at most once.
- **Matching stages (modules):** in the original METEOR, words are matched in three stages, and each stage only sees words still unmatched:
  1. **Exact:** identical words (`cat` ↔ `cat`).
  2. **Stem:** same Porter stem (`cats` → `cat` ↔ `cat`).
  3. **Synonym:** listed as synonyms in WordNet, a large English dictionary of word senses (`student` ↔ `pupil`).
- **Chunk:** a maximal group of matched words that are next to each other *and in the same order* in both sentences. Fewer chunks means better-preserved order.
- **Fragmentation:** chunks divided by matched words. One chunk for everything is the best case; one chunk per word is the worst.

6.D Formula and reasoning

Let m be the number of matched words (unigrams), t the number of words in the candidate (translation), r the number in the reference, and ch the number of chunks.

Precision and Recall:

$$P = \frac{m}{t}, \qquad R = \frac{m}{r}$$

Recall-weighted harmonic mean. The 2005 paper uses

$$F_{\text{mean}} = \frac{10PR}{R + 9P}$$

This is a weighted harmonic mean (compare F1, $2PR/(P+R)$, in Chapter 3). If $P = R$, it equals P. Otherwise it stays closer to R, because Banerjee and Lavie found that Recall agreed better with human judgments. Later versions write it with a parameter α: $F_{\text{mean}} = PR/(\alpha P + (1-\alpha)R)$; with $\alpha = 0.9$ this is exactly $10PR/(R+9P)$.

Fragmentation penalty:

$$\text{Pen} = \gamma \left(\frac{ch}{m} \right)^{\beta}, \qquad 2005 \text{ values: } \gamma = 0.5,\ \beta = 3$$

The fraction ch/m is between $1/m$ (one chunk) and 1 (every match separate). Cubing it ($\beta = 3$) keeps the penalty tiny when order is mostly preserved and makes it grow quickly as order breaks up. The maximum penalty is $\gamma = 0.5$.

Final score:

$$\text{METEOR} = F_{\text{mean}} \times (1 - \text{Pen})$$

Range and direction. Between 0 and 1; **higher is better**. Even a perfect copy does not reach exactly 1, because one chunk still gives a small penalty $0.5 \times (1/m)^3$ (Group 6 correctly noted "close to 1, not exactly 1"). If $m = 0$, the score is 0.

Versions and implementations — parameters are not universal.

Table 6.1 — METEOR versions. Scores from different versions are not comparable.

Version	What changes
Banerjee & Lavie (2005)	Fixed formula above; exact, Porter-stem and WordNet-synonym modules; alignment chosen to have the fewest crossing links
Lavie & Agarwal (2007)	Same structure but α, β, γ become parameters tuned to human judgments; support for more languages
Later Meteor releases, e.g. Meteor Universal (Denkowski & Lavie, 2014)	Paraphrase matching, different weights for function and content words (δ), language-specific resources and tuned parameter sets
NLTK `meteor_score` (version 3.10.4, used here and by Group 6)	Defaults $\alpha = 0.9$, $\beta = 3$, $\gamma = 0.5$ (reproducing the 2005 formula); exact → Porter stem → WordNet stages; lowercases input; requires pre-tokenized input; takes the best score over several references

Two NLTK behaviours found while checking this chapter (from reading NLTK 3.10.4's source code and running it):

1. **Synonyms are looked up on stemmed words.** After the stem stage, NLTK passes the *stemmed* unmatched words to the WordNet stage. `happy` has already become `happi`, which is not a WordNet word, so `glad` ↔ `happy` is **not** matched, although WordNet lists them as synonyms. (`pupil` ↔ `student` still works because their stems are unchanged.)
2. **Simplified alignment.** NLTK matches greedily within each stage instead of searching for the alignment with the fewest crossings, so its chunk counts (and scores) can differ from the official METEOR software.

Tokenization matters. NLTK compares tokens exactly as given. If you split on spaces, `data.` (with a full stop) and `data` are different tokens (Example 6.3).

6.E Worked examples

Example 6.1 — Same words, same order (Group 6's Example 1)

Reference `the cat sleeps`; candidate `the cat sleeps`.

1. Matches: the–the, cat–cat, sleeps–sleeps (exact): $m = 3$, $t = r = 3$.
2. $P = R = 3/3 = 1$, so $F_{\text{mean}} = 10 \times 1 \times 1/(1 + 9) = 1$.
3. All three matches are adjacent and in order: $ch = 1$.
4. $\text{Pen} = 0.5 \times (1/3)^3 = 0.5/27 = 0.0185$.
5. $\text{METEOR} = 1 \times (1 - 0.0185) = 0.9815$. ✔ matches Group 6's slide.

Example 6.2 — Same words, reversed order (Group 6's Example 2)

Candidate `sleeps cat the`. Still $m = 3$ and $F_{\text{mean}} = 1$, but now each match forms its own chunk: candidate positions 1, 2, 3 map to reference positions 3, 2, 1, which are not in the same order. $ch = 3$.

$\text{Pen} = 0.5 \times (3/3)^3 = 0.5$, so $\text{METEOR} = 1 \times 0.5 = 0.5000$. ✔ matches the slide. Note that BLEU-2 would give 0 here, while ROUGE-1 would give 1; METEOR sits in between.

Example 6.3 — Group 6's "experiment": a lesson in tokenization (teaching demo, re-checked)

Group 6 compared two *sample* answers (their slide says: "Teaching demo: these are example answers, not verified live LLM outputs"). Reference: `Machine learning helps computers learn from data.`

- Model A (identical copy): $m = 7$ of 7 words, one chunk: $\text{Pen} = 0.5(1/7)^3 = 0.0015$; METEOR $= 0.9985$ (slide: 0.999 ✔).
- Model B: `Computers learn from data with machine learning.` The slide reports **0.536**, which NLTK reproduces (0.5357) **when the sentences are split on spaces**. Then the reference has the token `data.` and the candidate has `data` and `learning.`, so these do not match. Only $m = 4$ words match (computers, learn, from, machine) in $ch = 2$ chunks: $P = R = 4/7 = 0.5714$; $\text{Pen} = 0.5(2/4)^3 = 0.0625$; METEOR $= 0.5714 \times 0.9375 = 0.5357$.
- If punctuation is split into separate tokens (`data`, `.`), then $m = 7$ of 8 tokens match in $ch = 3$ chunks: $P = R = 7/8$; $\text{Pen} = 0.5(3/7)^3 = 0.0394$; METEOR $= 0.875 \times 0.9606 = 0.8406$.

The same pair scores 0.54 or 0.84 depending only on tokenization. Group 6's conclusion ("Model A scores higher because its words and order match the reference") is still true, but the size of the gap is largely an artifact. Report tokenization with every METEOR score.

Example 6.4 — Synonyms: what NLTK actually matches (Group 6's alignment slide, corrected)

Reference `the student is happy`; candidate `the pupil is glad`. Group 6's slide states that all 4 words match (2 exact + 2 synonym). NLTK 3.10.4 actually finds **3** matches: the–the and is–is (exact), pupil–student (synonym). `glad`–`happy` fails because of the stemming issue described in 6.D.

$m = 3$, $P = R = 3/4$, $F_{\text{mean}} = 0.75$. The candidate matches at positions 1, 2, 3 map to reference positions 1, 2, 3 in order: $ch = 1$. $\text{Pen} = 0.5(1/3)^3 = 0.0185$. $\text{METEOR} = 0.75 \times 0.9815 = 0.7361$.

If all four words had matched in one chunk, the score would have been $1 \times (1 - 0.5(1/4)^3) = 0.9922$. The slide's explanation describes the *idea* of METEOR correctly; it does not describe what this implementation does.

Example 6.5 — Stemming helps, word order still costs (teaching example)

Reference `the cats are sleeping`; candidate `the cat is sleeping`.

- Exact: the–the, sleeping–sleeping. Stem: cat–cats (`cats` → `cat`). `is` and `are` do not match (different stems; not WordNet synonyms in NLTK's lookup). $m = 3$.
- $P = R = 3/4$; $F_{\text{mean}} = 0.75$.
- Chunks: (the, cat) are adjacent in both texts; `sleeping` is separated by an unmatched word: $ch = 2$.
- $\text{Pen} = 0.5(2/3)^3 = 0.1481$; METEOR $= 0.75 \times 0.8519 = 0.6389$.

6.F Python implementation

Packages: `nltk` (`pip install nltk`) plus the WordNet data (`python -m nltk.downloader wordnet`). **Input format:** a *list of tokenized references* (each a list of strings) and one tokenized hypothesis. Passing plain strings raises a `TypeError` in current NLTK versions.

```
import re

from nltk.translate.meteor_score import meteor_score

examples = [
    ("the cat sleeps", "the cat sleeps"),  # Group 6, Example 1
    ("the cat sleeps", "sleeps cat the"),  # Group 6, Example 2
    (
        "Machine learning helps computers learn from data.",
        "Machine learning helps computers learn from data.",
    ),
    (
        "Machine learning helps computers learn from data.",
        "Computers learn from data with machine learning.",
    ),
    ("the student is happy", "the pupil is glad"),
    ("the cats are sleeping", "the cat is sleeping"),
]
for ref, hyp in examples:
    # NLTK needs token lists, not strings. Here we split on spaces,
    # so "data." stays one token including its full stop.
    score = meteor_score([ref.split()], hyp.split())
    print(f"{hyp!r:52} {score:.4f}")
```

`meteor_score(references, hypothesis)` takes a list of references because METEOR can compare against several and keep the best. Optional arguments `alpha`, `beta` and `gamma` change the formula parameters; the defaults are 0.9, 3.0 and 0.5.

```
def words(text):
    """Separate punctuation from words: 'data.' -> 'data', '.'"""
    return re.findall(r"\w+|[^\w\s]", text)


ref = "Machine learning helps computers learn from data."
hyp = "Computers learn from data with machine learning."
print("Split on spaces     :", round(meteor_score([ref.split()], hyp.split()), 4))
print("Punctuation split   :", round(meteor_score([words(ref)], words(hyp)), 4))

try:
```

```
    meteor_score(["the cat sleeps"], "the cat sleeps")  # raw strings
except TypeError as error:
    print("TypeError:", str(error)[:60], "...")
```

The regular expression `\w+|[^\w\s]` keeps runs of word characters together and makes every other non-space character (such as `.`) a separate token.

Executed output (file `code/ch06_meteor.py`):

```
'the cat sleeps'                                    0.9815
'sleeps cat the'                                    0.5000
'Machine learning helps computers learn from data.'  0.9985
'Computers learn from data with machine learning.'   0.5357
'the pupil is glad'                                 0.7361
'the cat is sleeping'                               0.6389
Split on spaces     : 0.5357
Punctuation split   : 0.8406
TypeError: "hypothesis" expects pre-tokenized hypothesis (Iterable[str] ...
```

Configuration choices to report: METEOR version and software (NLTK version or official Java Meteor), α, β, γ, language, matching modules and resources (stemmer, WordNet version), tokenization, lowercasing, and how multiple references are combined.

6.G How to interpret results

- A **high** METEOR score supports "most words can be aligned to the reference in mostly the same order, counting stems and (some) synonyms". It does not show that the meaning or facts are correct.
- A **low** score can come from different wording, different order, *or* from tokenization and missing language resources (Example 6.3).
- Scores are comparable only within the same version, parameters and tokenization.

6.H Strengths and limitations

Strengths	Limitations (with examples)
Uses Recall and Precision, weighted towards Recall	Needs reference answers
Credits word forms (Example 6.5) and some synonyms	Synonym matching depends on resources and implementation (Example 6.4)
Explicit word-order penalty (Examples 6.1–6.2)	Sensitive to tokenization (Example 6.3)
Works at sentence level without BLEU's zero problem	Parameters and versions differ (Table 6.1)
Easy to run in Python with NLTK	English-centred resources: for Khmer, no Porter stemmer or WordNet synonyms exist in NLTK, so only exact matches remain, after word segmentation
	Does not check truth: one inserted "not" costs only a little (Section 10.6)

6.I When to use it

Use METEOR for translation, paraphrase or short generated answers in languages with good resources (especially English), when you want partial credit for word forms and synonyms and an explicit order check. Report the version and settings. Combine it with human review and, for factual tasks, with fact checking. For Khmer, prefer character-based measures (Chapter 9) or embedding-based measures with a model that supports Khmer (Chapter 7), together with human review.

6.J Common misunderstandings

- *"METEOR always understands synonyms."* → **Correction:** it matches only synonyms listed in its resources, and the NLTK implementation misses some of them (Example 6.4).
- *"A METEOR score is the same everywhere."* → **Correction:** versions use different parameters and modules (Table 6.1), and tokenization alone changed one score from 0.54 to 0.84 (Example 6.3).
- *"An exact copy scores 1."* → **Correction:** the one-chunk penalty keeps it slightly below 1 (0.9815 for three words).
- *"METEOR = matching quality − penalty."* (Group 6's summary line) → **Correction:** the penalty *multiplies*: $F_{\text{mean}} \times (1 - \text{Pen})$. The slide's full formula on the next slide is correct.

6.K Quick recap and practice

Recap. METEOR aligns words in stages (exact, stem, synonym), computes a Recall-weighted mean of Precision and Recall, and multiplies it by $(1 - \text{Pen})$ where $\text{Pen} = 0.5\,(ch/m)^3$ in the 2005 version. Versions, resources and tokenization strongly affect results.

Practice questions

1. $m = 4$, $t = 5$, $r = 4$, $ch = 2$. Compute METEOR (2005 parameters).
2. Why is F_{mean} closer to Recall than F1 is?
3. In NLTK, why does `glad` not match `happy`?

Answers

1. $P = 4/5 = 0.8$, $R = 4/4 = 1$. $F_{\text{mean}} = 10(0.8)(1)/(1 + 9 \times 0.8) = 8/8.2 = 0.9756$.
   $\text{Pen} = 0.5(2/4)^3 = 0.0625$. METEOR $= 0.9756 \times 0.9375 = 0.9146$.
2. It is a weighted harmonic mean with weight 9 on the Recall side of the denominator ($\alpha = 0.9$), chosen because Recall correlated better with human judgments in the 2005 study.
3. NLTK runs the synonym stage on words that the stem stage has already stemmed; `happy` has become `happi`, which WordNet does not contain.

6.L Sources

- Group 6 presentation: `Group6_METEOR Score.pdf` (steps, three-stage alignment, formula, two examples, NLTK code, sample-answer comparison).
- Banerjee, S., & Lavie, A. (2005). METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. *ACL Workshop on Evaluation Measures*. https://aclanthology.org/W05-0909/
- Lavie, A., & Agarwal, A. (2007). METEOR: An Automatic Metric for MT Evaluation with High Levels of Correlation with Human Judgments. *WMT*. https://aclanthology.org/W07-0734/
- Denkowski, M., & Lavie, A. (2014). Meteor Universal: Language Specific Translation Evaluation for Any Target Language. *WMT*. https://aclanthology.org/W14-3348/
- NLTK `meteor_score` documentation: https://www.nltk.org/api/nltk.translate.meteor_score.html
- NLTK source code, `nltk/translate/meteor_score.py`: https://github.com/nltk/nltk/blob/develop/nltk/translate/meteor_score.py
- WordNet, Princeton University: https://wordnet.princeton.edu/

Chapter 7: BERTScore (Group 7)

7.A Learning goals

By the end of this chapter you should be able to:

- explain in simple words what an embedding and a *contextual* embedding are;
- explain cosine similarity and how BERTScore matches tokens greedily;
- compute BERTScore Precision, Recall and F1 from a similarity table;
- explain the effect of the model and layer, IDF weighting and baseline rescaling;
- check whether a model supports a language (with Khmer as the example); and
- explain why high semantic similarity does not guarantee factual correctness.

7.B Plain-English introduction

BERTScore was introduced by Zhang et al. (2020). **Question answered:** "Do the pieces of the candidate mean roughly the same as the pieces of the reference, even if different words are used?"

String-overlap metrics such as BLEU and ROUGE give no credit for `kid` versus `boy` unless a synonym list happens to contain them. BERTScore instead uses a pretrained neural language model (such as BERT or RoBERTa) to turn every token into a vector of numbers that reflects its meaning *in context*, and then compares those vectors.

What it measures: token-by-token semantic similarity between candidate and reference, summarized as Precision, Recall and F1. **What it does not measure:** truth. Group 7 listed "negation blindness" as a limitation, and the examples below confirm that a sentence and its negation can score very high.

7.C Key terms

- **Embedding:** a list of numbers (a *vector*) that represents a token. Models are trained so that tokens used in similar ways get similar vectors. Think of each token as a point in a space with hundreds of directions (bert-base uses 768 numbers per token).
- **Contextual embedding:** the vector depends on the surrounding words. Group 7's example: "bank" in *river bank* and in *investment bank* receive different vectors.
- **Subword token:** BERT-style models split rare words into pieces (`joyful` might stay whole; a rare name may become several pieces). BERTScore works on these pieces.
- **Cosine similarity:** a measure of how closely two vectors point in the same direction:

$$\cos(\boldsymbol{a}, \boldsymbol{b}) = \frac{\boldsymbol{a} \cdot \boldsymbol{b}}{\|\boldsymbol{a}\| \, \|\boldsymbol{b}\|}$$

It ranges from -1 (opposite) through 0 (unrelated directions) to 1 (same direction). If both vectors are first scaled to length 1 (*normalized*), cosine similarity is just the dot product $\boldsymbol{a} \cdot \boldsymbol{b} = \sum_k a_k b_k$.

- **Layer:** a transformer model has many layers; each produces its own token vectors. BERTScore uses one chosen layer (for `bert-base-uncased`, layer 9 by default in the `bert-score` package; for `roberta-large`, layer 17).
- **IDF (inverse document frequency):** a weight that is high for rare words and low for common words such as "the".

7.D Formula and reasoning

Let the reference have tokens $x_1, \ldots, x_k$ and the candidate $\widehat{x}_1, \ldots, \widehat{x}_l$, each represented by a normalized contextual embedding. Compute the similarity $\boldsymbol{x}_i^\top \widehat{\boldsymbol{x}}_j$ for every pair: a $k \times l$ table (Group 7's "similarity matrix").

Greedy matching. Each token is matched to its single most similar token on the other side (a token may be used by several matches):

$$R_{\text{BERT}} = \frac{1}{k}\sum_{i=1}^{k} \max_j \boldsymbol{x}_i^\top \widehat{\boldsymbol{x}}_j \qquad P_{\text{BERT}} = \frac{1}{l}\sum_{j=1}^{l} \max_i \boldsymbol{x}_i^\top \widehat{\boldsymbol{x}}_j \qquad F_{\text{BERT}} = \frac{2P_{\text{BERT}}R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}$$

- **Recall** takes, for every *reference* token, its best match in the candidate (the maximum of each row), then averages: "Is the reference covered?"
- **Precision** takes, for every *candidate* token, its best match in the reference (the maximum of each column), then averages: "Is every part of the candidate supported?"
- **F1** is their harmonic mean (Chapter 3).

Optional IDF weighting. With M reference sentences in the test set, $\text{idf}(w) = -\log \frac{1}{M}\sum_{i=1}^{M} \mathbb{1}[w \in x^{(i)}]$ (with plus-one smoothing for unseen pieces). Recall becomes a weighted average:

$$R_{\text{BERT}} = \frac{\sum_i \text{idf}(x_i) \max_j \boldsymbol{x}_i^\top \widehat{\boldsymbol{x}}_j}{\sum_i \text{idf}(x_i)}$$

and Precision likewise. IDF is computed from the references of the test set, so it needs a reasonably large set of references to be meaningful.

Optional baseline rescaling. Raw scores in practice fall in a narrow high band (Group 7: "for example 0.80 to 1.00"), which makes differences hard to read. The authors compute a baseline b, the average score of random, unrelated sentence pairs from Common Crawl, for each model and layer, and rescale linearly:

$$\widehat{R}_{\text{BERT}} = \frac{R_{\text{BERT}} - b}{1 - b}$$

(and the same for P and F with their own baselines). Because this is a straight-line transformation with a positive slope, it **does not change the ranking** of candidates scored with the same model, layer and score type (Group 7 is correct). It does change the numbers, so raw and rescaled scores must never be mixed.

Range and direction. Raw scores are cosine similarities, so mathematically between -1 and 1; in practice they are mostly high positive values. Rescaled scores are "typically between 0 and 1" according to the paper, but they can be negative for very unrelated texts. **Higher means more similar** under the chosen model.

Configuration choices that change scores.

Table 7.1 — BERTScore configuration.

Setting	Why it matters
Model (`model_type`)	Different models give different vectors; `bert-score` defaults to `roberta-large` for `lang="en"` and `bert-base-multilingual-cased` for most other languages
Layer (`num_layers`)	Each layer encodes different information; the package has a tuned default per model
IDF (`idf=True`)	Up-weights rare words; computed from the references
Baseline rescaling	Changes the numeric scale; available only for languages with a baseline file

Setting	Why it matters
Language support	A model that does not know a script cannot represent it (see below)
Package version	Recorded in the hash string returned by `return_hash=True`

Language support: a Khmer check (code verification). For a language code such as `"km"` (Khmer), `bert-score` 0.3.13 falls back to `bert-base-multilingual-cased`. Its tokenizer turned the Khmer sentence ខ្ញុំចូលចិត្តអានសៀវភៅ ("I like reading books") into a single `[UNK]` (unknown) token, so every Khmer sentence would be represented almost identically and the scores would be meaningless. The tokenizer of `xlm-roberta-base` split the same sentence into meaningful pieces (▁ខ្ញុំ, ចូល, ចិត្ត, អាន, សៀវភៅ). A model whose tokenizer knows Khmer (for example from the XLM-RoBERTa family, selected with `model_type`) is therefore necessary, but tokenization alone does not prove the scores agree with human judgments for Khmer. In addition, `bert-score` 0.3.13 ships rescaling baselines only for cs, de, en, en-sci, es, et, fi, fr, it, lv, pt and zh, not for Khmer.

7.E Worked examples

Example 7.1 — Greedy matching by hand (Group 7's illustrative matrix)

Group 7's slide 6 used *invented* similarity values to teach the idea ("Illustrative values chosen for teaching; real values come from the model"). Reference `the boy is happy` (rows); candidate `the kid feels joyful` (columns):

Table 7.2 — Group 7's teaching matrix (not model output).

	the	kid	feels	joyful	row max
the	0.95	0.20	0.22	0.18	0.95
boy	0.25	0.82	0.30	0.28	0.82
is	0.30	0.28	0.78	0.35	0.78
happy	0.20	0.30	0.40	0.88	0.88
column max	0.95	0.82	0.78	0.88	

- Recall $= (0.95 + 0.82 + 0.78 + 0.88)/4 = 3.43/4 = 0.8575$.
- Precision $= (0.95 + 0.82 + 0.78 + 0.88)/4 = 0.8575$ (here each column's maximum happens to be the same cell as a row's maximum).
- F1 $= 0.8575$, which Group 7 rounded to 0.86. ✔

With real vectors (`bert-base-uncased`, layer 9) the same pair scores P = 0.7235, R = 0.7766, F1 = 0.7491 (code verification; the F1 value is printed by the code in 7.F), lower than the teaching values but still clearly above the near-zero word overlap.

Example 7.2 — Similar words, different meaning (teaching examples; code verification)

Table 7.3 — `bert-base-uncased`, layer 9, no IDF; rescaled with the package's English baseline.

Reference	Candidate	What changed	Raw F1	Rescaled F1
The boy is happy	The kid feels joyful	same meaning, other words	0.7491	0.6127
The dog chased the cat	The cat chased the dog	roles reversed	0.9356	0.9006
The medicine is safe for children.	The medicine is not safe for children.	meaning negated	0.9362	0.9015
The train leaves at 9 a.m.	The train leaves at 5 p.m.	wrong time	0.9700	0.9537

The correct paraphrase gets the **lowest** score, and the three meaning-changing edits get much higher scores. Contextual embeddings help, but token-level matching is dominated by the many shared tokens. BERTScore must not be used as a fact or negation check. Rescaling changed every number but kept the same order.

7.F Python implementation

Packages: `torch`, `transformers`, `bert-score` (`pip install torch transformers bert-score`). The first run downloads the model (about 440 MB for `bert-base-uncased`); a CPU is enough for these examples.
Input format: `score(cands, refs, ...)` takes two lists of strings of equal length; `cands[i]` is compared with `refs[i]`. It returns three PyTorch tensors P, R, F with one value per pair.

```
from bert_score import score
from transformers import AutoTokenizer

CONFIG = {"model_type": "bert-base-uncased", "num_layers": 9}  # Group 7

# Group 7, Example 1 (paraphrasing). Outputs copied from the slides.
ref = (
    "Artificial intelligence accelerates scientific discovery by "
    "analyzing massive datasets quickly."
)
cands = [
    "AI accelerates scientific discovery by rapidly analyzing vast datasets.",
    ref,
]  # SmolLM2 copied the reference
P, R, F = score(cands, [ref] * len(cands), **CONFIG)
for name, p, r, f in zip(["Qwen2.5-0.5B", "SmolLM2-135M"], P, R, F):
    print(f"Ex1 {name:13} P={p:.4f} R={r:.4f} F1={f:.4f}")

# Group 7, Example 2 (summary with missing details).
ref2 = "The city opened a new public library with free Wi-Fi and study rooms."
cands2 = [
    (
        "The city has launched a new public library that provides free "
        "Wi-Fi and study spaces for residents."
    ),
    "The city opened a new public library.",
]
P, R, F = score(cands2, [ref2] * 2, **CONFIG)
for name, p, r, f in zip(["Qwen2.5-0.5B", "SmolLM2-135M"], P, R, F):
    print(f"Ex2 {name:13} P={p:.4f} R={r:.4f} F1={f:.4f}")
```

`[ref] * len(cands)` repeats the single reference so that each candidate has a partner. `CONFIG` fixes the model and layer that Group 7 reported; changing either changes every score.

```
# Teaching examples: similar wording, different meaning.
refs = [
    "The boy is happy",
    "The dog chased the cat",
    "The medicine is safe for children.",
    "The train leaves at 9 a.m.",
]
hyps = [
    "The kid feels joyful",
    "The cat chased the dog",
    "The medicine is not safe for children.",
    "The train leaves at 5 p.m.",
]
_, _, F_raw = score(hyps, refs, **CONFIG)
_, _, F_res = score(hyps, refs, lang="en", rescale_with_baseline=True, **CONFIG)
for h, raw, res in zip(hyps, F_raw, F_res):
    print(f"{h!r:42} raw F1={raw:.4f} rescaled F1={res:.4f}")

(_, _, _), config_hash = score(hyps[:1], refs[:1], return_hash=True, **CONFIG)
print("Hash:", config_hash)
```

`rescale_with_baseline=True` needs `lang` so the package can find the baseline file. The **hash** string records the model, layer, IDF and version settings; report it with your scores.

```
# Language support: look at how each model's tokenizer splits Khmer text.
# bert-score uses bert-base-multilingual-cased by default for lang="km".
khmer = "ខ្ញុំចូលចិត្តអានសៀវភៅ"  # "I like reading books"
for name in ["bert-base-multilingual-cased", "xlm-roberta-base"]:
    tokenizer = AutoTokenizer.from_pretrained(name)
    print(name, tokenizer.tokenize(khmer))
```

Executed output (file `code/ch07_bertscore.py`; model-loading messages omitted):

```
Ex1 Qwen2.5-0.5B  P=0.9226 R=0.9042 F1=0.9133
Ex1 SmolLM2-135M  P=1.0000 R=1.0000 F1=1.0000
Ex2 Qwen2.5-0.5B  P=0.8497 R=0.9181 F1=0.8826
Ex2 SmolLM2-135M  P=0.9563 R=0.7136 F1=0.8173
'The kid feels joyful'                     raw F1=0.7491 rescaled F1=0.6127
'The cat chased the dog'                   raw F1=0.9356 rescaled F1=0.9006
'The medicine is not safe for children.'   raw F1=0.9362 rescaled F1=0.9015
'The train leaves at 5 p.m.'               raw F1=0.9700 rescaled F1=0.9537
Hash: bert-base-uncased_L9_no-idf_version=0.3.12(hug_trans=5.23.0)
bert-base-multilingual-cased ['[UNK]']
xlm-roberta-base ['▁ខ្ញុំ', 'ចូល', 'ចិត្ត', 'អាន', 'សៀវភៅ']
```

The hash reports `version=0.3.12`: that is the version string stored inside the installed package, although the package metadata (`pip show bert-score`) reports 0.3.13. The last two lines confirm the Khmer tokenizer check described in 7.D.

7.G How to interpret results

- **Higher BERTScore** means the candidate's tokens have closer contextual vectors to the reference's tokens *under this model and layer*. It supports "similar wording or meaning", not "correct".
- **Precision vs Recall** is informative: high Precision with low Recall suggests the candidate is accurate but leaves things out (Group 7's Example 2: SmolLM2 P = 0.9563, R = 0.7136).
- **Differences** between raw scores are small in absolute size (often a few hundredths). Use rescaled scores for readability, and compare only scores from the same configuration.
- A **perfect score** may mean the model copied the reference, which can be a failure for paraphrasing tasks (Group 7's Example 1).

7.H Strengths and limitations

Strengths	Limitations (with examples)
Credits paraphrases that string metrics miss (Section 10.6: paraphrase BLEU 7.8 vs BERTScore 0.75)	Not a fact or negation check (Table 7.3)
Separates coverage (Recall) from support (Precision)	Scores depend on model, layer, IDF and rescaling (Table 7.1)
Works at sentence level; no zero problem	Needs a neural model: slower, more memory than string metrics
Can use multilingual models	Language support must be checked: Khmer → `[UNK]` with the default multilingual BERT
	Raw scores are compressed into a narrow band; small differences are hard to judge

7.I When to use it

Use BERTScore when several phrasings are acceptable — abstractive summarization, paraphrasing, translation, open-ended answers — and report it **alongside** a lexical metric (BLEU, ROUGE) and with the model, layer, IDF and rescaling settings (Group 7's best-practice advice). Add task-specific checks for facts, numbers and negation whenever correctness matters. For Khmer, choose a model whose tokenizer and training data cover Khmer, and validate against human judgments on a sample.

7.J Common misunderstandings

- *"BERTScore measures whether the answer is true."* → **Correction:** it measures similarity of contextual token vectors. A negated sentence scored 0.94 (Table 7.3).
- *"BERTScore F1 0.90 is excellent."* → **Correction:** raw scores are compressed; 0.90 can be ordinary. Compare with other candidates under the same settings, or use rescaled scores.
- *"Rescaling makes the scores more accurate."* → **Correction:** it only stretches the scale; rankings stay the same.
- *"Multilingual means every language works."* → **Correction:** check the tokenizer; multilingual BERT maps Khmer to `[UNK]`.

7.K Quick recap and practice

Recap. BERTScore embeds every token with a pretrained model, matches each token to its most similar partner by cosine similarity, and averages: Recall over reference tokens, Precision over candidate tokens, F1 combines them. Model, layer, IDF and rescaling must be reported; similarity is not truth.

Practice questions

1. Row maxima of a similarity table are 0.9, 0.8, 0.7 (three reference tokens) and column maxima are 0.9, 0.6 (two candidate tokens). Compute P, R and F1.
2. Raw F1 = 0.92 and the baseline is $b = 0.83$. What is the rescaled F1?
3. Why did SmolLM2 receive F1 = 1.0000 in Group 7's paraphrase experiment, and why is that not a success?

Answers

1. $R = (0.9 + 0.8 + 0.7)/3 = 0.8$; $P = (0.9 + 0.6)/2 = 0.75$; $F_1 = 2(0.75)(0.8)/1.55 = 0.7742$.
2. $(0.92 - 0.83)/(1 - 0.83) = 0.09/0.17 = 0.529$.
3. It copied the reference word for word, so every token matched itself perfectly. The task asked for a paraphrase, which the copy does not provide; the metric cannot see the instruction.

Reported experiment by Group 7

Table 7.4 — Group 7's experiment record.

Item	As documented on Group 7's slides
Models	`Qwen/Qwen2.5-0.5B-Instruct` and `HuggingFaceTB/SmolLM2-135M-Instruct`
Generation settings	Slide 9: "llama.cpp Q4_K_M • temperature 0 • maximum 96 new tokens". The code on slide 11 instead uses the Hugging Face Transformers `pipeline` with `do_sample=False` and `max_new_tokens=96`. **The two descriptions differ**; the slides do not say which produced the outputs.
Tasks	Example 1: paraphrase one sentence "while preserving all information" (the code's prompt is "Paraphrase in one sentence: " + reference). Example 2: summarize a library announcement in one sentence.
Sample size	One sentence per task, one output per model
Scoring	`bert-score`, `model_type="bert-base-uncased"`, `num_layers=9`, no IDF, no rescaling; sacreBLEU `sentence_bleu` with effective order; `rouge-score` ROUGE-L with stemming
Missing	Run date, library versions, the full prompt for Example 2, the script `compare_metrics.py` mentioned on slide 12 (not supplied), and any repeated runs

*Table 7.5 — Group 7's reported results. This guide **re-scored the reported output texts** with the same scoring settings and obtained exactly these values (7.F output; BLEU and ROUGE-L checked separately with sacreBLEU 2.6.0 and rouge-score 0.1.2). The model outputs themselves were **not regenerated**.*

Metric	Qwen2.5-0.5B-Instruct	SmolLM2-135M-Instruct
BLEU (0–100)	27.34	100.00
ROUGE-L F1	0.6316	1.0000
BERTScore P / R / F1 (Example 1)	0.9226 / 0.9042 / 0.9133	1.0000 / 1.0000 / 1.0000
BERTScore P / R / F1 (Example 2)	0.8497 / 0.9181 / 0.8826	0.9563 / 0.7136 / 0.8173

Interpretation. In Example 1 SmolLM2 copied the reference, so every overlap metric is perfect, but it did not paraphrase; Qwen produced a real paraphrase with lower BLEU (27.34) but high BERTScore (0.9133). In Example 2 SmolLM2 omitted the Wi-Fi and study rooms, which shows up as low Recall. With one sentence per task, these results illustrate the metrics; they do not rank the models in general.

7.L Sources

- Group 7 presentation: `Group7_BERTScore - Group 07.pptx` (definition, motivation, four stages, illustrative matrix, experiment with two small models, code, metric comparison, limitations).
- Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating Text Generation with BERT. *ICLR*. https://arxiv.org/abs/1904.09675
- `bert-score` package source and README: https://github.com/Tiiiger/bert_score
- Baseline rescaling notes: https://github.com/Tiiiger/bert_score/blob/master/journal/rescale_baseline.md
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL*. https://aclanthology.org/N19-1423/
- Conneau, A., et al. (2020). Unsupervised Cross-lingual Representation Learning at Scale (XLM-RoBERTa). *ACL*. https://aclanthology.org/2020.acl-main.747/

Chapter 8: Word Error Rate — WER (Group 8)

8.A Learning goals

By the end of this chapter you should be able to:

- define substitutions, deletions and insertions at the word level;
- compute the minimum word edit distance with a dynamic-programming table;
- calculate WER and explain why the reference length is the denominator;
- show that WER can exceed 100%;
- explain why different minimum alignments can give different S/D/I counts;
- choose and report text-cleanup and empty-reference rules; and
- compute corpus WER correctly.

8.B Plain-English introduction

Question answered: "How many word mistakes would we have to fix to turn the system's transcript into the correct transcript, relative to the length of the correct transcript?"

WER is the standard measure for automatic speech recognition (ASR: turning speech into text), as Group 8 explained. It is also used for any output that should reproduce a known word sequence exactly.

What it measures: word-level differences after the best possible alignment. **What it does not measure:** meaning or importance. Group 8's example: "turn off" versus "turn of" is one small edit, but the meaning changes; "books" versus "book" is also one edit, but the meaning mostly survives. WER counts both the same.

8.C Key terms

Group 8's slide 4 uses these three error types, always described as changes **from the reference to the hypothesis** (the system output):

Table 8.1 — Word error types.

Error	Symbol	Meaning	Example (reference → hypothesis)
Substitution	S	a reference word is replaced by a different word	`sits` → `sit`
Deletion	D	a reference word is missing	`the` → (nothing)
Insertion	I	an extra word appears	(nothing) → `today`

- **Correct (hit), C:** a reference word matched by the same word.
- **Alignment:** a pairing of reference words and hypothesis words that shows where each S, D, I and C happens.
- **Minimum edit distance (word-level Levenshtein distance):** the smallest possible $S + D + I$ over all alignments.
- **Dynamic programming:** solving the problem for short prefixes first, storing the answers in a table, and reusing them (see 8.D).

8.D Formula and reasoning

$$\text{WER} = \frac{S + D + I}{N} = \frac{S + D + I}{S + D + C}$$

- **Numerator:** the minimum number of word edits, taken from the best alignment.
- **Denominator:** N, the number of words in the **reference**. Every reference word ends up as exactly one of: correct, substituted, or deleted, so $N = S + D + C$. Insertions are not reference words, so they are not in N.

Why divide by the reference length? The reference is the fixed target. Dividing by it means that two systems tested on the same references are measured against the same denominator; a system cannot lower its WER by producing a different amount of text. (If we divided by the hypothesis length, a system could reduce its rate by adding extra words.)

Range and direction. $\text{WER} \geq 0$; **lower is better**; 0 means a perfect transcript under the stated text rules. There is **no upper limit of 1**, because insertions can exceed the number of reference words: reference `yes`, hypothesis `yes yes yes` gives $I = 2$, $N = 1$, $\text{WER} = 2/1 = 200\%$. For the same reason, "word accuracy" $= 1 - \text{WER}$, which Group 8 reported as "accuracy 90.6%", can become negative and is not a true accuracy (Chapter 2).

How the minimum is found. Let $D[i][j]$ be the minimum number of edits needed to turn the first i reference words into the first j hypothesis words. The first row and column are $D[0][j] = j$ (insert j words) and $D[i][0] = i$ (delete i words). Every other cell takes the cheapest of three moves:

$$D[i][j] = \min \begin{cases} D[i-1][j] + 1 & (\text{delete reference word } i) \\ D[i][j-1] + 1 & (\text{insert hypothesis word } j) \\ D[i-1][j-1] + \mathbb{1}[\text{ref}_i \neq \text{hyp}_j] & (\text{match: cost 0, or substitute: cost 1}) \end{cases}$$

The answer is the bottom-right cell. Following the choices back from that cell gives an alignment and its S, D and I counts.

Several minimum alignments can exist. Reference `a b`, hypothesis `b a`: one alignment substitutes both words ($S = 2$); another deletes `a`, matches `b`, and inserts `a` ($D = 1$, $I = 1$). Both cost 2, so WER = 100% either way, but the reported S/D/I counts depend on the tool's tie-breaking rule. Compare totals, not individual counts, across tools.

Text cleanup changes the score. Is `Hello` the same word as `hello`? Is `lights.` the same as `lights`? Is `nine` the same as `9`? WER compares exact word strings, so these must be decided *before* scoring and applied identically to reference and hypothesis. The `jiwer` library's default only removes extra spaces and splits on spaces; it does **not** lowercase or remove punctuation (jiwer documentation).

Corpus WER. For a test set, add all errors and all reference words, then divide: $\sum(S + D + I) / \sum N$. This is not the same as averaging per-sentence WERs, which would give short sentences too much weight. `jiwer.wer` does the pooled calculation when given lists.

Empty reference. If $N = 0$, the formula divides by zero. Tools need a rule. In jiwer 4.0, an empty reference gives a WER equal to the number of inserted words (so 0 if the hypothesis is also empty); other tools raise an error. State your rule. Group 9's CER notebook chose to reject empty references (Chapter 9).

Languages without spaces. Khmer, Thai, Chinese and Japanese are not written with spaces between words, so "words" must first be created by a word segmenter. Different segmenters give different word counts and therefore different WERs; CER is often reported instead or as well.

8.E Worked examples

Example 8.1 — One substitution (Group 8's slide 6)

Reference `I love to read books` ($N = 5$); hypothesis `I love to read book`.

Reference	I	love	to	read	books
Hypothesis	I	love	to	read	book
Operation	C	C	C	C	**S**

$\text{WER} = (1 + 0 + 0)/5 = 0.20 = 20\%$. ✔

Example 8.2 — A deletion and an insertion (Group 8's slide 7)

Reference `The weather is nice today` ($N = 5$); hypothesis `weather is very nice today`.

Reference	The	weather	is	—	nice	today
Hypothesis	—	weather	is	very	nice	today
Operation	**D**	C	C	**I**	C	C

$\text{WER} = (0 + 1 + 1)/5 = 0.40 = 40\%$. ✔ The dash means "no word". Group 8's takeaway applies: with $N = 5$, each error costs 20 percentage points, so short sentences make WER jumpy.

Example 8.3 — Filling the dynamic-programming table (teaching example)

Reference `turn off the light` (4 words); hypothesis `turn of light` (3 words). Rows are reference prefixes, columns hypothesis prefixes; ∅ means the empty prefix.

Table 8.2 — Minimum edits for every pair of prefixes. Bold cells trace one best path.

	∅	turn	of	light
∅	0	1	2	3
turn	1	**0**	1	2
off	2	1	**1**	2
the	3	2	**2**	2
light	4	3	3	**2**

How three cells were filled:

- Row *turn*, column *turn*: the words match, so take the diagonal $D[0][0] + 0 = 0$.
- Row *off*, column *of*: different words; options are delete $D[1][2] + 1 = 2$, insert $D[2][1] + 1 = 2$, substitute $D[1][1] + 1 = 1$. Minimum **1**.
- Row *light*, column *light*: match, so diagonal $D[3][2] + 0 = 2$.

The bottom-right cell is 2. Tracing back: turn = turn (C), off → of (S), the → nothing (D), light = light (C). $S = 1$, $D = 1$, $I = 0$; $\text{WER} = 2/4 = 50\%$. One wrong letter in "of" turned a command into a different phrase, which shows why WER should be paired with a check of meaning when commands or numbers matter.

Example 8.4 — WER above 100% (teaching example)

Reference `yes` ($N = 1$); hypothesis `yes yes yes`. The best alignment matches one `yes` and inserts two: $\text{WER} = (0 + 0 + 2)/1 = 200\%$. Such cases happen in practice when a system "hallucinates" extra words during silence or repeats itself.

Example 8.5 — Cleanup can decide the result (teaching example)

Reference `Turn off the lights.`; hypothesis `turn off the lights`. Raw WER: `Turn`≠`turn` and `lights.`≠`lights`, so $S = 2$, $N = 4$, WER = 50%. After lowercasing and removing punctuation on *both* sides: WER = 0%. Neither number is "wrong"; they answer different questions. Report the rule.

8.F Python implementation

Package: `jiwer` (`pip install jiwer`; version 4.0.0 used here). **Input format:** reference first, hypothesis second; each is a string or a list of strings (one per sentence). `jiwer.process_words` returns an object with `.wer`, `.substitutions`, `.deletions`, `.insertions` and `.hits`.

```
import jiwer

pairs = [
    ("I love to read books", "I love to read book"),  # Group 8 ex. 1
    ("The weather is nice today", "weather is very nice today"),  # Group 8 ex. 2
    ("the cat sits on the mat", "the cat sit on the mat"),  # Group 8 code
    ("turn off the light", "turn of light"),  # Example 8.3
    ("yes", "yes yes yes"),  # above 100%
]
for ref, hyp in pairs:
    out = jiwer.process_words(ref, hyp)  # reference first, hypothesis second
    print(
        f"{hyp!r:30} S={out.substitutions} D={out.deletions} "
        f"I={out.insertions} WER={out.wer:.4f}"
    )

print(
    jiwer.visualize_alignment(
        jiwer.process_words("The weather is nice today", "weather is very nice today")
    )
)
```

`visualize_alignment` prints the alignment with `*` for missing words and letters `S`, `D`, `I` under each error.

```
# Two different minimum alignments exist here (2 S, or 1 D + 1 I);
# jiwer reports one of them. The total, 2 edits, is the same.
swap = jiwer.process_words("a b", "b a")
print("Swap:", swap.substitutions, swap.deletions, swap.insertions, swap.wer)

# Lists: total errors / total reference words (not an average of rates).
refs = ["a b c d", "e f"]
hyps = ["a b c d", "x y"]
print("Corpus WER:", round(jiwer.wer(refs, hyps), 4))

# Same cleanup applied to reference and hypothesis.
clean = jiwer.Compose(
    [
        jiwer.ToLowerCase(),
        jiwer.RemovePunctuation(),
        jiwer.RemoveMultipleSpaces(),
        jiwer.Strip(),
        jiwer.ReduceToListOfListOfWords(),
    ]
```

```
)
print("Raw WER    :", jiwer.wer("Turn off the lights.", "turn off the lights"))
print(
    "Cleaned WER:",
    jiwer.wer(
        "Turn off the lights.",
        "turn off the lights",
        reference_transform=clean,
        hypothesis_transform=clean,
    ),
)

# jiwer 4 edge case: an empty reference returns the number of insertions.
print("Empty reference:", jiwer.wer("", "a b"))
```

In the corpus example, sentence 1 is perfect and sentence 2 has 2 errors in 2 words (100%). The pooled WER is $(0+2)/(4+2) = 2/6 = 0.3333$, whereas the average of the two sentence WERs would be $(0\% + 100\%)/2 = 50\%$. A custom transform must end with `ReduceToListOfListOfWords()` so jiwer receives lists of words.

Executed output (file `code/ch08_wer.py`):

```
'I love to read book'          S=1 D=0 I=0 WER=0.2000
'weather is very nice today'   S=0 D=1 I=1 WER=0.4000
'the cat sit on the mat'       S=1 D=0 I=0 WER=0.1667
'turn of light'                S=1 D=1 I=0 WER=0.5000
'yes yes yes'                  S=0 D=0 I=2 WER=2.0000
=== SENTENCE 1 ===

REF: The weather is **** nice today
HYP: *** weather is very nice today
       D               I

=== SUMMARY ===
number of sentences: 1
substitutions=0 deletions=1 insertions=1 hits=4

mer=33.33%
wil=36.00%
wip=64.00%
wer=40.00%

Swap: 0 1 1 1.0
Corpus WER: 0.3333
Raw WER    : 0.5
Cleaned WER: 0.0
Empty reference: 2
```

jiwer chose the deletion + insertion alignment for `a b` → `b a`. The summary also prints MER, WIL and WIP, related measures that this guide does not cover. **Configuration choices to report:** library and version, cleanup steps, number normalization, word segmentation (for Khmer), empty-reference rule, and whether WER is pooled over the corpus.

8.G How to interpret results

- **WER 10%** means about one word error per ten reference words, under the stated rules. It does not say *which* words: a wrong name or number may matter more than a dropped "the".
- **Lower WER** on the *same* audio and references, with the *same* rules, supports "fewer word errors". With very small test sets, one error changes WER a lot: in Group 8's 32-word test, each error is $1/32 \approx 3.1$ percentage points.
- Look at **S/D/I separately** for diagnosis (many deletions may mean quiet speech was missed; many insertions may mean noise is being transcribed), keeping in mind that different tools may split ties differently.

8.H Strengths and limitations

Strengths	Limitations (with examples)
Simple, standard for speech recognition	Ignores meaning: `book`/`books` and `off`/`of` count the same (Examples 8.1, 8.3)
Exact and reproducible once rules are fixed	Sensitive to cleanup rules (Example 8.5)
Shows error types (S/D/I)	Can exceed 100% (Example 8.4); "1 − WER" is not a true accuracy
Pooled corpus WER handles different sentence lengths	Penalizes acceptable variants such as synonyms or word order
	Needs word boundaries; Khmer requires segmentation first
	Small tests are noisy (Group 8's own honest note)

8.I When to use it

Use WER for speech-to-text, dictation, subtitles and any task where the output must reproduce a reference word sequence. Pair it with CER (Chapter 9) for languages with complex word boundaries or for spelling-level errors, and with human listening or task-specific checks (names, numbers, commands). For translation or summaries, where many wordings are valid, use BLEU, ROUGE or BERTScore instead.

8.J Common misunderstandings

- *"WER is a percentage, so it is at most 100%."* → **Correction:** insertions are unbounded; WER can be 200% or more (Example 8.4).
- *"WER is divided by the number of words the system produced."* → **Correction:** it is divided by the number of **reference** words.
- *"If two tools give different S/D/I counts, one is wrong."* → **Correction:** several minimum alignments can exist; only the total must agree.
- *"Average the sentence WERs to get the test-set WER."* → **Correction:** pool errors and reference words over the corpus.

8.K Quick recap and practice

Recap. WER = (S + D + I) ÷ N, with the minimum edits found by dynamic programming and N the number of reference words. Lower is better, 0 is perfect, and values above 100% are possible. State cleanup and empty-reference rules, and pool over the corpus.

Practice questions

1. Reference has 12 words; the best alignment has 2 substitutions, 1 deletion and 3 insertions. What is WER?
2. Reference `please call me`; hypothesis `please call me now please`. What is WER?
3. Two sentences: 0 errors in 20 words, and 3 errors in 4 words. Give the corpus WER and the average of the sentence WERs.

Answers

1. $(2 + 1 + 3)/12 = 6/12 = 50\%$.
2. Two insertions, $N = 3$: $2/3 = 66.7\%$.
3. Corpus: $3/24 = 12.5\%$. Average of sentence WERs: $(0\% + 75\%)/2 = 37.5\%$. The corpus figure is the standard one.

Reported experiment by Group 8

Table 8.3 — Group 8's experiment record.

Item	As documented on Group 8's slides 9–10
Systems	Whisper-base ("Model A") and Whisper-tiny ("Model B"), OpenAI's speech-recognition models (Radford et al., 2023); tiny is the smallest released size and base the next
Test data	5 sentences, 32 reference words, "same audio, same text" for both models
Scoring	Align, count S + D + I, WER = errors ÷ 32
Missing	The hypothesis transcripts, the audio source (speaker, recording or synthetic speech), Whisper version, language and decoding settings, text-normalization rules (the references are lowercase and unpunctuated, which suggests normalization), and the date

Table 8.4 — Group 8's reported results (word counts and totals re-checked: $3/32 = 0.09375$, $6/32 = 0.1875$).

#	Reference sentence	Words	Whisper-base	Whisper-tiny
1	the cat sits on the mat	6	0 errors	1 S
2	please turn off the lights	5	1 D	1 S
3	i want to book a flight to siem reap	9	1 I	1 D
4	she sells seashells by the seashore	6	0 errors	1 S + 1 I
5	the meeting starts at nine o'clock	6	1 D	1 S
	Total	**32**	**3 errors → 3/32 = 9.4%**	**6 errors → 6/32 = 18.8%**

Interpretation. On these 32 words, Whisper-base made half as many errors as Whisper-tiny, consistent with Group 8's expectation that the larger model errs less. With only 5 sentences, one speaker setting and no transcripts available, the result illustrates the method; it is not a benchmark (Group 8 said this clearly). This guide could not re-score the outputs because the transcripts were not supplied.

8.L Sources

- Group 8 presentation: `Group8_Word_Error_Rate_WER_13_Slides.pdf` (error types, formula, two examples, jiwer code, Whisper experiment, advantages and limitations).
- jiwer documentation (measures, transformations, alignment visualization): https://jitsi.github.io/jiwer/
- jiwer source code and README (including empty-reference handling in version 4): https://github.com/jitsi/jiwer
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707–710.
- Wagner, R. A., & Fischer, M. J. (1974). The String-to-String Correction Problem. *Journal of the ACM*, 21(1), 168–173. https://doi.org/10.1145/321796.321811
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust Speech Recognition via Large-Scale Weak Supervision (Whisper). *ICML*. https://arxiv.org/abs/2212.04356

Chapter 9: Character Error Rate — CER (Group 9)

9.A Learning goals

By the end of this chapter you should be able to:

- define character substitutions, deletions and insertions and compute CER by hand;
- fill a dynamic-programming table for character edit distance;
- explain why CER divides by the reference length and can exceed 100%;
- state rules for spaces, capital letters, punctuation, Unicode and empty references;
- explain the difference between Unicode code points and the symbols a reader sees, using Khmer; and
- describe Group 9's two-model experiment accurately, including what it does not show.

9.B Plain-English introduction

Question answered: "What is the smallest number of single-character changes needed to turn the model's text into the correct text, relative to the length of the correct text?" Group 9 defined CER as "the smallest number of character edits from reference to prediction, divided by the reference length."

CER is WER (Chapter 8) with **characters instead of words** as the unit. It is used for optical character recognition (OCR: reading text from images), handwriting recognition, speech recognition in languages with unclear word boundaries, spelling correction, and checking exact names, codes or fixed-format LLM outputs.

What it measures: closeness of the *form* of the text, character by character. **What it does not measure:** meaning or truth. Group 9: "CER focuses on the form of the text (characters), not whether two sentences have the same meaning."

9.C Key terms

Table 9.1 — Character error types (Group 9's examples).

Type	Symbol	Meaning	Example (reference → prediction)
Substitution	S	a character is replaced	`cat` → `cut` (a → u)
Deletion	D	a reference character is missing	`plane` → `pane` (l missing)
Insertion	I	an extra character appears	`cat` → `cart` (r added)

- **N (reference length):** the number of characters in the reference *under the chosen rules* (for example, counting spaces and punctuation).
- **Unicode code point:** the number Unicode gives to one encoded character, written like `U+179F`. Python's `len()` counts code points.
- **Grapheme cluster:** a group of code points that software treats as one user-perceived character, defined by Unicode's text-segmentation rules (UAX #29).
- **Normalization (Unicode):** converting text to a standard encoded form, such as NFC, so that sequences Unicode defines as equivalent are stored the same way (UAX #15).

9.D Formula and reasoning

$$\mathrm{CER} = \frac{S + D + I}{N}$$

- **Numerator:** the minimum total number of character edits (the character-level Levenshtein distance). We always take the *fewest* edits, not just the positions that differ: comparing `cat` and `cart` position by position would wrongly report two differences (`t`≠`r`, extra `t`), but one insertion is enough.
- **Denominator:** N, the number of **reference** characters. As with WER, the target is fixed, so every system is measured against the same length, and producing extra text cannot lower the rate.

Range and direction. $\mathrm{CER} \geq 0$; **lower is better**; 0 means an exact match under the chosen rules (it does not prove the reference itself is correct). CER can exceed 100%: reference `a`, prediction `abcd` needs 3 insertions, so $\mathrm{CER} = 3/1 = 300\%$ (Group 9: "CER is an error ratio, not a probability").

Finding the minimum (dynamic programming). Let $D[i][j]$ be the fewest edits to turn the first i reference characters into the first j predicted characters. Start with $D[i][0] = i$ and $D[0][j] = j$, then

$$D[i][j] = \min\Big(D[i-1][j] + 1,\ \ D[i][j-1] + 1,\ \ D[i-1][j-1] + \mathbb{1}[\mathrm{ref}_i \neq \mathrm{pred}_j] \Big)$$

for deletion, insertion, and match/substitution respectively. The answer is $D[N][M]$. Group 9's notebook stores only two rows at a time (`previous_row` and `current_row`) because each row depends only on the row above, which saves memory. The work grows with (reference length × prediction length).

Several minimum alignments. As with WER, different alignments can reach the same minimum with different S/D/I splits (Group 9's notebook and web demo both note this). The total and the CER do not change.

Text rules (Group 9's explicit choices).

Table 9.2 — Group 9's comparison rules, taken from the notebook. These are valid choices, not universal requirements.

Item	Group 9's rule
Capital letters	`C` and `c` are different
Punctuation	kept and counted
Spaces	kept, including leading and trailing spaces
Line breaks	kept and counted
Cleanup	none (no stripping, no lowercasing, no Unicode normalization)
Empty reference	rejected with an error (division by zero)
Empty prediction	allowed: every reference character is deleted (CER = 100%)
Character unit	Python Unicode code points

By contrast, `jiwer.cer` by default **removes leading and trailing spaces**, and jiwer 4.0 has its own rule for empty references. Group 9 configured jiwer to keep all characters so that its results matched their own function exactly.

Code points versus visible symbols, especially for Khmer. One visible Khmer syllable usually contains several code points: a consonant, a subscript consonant written with the invisible sign COENG (U+17D2), and vowel signs. For the Khmer word `សួស្តី` (“hello”):

Table 9.3 — Three ways to count the “characters” of one Khmer word (code verification).

Unit	Count	Pieces
Code points (Python `len`)	6	ស U+179F · ◌ួ U+17BD · ស U+179F · ◌្ U+17D2 · ត U+178F · ◌ី U+17B8
Extended grapheme clusters (UAX #29, `regex` module `\X`)	3	សួ · ស្ · តី
Syllables a Khmer reader sees	2	សួ · ស្តី

So “one character error” can mean different things. If a model misses the subscript `◌្`, that is **2 code-point deletions** (COENG and TA), even though a reader sees one damaged syllable. CER on code points is therefore not directly comparable to CER on grapheme clusters, and Khmer CER values are not directly comparable to English CER values. State the unit. Also note that Unicode normalization (NFC) merges only sequences that Unicode defines as equivalent (for example, `é` as one code point versus `e` + combining accent); it does not repair every difference in typing order, and it is not applied by Group 9’s rules.

Overall CER for several items. Add errors and reference characters first: $\sum(S+D+I)/\sum N$ (Group 9’s notebook, section F4). This is not the same as averaging per-item percentages.

9.E Worked examples

Example 9.1 — Group 9’s slide examples

- `cat` → `cut`: $S=1$, $N=3$: $\mathrm{CER}=1/3=0.3333=33.33\%$.
- `cat` → `cart`: $I=1$, $N=3$: $\mathrm{CER}=1/3=33.33\%$.
- `kitten` → `sitting` (code slide): k→s (S), e→i (S), insert g (I): 3 edits, $N=6$: $\mathrm{CER}=3/6=50.00\%$. ✔ matches the slide output.

Example 9.2 — A full dynamic-programming table (Group 9’s notebook example)

Reference `plant` ($N=5$); prediction `blat`.

Table 9.4 — Edit-distance table for `plant` → `blat` (values printed by the code in 9.F). Bold cells show the best path.

	∅	**b**	**l**	**a**	**t**
∅	0	1	2	3	4
p	1	**1**	2	3	4
l	2	2	**1**	2	3
a	3	3	2	**1**	2
n	4	4	3	**2**	2
t	5	5	4	3	**2**

Tracing the bold path from the top-left: p→b (substitution, 1), l = l (0), a = a (0), n deleted (moving down, +1), t = t (0). Total 2 edits: $S=1$, $D=1$, $I=0$.

$$\mathrm{CER}=\frac{1+1+0}{5}=0.40=40\%$$

Example 9.3 — A low CER that hides a wrong fact (Group 9's teaching example)

Reference `The card is valid for 4 years.` (30 characters); prediction `The card is valid for 5 years.` One substitution: $\mathrm{CER} = 1/30 = 3.33\%$. The text is almost identical, but the stated duration is wrong. CER cannot tell an important character from an unimportant one.

Example 9.4 — Unicode representation (teaching example)

`é` can be stored as one code point (U+00E9) or as two (`e` + U+0301 COMBINING ACUTE ACCENT). They look the same, but raw CER between them is $2/1 = 200\%$ (one substitution and one insertion against a 1-character reference). After NFC normalization of both, CER = 0. Whether to normalize is a rule to state; Group 9 chose not to.

9.F Python implementation

Packages: `jiwer` (for comparison) and `regex` (for grapheme clusters): `pip install jiwer regex`. The two core functions are copied from Group 9's notebook `CER_Evaluation.ipynb` (comments shortened). **Input format:** two Python strings; reference first.

```
import unicodedata

import jiwer
import regex  # third-party module; supports \X (grapheme clusters)


def character_distance(reference, prediction):
    """Minimum character edits (Levenshtein distance), two-row version."""
    previous_row = list(range(len(prediction) + 1))
    for i, reference_char in enumerate(reference, start=1):
        current_row = [i]
        for j, prediction_char in enumerate(prediction, start=1):
            replacement_cost = 0 if reference_char == prediction_char else 1
            current_row.append(
                min(
                    previous_row[j] + 1,  # deletion
                    current_row[j - 1] + 1,  # insertion
                    previous_row[j - 1] + replacement_cost,
                )
            )
        previous_row = current_row
    return previous_row[-1]


def calculate_cer(reference, prediction):
    """CER as a ratio; empty references are rejected (Group 9's rule)."""
    if not isinstance(reference, str) or not isinstance(prediction, str):
        raise TypeError("Both inputs must be text.")
    if len(reference) == 0:
        raise ValueError("Reference must contain at least one character.")
    return character_distance(reference, prediction) / len(reference)
```

`previous_row` starts as `[0, 1, …, M]` (the first table row). For each reference character, `current_row` begins with `i` (deleting `i` characters) and each new cell is the minimum of the three moves. Only the final number is returned; to see S, D and I separately, use jiwer as below.

```
def edit_table(reference, prediction):
    """Full table D[i][j] for teaching (Table 9.4)."""
    table = [[0] * (len(prediction) + 1) for _ in range(len(reference) + 1)]
    for i in range(len(reference) + 1):
        table[i][0] = i
```

```
    for j in range(len(prediction) + 1):
        table[0][j] = j
    for i in range(1, len(reference) + 1):
        for j in range(1, len(prediction) + 1):
            cost = 0 if reference[i - 1] == prediction[j - 1] else 1
            table[i][j] = min(
                table[i - 1][j] + 1, table[i][j - 1] + 1, table[i - 1][j - 1] + cost
            )
    return table


for row in edit_table("plant", "blat"):
    print(row)

for ref, pred in [
    ("cat", "cut"),
    ("cat", "cart"),
    ("plant", "blat"),
    ("kitten", "sitting"),
    ("a", "abcd"),
    ("The card is valid for 4 years.", "The card is valid for 5 years."),
]:
    print(f"{ref!r:34} -> {pred!r:34} CER={calculate_cer(ref, pred):.2%}")
```

```
# Group 9's stored answers (Claude and Gemini gave identical answers).
tasks = {
    "The cat is sleeping.": "The cat is sleeping.",
    "I drink water every day.": "I drink water every day.",
}
errors = sum(character_distance(r, p) for r, p in tasks.items())
chars = sum(len(r) for r in tasks)
print(f"Overall: {errors} errors / {chars} characters = {errors / chars:.2%}")

# Code points versus visible clusters in Khmer.
word = "ស្តី"
print("Code points:", len(word), [f"U+{ord(c):04X}" for c in word])
print("Grapheme clusters (\\X):", regex.findall(r"\X", word))

# The same visible letter can be stored in two ways.
composed, decomposed = "é", "é"
print("Raw CER:", calculate_cer(composed, decomposed))
print(
    "CER after NFC:",
    calculate_cer(
        unicodedata.normalize("NFC", composed), unicodedata.normalize("NFC", decomposed)
    ),
)

# jiwer's default cleanup differs from Group 9's rules.
print("jiwer default strips outer spaces:", jiwer.cer("cat", " cat"))
detail = jiwer.process_characters("plant", "blat")
print("jiwer S/D/I:", detail.substitutions, detail.deletions, detail.insertions)
```

Executed output (file `code/ch09_cer.py`):

```
[0, 1, 2, 3, 4]
[1, 1, 2, 3, 4]
[2, 2, 1, 2, 3]
[3, 3, 2, 1, 2]
[4, 4, 3, 2, 2]
[5, 5, 4, 3, 2]
'cat'                              -> 'cut'                              CER=33.33%
'cat'                              -> 'cart'                             CER=33.33%
'plant'                            -> 'blat'                             CER=40.00%
'kitten'                           -> 'sitting'                          CER=50.00%
'a'                                -> 'abcd'                             CER=300.00%
'The card is valid for 4 years.'   -> 'The card is valid for 5 years.'   CER=3.33%
Overall: 0 errors / 44 characters = 0.00%
Code points: 6 ['U+179F', 'U+17BD', 'U+179F', 'U+17D2', 'U+178F', 'U+17B8']
Grapheme clusters (\X): ['សួ', 'ស្', 'តី']
Raw CER: 2.0
CER after NFC: 0.0
jiwer default strips outer spaces: 0.0
jiwer S/D/I: 1 1 0
```

With Group 9's rules, `cat` → `cat` is one insertion (33.33%); jiwer's default gives 0.0 because it strips the leading space first. **Configuration choices to report:** character unit (code points or grapheme clusters), case, punctuation, spaces, Unicode normalization, empty-reference rule, and how scores are combined over items.

Additional verification of the group's work. Group 9's notebook contains 13 known-answer checks and a comparison with jiwer 4.0.0 (all passing in its saved outputs, run with Python 3.11.15). Group 9 also built a browser demo (`cer-demo/`); its 21 Node.js tests passed when run for this guide with Node.js 22.23.2 (`node --test cer.test.cjs`). The demo's files were not modified.

9.G How to interpret results

- **CER 5%** means about one character edit per 20 reference characters, under the stated unit and rules.
- **Low CER** supports "the text form is very close to the reference". It does not support "the answer is correct" (Example 9.3) or "the meaning is the same".
- **High CER** can come from real errors, from correct rewording, or from rule differences (spaces, capitals, Unicode form).
- Compare CERs only when the reference texts, unit and cleanup rules are identical.

9.H Strengths and limitations

Strengths	Limitations (with examples)
Simple and exact once rules are fixed	Does not understand meaning; one digit can change a fact (Example 9.3)
Detects small spelling differences (Example 9.1)	Penalizes correct rewordings
No word splitting needed, useful for Khmer	Unit matters: code points vs clusters vs syllables (Table 9.3)
Natural for OCR, handwriting and spelling tasks	Depends on spaces, case, punctuation and Unicode form (Example 9.4)
	Can exceed 100%; needs an empty-reference rule

9.I When to use it

Use CER when exact text form matters: OCR, handwriting, speech transcription (especially for languages without spaces between words, such as Khmer), spelling correction, and fixed strings such as names, codes and identifiers. Report the unit and rules, and pair CER with WER where words are well defined. For open-ended answers, check meaning and facts separately; Group 9's conclusion puts it plainly: "A close text match alone does not prove that an answer is correct."

9.J Common misunderstandings

- *"CER counts the positions where the two texts differ."* → **Correction:** it counts the **minimum** number of edits; one insertion can shift all later letters without causing more errors (`cat` → `cart`).
- *"CER is a probability, so it is at most 100%."* → **Correction:** it is an error ratio and can exceed 100% (`a` → `abcd` = 300%).
- *"A character is what I see on the screen."* → **Correction:** Python counts code points; a Khmer syllable can contain several (Table 9.3).
- *"0% CER means the model is perfect."* → **Correction:** it means an exact match on the tested items under the stated rules — nothing more (see Group 9's experiment below).

9.K Quick recap and practice

Recap. CER = (S + D + I) ÷ N at the character level, using the minimum edits found by dynamic programming and the reference length as N. It is exact and useful for text-form tasks, but its value depends on the character unit and cleanup rules, and it says nothing about meaning.

Practice questions

1. Reference `hello` (5 characters), prediction `helo`. What is CER? What is WER if each is one word?
2. Reference `Phnom Penh`, prediction `phnom penh`, case-sensitive rules. What is CER? (Count the space.)
3. Why can a Khmer CER computed on code points be higher than one computed on grapheme clusters for the same error?

Answers

1. One deletion: $\mathrm{CER} = 1/5 = 20\%$. WER: one wrong word out of one: $1/1 = 100\%$ (Group 10's comparison slide).
2. Two substitutions (P→p, P→p), $N = 10$: $\mathrm{CER} = 2/10 = 20\%$.
3. One visible error can involve several code points (for example, a missing subscript consonant is COENG + consonant = 2 code points but part of one cluster), so the numerator and denominator are counted in different units.

Reported experiment by Group 9 (from the notebook)

Table 9.5 — Group 9's experiment record (verified against the current notebook).

Item	As recorded in `CER_Evaluation.ipynb`
Services and settings	**Claude**, student-reported setting "Sonnet 5.5 Medium" (exact model version not independently checked); **Gemini**, setting "Auto" (an automatic selection mode; no fixed underlying model identified)
Collection method	Manual use of the chat services; no API calls; a new chat per task (instructed, but chat history not stored)
Task	Correct spelling while keeping wording, capitalization and punctuation; return only the corrected sentence
Prompt	"Correct the spelling mistakes in the sentence below. Keep the wording, capitalization, and punctuation unchanged except where needed to fix spelling. Return only the corrected sentence, without explanations." followed by the input
Tested items	Task 1 input `The cat is sleping.` → reference `The cat is sleeping.` (20 characters); Task 2 input `I drink watter every day.` → reference `I drink water every day.` (24 characters)
Untested	Tasks 3–5 were prepared but **not tested**; they have no answers and no scores
Scoring rules	Table 9.2 (strict code-point CER, no cleanup), checked against jiwer 4.0.0
Missing	Collection date, full chat settings, and chat records; invisible characters lost in copying cannot be checked

Table 9.6 — Group 9's results. This guide re-scored the stored answers (9.F output: 0 errors / 44 characters).

Task	Service	Answer (as reported)	S	D	I	N	CER
1	Claude — Sonnet 5.5 Medium	The cat is sleeping.	0	0	0	20	0.00%
1	Gemini — Auto	The cat is sleeping.	0	0	0	20	0.00%
2	Claude — Sonnet 5.5 Medium	I drink water every day.	0	0	0	24	0.00%
2	Gemini — Auto	I drink water every day.	0	0	0	24	0.00%
All	**each service**					**44**	**0 errors → 0%**

Interpretation. Both services matched both references exactly, so both have an overall CER of 0% across 44 reference characters. The experiment **does not establish an overall winner**: two short, simple tasks with one answer each cannot separate the services, the Gemini setting does not identify a fixed model, and repeating the test later may give different answers.

Unverified table on slide 9. Group 9's presentation also shows a results table for three Khmer OCR systems (Tesseract 5 "khm+eng, best": CER 20.55%, lines fully right 16.5%; Netra-OCR: 5.87%, 56.9%; Chakkhu: 4.50%, 58.8%) with the caption "Chakku has the lowest CER in the results shown." This table is **not part of the notebook**, it is not an LLM experiment, and the slides do not state its source, test images, number of lines, text rules or date. It cannot be checked from the supplied materials, so this guide does not treat it as Group 9's experiment and draws no conclusion from it. (The table spells the system "Chakkhu" and the caption "Chakku".)

9.L Sources

- Group 9 presentation: `Group9_CER_Presentation.pptx.pdf` (definition, formula, error types, examples, jiwer code, experimental setup, results slide, limitations).
- Group 9 notebook: `CER_Evaluation.ipynb` (rules, functions, known-answer checks, jiwer comparison, experiment record and results).
- Group 9 browser demo: `cer-demo/` (README, `cer.js`, `cer.test.cjs`).
- jiwer documentation, character processing: https://jitsi.github.io/jiwer/reference/process/; transformations (default whitespace stripping): https://jitsi.github.io/jiwer/reference/transformations/
- jiwer README, note on empty references: https://github.com/jitsi/jiwer#a-note-on-empty-references
- Python documentation, Unicode HOWTO: https://docs.python.org/3/howto/unicode.html
- Unicode Standard Annex #29, Unicode Text Segmentation: https://www.unicode.org/reports/tr29/
- Unicode Standard Annex #15, Unicode Normalization Forms: https://www.unicode.org/reports/tr15/
- `regex` module (supports `\X`): https://pypi.org/project/regex/
- Levenshtein (1966) and Wagner & Fischer (1974), as in Chapter 8.

Chapter 10: Comparative Evaluation and Choosing Methods (Group 10)

10.1 Learning goals

By the end of this chapter you should be able to:

- group the nine metric topics into families by what they compare;
- compare the methods by unit, purpose, reference needs, score direction, tasks, limits and settings;
- choose a sensible set of methods for common tasks, including Khmer-language outputs;
- explain, with real numbers, why different methods disagree on the same answer;
- combine automatic metrics with human checks; and
- explain why results from different groups' experiments cannot be used to rank models.

10.2 Plain-English introduction

Group 10 asked the central question of the whole activity: "Which evaluation metric should we use for which NLP task—and why?" Their starting example shows the problem: reference `Paris`, prediction `The capital is Paris.` A person sees a correct answer; Exact Match gives 0; a word-overlap score gives partial credit; a meaning-based score gives a high value. None of them is "the" right number. Each answers a different question, and Group 10 named three questions that evaluation must address: **correctness** (is it right?), **similarity** (how close is it to a reference?), and **reliability** (can the metric detect meaning changes?).

This chapter builds on Group 10's comparison slides, corrects and extends them with the findings of Chapters 1–9, and adds practical selection advice.

10.3 Four families of methods

Group 10 proposed a useful grouping (their slide 3), which this guide keeps:

Table 10.1 — Group 10's four perspectives. Precision, Recall and F1 are one topic but three separate scores.

Family	Methods	What they compare
1. Exact / label-based correctness	Exact Match, Accuracy, Precision/Recall/F1	whole answers or labels against ground truth
2. Lexical (word) similarity	BLEU, ROUGE, METEOR	shared words, n-grams, subsequences, alignments
3. Semantic similarity	BERTScore	contextual token embeddings
4. Edit-based errors	WER, CER	minimum insertions, deletions, substitutions

Two of Group 10's pairwise comparisons are worth repeating because they are often confused:

- **Exact Match vs Accuracy.** Same counting formula (correct ÷ N), different judging rule: EM requires identical strings; accuracy uses the task's definition of correct. Reference `Paris`: prediction `Paris` gets EM = 1; `The capital is Paris` gets EM = 0, but could be counted correct by an accuracy rule that extracts the answer.

- **WER vs CER.** Same formula, different unit. Reference `hello`, prediction `helo`: one wrong word out of one gives WER = 100%; one deleted character out of five gives CER = 20%. Both can exceed 100% when there are many insertions.

10.4 Comprehensive comparison table

The table is split into two parts so that it fits the page; rows are in the same order in both parts.

Table 10.2a — Comparison of the methods (part 1 of 2).

Method	Main comparison unit	What it measures	Reference requirements	Score direction and range
Exact Match	whole answer string	exact agreement with an accepted answer	one or more accepted answers per item	higher better; 0–1
Accuracy	label or judged answer	share of correct decisions	gold label or a correctness rule per item	higher better; 0–1
Precision / Recall / F1	labels per class (or answer tokens)	false alarms vs misses for a chosen class	gold labels; a stated positive class	higher better; 0–1 each
BLEU	n-grams (1–4)	clipped n-gram precision with brevity penalty	one or more reference texts; corpus-level	higher better; 0–1 (often 0–100)
ROUGE	n-grams, LCS, skip-bigrams	overlap, originally recall-oriented	one or more reference summaries	higher better; 0–1
METEOR	aligned words (exact/stem/synonym)	Recall-weighted match with order penalty	references; language resources	higher better; 0–1
BERTScore	subword tokens as contextual vectors	soft semantic token similarity (P/R/F1)	references; a suitable pretrained model	higher better; raw cosine, mostly high; rescaled typically 0–1
WER	words	minimum word edits ÷ reference words	exact reference transcript; word boundaries	lower better; 0 upward, can exceed 100%
CER	characters (code points or clusters)	minimum character edits ÷ reference characters	exact reference text; a stated character unit	lower better; 0 upward, can exceed 100%

Table 10.2b — Comparison of the methods (part 2 of 2).

Method	Suitable tasks	Main limitations	Important configuration choices
Exact Match	short-answer QA, numbers, names, multiple-choice letters	rejects valid paraphrases; no partial credit	normalization rules; multiple references; answer extraction
Accuracy	balanced classification, multiple choice	misleading under imbalance; hides error types	correctness rule; label mapping; baseline
Precision / Recall / F1	imbalanced classification, detection, extraction, QA token overlap	depend on positive class and averaging; ignore TN	positive class; macro/micro/weighted; `zero_division`
BLEU	machine translation (corpus level)	misses paraphrases; unstable per sentence	tokenizer; case; references; smoothing; signature
ROUGE	summarization	surface overlap only; recall favours length	variant; stemming; tokenizer; sentence splitting
METEOR	translation, short generated text (resource-rich languages)	depends on resources, version, tokenization	version; α, β, γ; modules; tokenization

Method	Suitable tasks	Main limitations	Important configuration choices
BERTScore	paraphrase, summarization, translation, open answers	not a fact or negation check; model-dependent	model; layer; IDF; rescaling; language support
WER	speech transcription, dictation, subtitles	ignores meaning; needs word segmentation	cleanup; number handling; segmentation; empty reference
CER	OCR, handwriting, spelling, Khmer transcription, codes	ignores meaning; unit-dependent	character unit; case; spaces; Unicode normalization

10.5 Practical selection examples

The right choice follows from three questions: *What form does the output take? What counts as correct? Which mistakes are costly?* The suggestions below build on Group 10's task table; none of them is the only valid choice.

Multiple-choice questions. The output is one option letter. Extract the letter with a fixed rule (and count unreadable answers as wrong, stating this), then report **accuracy**, compared with random guessing ($1/k$). EM on the letter is the same thing. Report the extraction rule.

Sentiment classification. Report **accuracy** and the **confusion matrix**; if classes are imbalanced or one error is worse, add **Precision, Recall and F1** for the important class, plus macro F1 across classes. Check a sample of disputed labels (Group 2's "It was okay." example shows that labels themselves can be debatable).

Translation. Use corpus-level **BLEU** with sacreBLEU and its signature, plus a meaning-sensitive measure (**BERTScore** with a model that supports the language, or **METEOR** in English), plus human review of a sample for adequacy and fluency. Group 10 recommended "BLEU + METEOR / BERTScore".

Summarization. Use **ROUGE-1/2/L(sum)** for content overlap and **BERTScore** for meaning similarity, *and* a factual-consistency check (human or task-specific), because summaries can be fluent and overlapping yet unfaithful. Group 10: "ROUGE + BERTScore + factuality".

Speech transcription. Use pooled **WER** with stated normalization; add **CER** for spelling-level errors or languages with unclear word boundaries; listen to samples, especially for names, numbers and commands.

Text recognition from images (OCR). Use **CER** as the main score, **WER** where words are well defined, and an exact-match check on critical fields (dates, ID numbers, amounts).

Open-ended factual Q&A. Automatic overlap metrics are weak here because many correct answers exist and small changes can flip the facts. Use **EM and token-F1** only for short answers with clear references; otherwise use **human judgment or a rubric** that checks each required fact, possibly supported by BERTScore to find candidates for review. Check numbers, names and negations explicitly (Section 10.6).

Khmer-language outputs. Khmer is written without spaces between words and uses stacked consonants and vowel signs encoded as several code points. Therefore: (1) for word-based metrics (WER, BLEU, ROUGE, METEOR), first segment the text with a Khmer word segmenter, or use a tokenizer that supports Khmer (for example sacreBLEU's SentencePiece-based `flores200`), and report it; (2) do not use the default `rouge-score` tokenizer, which deletes Khmer letters (Chapter 5, Example 5.6); (3) for **CER**, state whether the unit is code points or grapheme clusters (Chapter 9, Table 9.3); (4) for **BERTScore**, use a model whose tokenizer covers Khmer — the package default for non-English languages maps Khmer to `[UNK]` (Chapter 7) — and note that no rescaling baseline exists for Khmer; (5) METEOR in NLTK has no Khmer stemmer or synonym resource. In all cases, confirm with Khmer-speaking human reviewers.

10.6 Why methods can disagree

Methods disagree because they count different things. The table below scores four candidates against one reference with seven metrics (teaching example; code verification, script in 10.9).

Reference: `The medicine is safe for children.`

Table 10.3 — Seven metrics on four candidates. Settings: raw string EM; sacreBLEU sentence BLEU (13a, case-sensitive, exp smoothing); rouge-score ROUGE-L (lowercasing tokenizer, no stemming); NLTK METEOR with punctuation split off; BERTScore `bert-base-uncased` layer 9; jiwer WER and CER with default settings. Higher is better for the first five metrics; lower is better for WER and CER.

Candidate	Text	EM	BLEU	ROUGE-L F1	METEOR	BERTScore F1	WER	CER
A exact copy	The medicine is safe for children.	1	100.00	1.0000	0.9985	1.0000	0.0000	0.0000
B paraphrase	Children can take this medicine safely.	0	7.81	0.1667	0.2857	0.7457	1.0000	0.8824
C negation	The medicine is not safe for children.	0	50.00	0.9231	0.9744	0.9362	0.1667	0.1176
D lowercase	the medicine is safe for children	0	64.32	1.0000	0.8676	0.9241	0.3333	0.0588

What the table shows.

- **B (correct paraphrase)** is the *worst* candidate by every metric except EM (where everything except A gets 0). Lexical and edit-based metrics require the same words; even BERTScore ranks B below the dangerous C.
- **C (negation)** reverses the meaning but scores high everywhere except EM: one inserted word is a small change for every counting method.
- **D (lowercase, no full stop)** is harmless, but case-sensitive metrics (EM, BLEU, WER) punish it, while ROUGE-L (which lowercases and drops punctuation) gives a perfect 1.0.
- So the ranking depends on the metric: ROUGE-L ranks A = D > C > B; WER ranks A > C > D > B; no metric ranks the candidates by *safety of meaning* (A ≈ B ≈ D > C).

General reasons for disagreement. Different units (labels, words, characters, subwords); different directions (precision-oriented BLEU vs recall-oriented ROUGE); different cleanup (case, punctuation); different handling of order; different knowledge (none, a synonym dictionary, or a neural model); and different aggregation (corpus vs sentence averages). Disagreement is useful information: it shows *which aspect* of quality differs.

10.7 Combining automatic metrics with human checks

A practical evaluation plan for an LLM task:

1. **Define quality first.** Write down what a good output must do (correct facts, required format, coverage, tone). Choose metrics that measure those properties, not the other way round.
2. **Fix the test set and rules.** Same inputs, references, prompts, model settings and scoring configuration for every system. Record model versions and dates.
3. **Use complementary metrics.** For example, one lexical or edit metric (BLEU/ROUGE/WER/CER), one meaning-based metric (BERTScore with a suitable model), and one correctness measure (EM, accuracy or a fact check).

4. **Add human review.** Review a random sample plus the items where metrics disagree most. Use a short rubric (for example: correct? complete? faithful? fluent? safe?), and, if possible, two reviewers who do not know which system produced which output. Measure how often reviewers agree.
5. **Check critical content directly.** Numbers, names, dates, negations and safety-relevant statements need explicit checks; similarity metrics do not catch them (Table 10.3, candidate C).
6. **Report honestly.** Report sample size, settings, uncertainty (small samples give unstable scores), and limitations. Show examples of successes and failures.

Using another LLM as an automatic judge is increasingly common, but it is a separate method with its own biases and was not part of this activity (Zheng et al., 2023).

10.8 Why the groups' experiments cannot rank models

Four groups reported experiments with real systems. They are summarized together here only to show that they **must not be compared with one another**.

Table 10.4 — Reported experiments (details and missing information in Appendix E).

Group	Systems compared	Task and data	Metric	Reported result
2	Gemini 1.5 vs Llama 3	sentiment of 5 short reviews	accuracy	80% vs 80%
7	Qwen2.5-0.5B-Instruct vs SmolLM2-135M-Instruct	1 paraphrase + 1 summary sentence	BERTScore, BLEU, ROUGE-L	e.g. BERTScore F1 0.9133 vs 1.0000 (paraphrase)
8	Whisper-base vs Whisper-tiny	speech transcription, 5 sentences / 32 words	WER	9.4% vs 18.8%
9	Claude ("Sonnet 5.5 Medium") vs Gemini ("Auto")	spelling correction, 2 sentences / 44 characters	CER	0% vs 0%

Groups 1, 3, 4, 5, 6 and 10 did not report experiments with collected model outputs: Group 1's and Group 6's comparisons use illustrative or sample answers (labelled as such on their slides); Groups 3, 4, 5 and 10 present no experiment results.

Why these numbers cannot be compared. The experiments use different systems (chat services, small open models, speech models), different tasks, different inputs and references, different metrics with different scales and directions (accuracy and BERTScore: higher is better; WER and CER: lower is better), and very small samples (one to five items). An accuracy of 80% and a CER of 0% are not measurements of the same thing. Even within one experiment, the samples are too small to establish a general winner. A fair comparison needs the *same* inputs, references, settings and metric for every system, a larger sample, and recorded model versions and dates.

10.9 Python implementation

Packages: `sacrebleu`, `rouge-score`, `nltk` (+ WordNet data), `jiwer`, `bert-score`, `torch`, `transformers`.
Input format: one reference string and a dictionary of candidate strings. This script produced Table 10.3.

```
import re

import jiwer
import sacrebleu
from bert_score import score as bertscore
from nltk.translate.meteor_score import meteor_score
from rouge_score import rouge_scorer
```

```
reference = "The medicine is safe for children."
candidates = {
    "A exact copy": "The medicine is safe for children.",
    "B paraphrase": "Children can take this medicine safely.",
    "C negation": "The medicine is not safe for children.",
    "D lowercase": "the medicine is safe for children",
}

rouge = rouge_scorer.RougeScorer(["rougeL"])


def words(text):
    """Split words and punctuation for METEOR: "children." -> children, ."""
    return re.findall(r"\w+|[^\w\s]", text)


_, _, bert_f1 = bertscore(
    list(candidates.values()),
    [reference] * len(candidates),
    model_type="bert-base-uncased",
    num_layers=9,
)
```

BERTScore is computed for all candidates in one call (faster than one call per candidate). The other metrics are cheap and computed inside the loop.

```
print("cand          EM  BLEU   ROUGE-L METEOR BERT-F1  WER    CER")
for (name, cand), bf1 in zip(candidates.items(), bert_f1):
    em = int(cand == reference)
    bleu = sacrebleu.sentence_bleu(cand, [reference]).score
    rl = rouge.score(reference, cand)["rougeL"].fmeasure
    met = meteor_score([words(reference)], words(cand))
    wer = jiwer.wer(reference, cand)
    cer = jiwer.cer(reference, cand)
    print(
        f"{name:13} {em}  {bleu:6.2f} {rl:7.4f} {met:6.4f} {bf1:7.4f} "
        f"{wer:6.4f} {cer:6.4f}"
    )
```

Note the different argument orders: `sentence_bleu(candidate, [references])`, `rouge.score(reference, candidate)`, `meteor_score([reference_tokens], candidate_tokens)`, `jiwer.wer(reference, hypothesis)`. Mixing them up silently swaps Precision and Recall or changes the denominator.

Executed output (file `code/ch10_compare.py`; model-loading messages omitted):

```
cand          EM  BLEU   ROUGE-L METEOR BERT-F1  WER    CER
A exact copy  1  100.00  1.0000 0.9985  1.0000 0.0000 0.0000
B paraphrase  0    7.81  0.1667 0.2857  0.7457 1.0000 0.8824
C negation    0   50.00  0.9231 0.9744  0.9362 0.1667 0.1176
D lowercase   0   64.32  1.0000 0.8676  0.9241 0.3333 0.0588
```

Two values checked by hand: candidate C's WER is one inserted word over six reference words, $1/6 = 0.1667$; its CER is four inserted characters (`not` plus a space) over 34 reference characters, $4/34 = 0.1176$.

10.10 Common misunderstandings

- *"There is one best metric for LLMs."* → **Correction:** Group 10's first finding: "No universally best metric." Each metric measures one aspect (Table 10.3).
- *"High similarity means a correct answer."* → **Correction:** Group 10's second finding: "Similarity is not factuality." Candidate C reverses the meaning but scores 0.92–0.97 on ROUGE-L, METEOR and BERTScore.
- *"Using more metrics always helps."* → **Correction:** only metrics that measure different, relevant aspects help; several lexical metrics mostly repeat the same information.
- *"Scores from different experiments can be lined up into a ranking."* → **Correction:** only scores from the same test set, references, settings and metric are comparable (Section 10.8).

10.11 Quick recap and practice

Recap. Methods fall into four families: exact/label correctness, lexical similarity, semantic similarity, and edit-based errors. Choose by output type, correctness definition and error costs; combine complementary metrics; add human and factual checks; report all settings; and never rank systems using scores from different experiments.

Practice questions

1. Which two metrics would you report for a Khmer OCR system, and what must you state about each?
2. A summarizer gets ROUGE-L 0.45 and BERTScore F1 0.91. What extra check is still needed before calling the summaries good?
3. Why does EM give 0 to `The capital is Paris.` while an accuracy rule might give 1?

Answers

1. CER (state the character unit — code points or grapheme clusters — and the cleanup and Unicode rules) and WER (state the Khmer word segmenter used). An exact-match check on critical fields is also useful.
2. A factual-consistency (faithfulness) check, by humans or a task-specific method, because neither ROUGE nor BERTScore detects invented or reversed facts.
3. EM compares whole strings, which differ. Accuracy uses the task's correctness rule; if the rule extracts the answer entity or a human judges correctness, the answer counts as correct.

10.12 Sources

- Group 10 presentation: `Group10_Comparative_LLM_Evaluation (2).pdf` (evaluation problem, four perspectives, pairwise comparisons, overall table, task table, three findings, references).
- Papineni et al. (2002); Lin (2004); Banerjee & Lavie (2005); Zhang et al. (2020); Rajpurkar et al. (2016); jiwer documentation — as cited in Chapters 1–9 and listed in Appendix F.
- Zheng, L., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*. https://arxiv.org/abs/2306.05685
- Maynez et al. (2020), On Faithfulness and Factuality in Abstractive Summarization: https://aclanthology.org/2020.acl-main.173/

Conclusion: Main Lessons

1. **Every metric answers one narrow question.** EM asks “identical?”, accuracy “judged correct?”, Precision/Recall “which errors?”, BLEU/ROUGE/METEOR “which words overlap?”, BERTScore “how similar are the token meanings?”, WER/CER “how many edits?”. None asks “is this true and useful?” by itself.
2. **Formulas are simple; settings are not.** Tokenization, case, punctuation, stemming, smoothing, model choice, character unit and empty-reference rules changed scores throughout this guide — sometimes from 0.54 to 0.84 (METEOR, Example 6.3) or from 0 to 1 (ROUGE on Khmer, Example 5.6). Always report them.
3. **Ratios need context.** Compare accuracy with a baseline, Precision with the number of predictions, and BLEU only with the same signature. Error rates (WER, CER) can exceed 100%.
4. **Similarity is not correctness.** A negated sentence scored above 0.92 on three similarity metrics (Table 10.3). Facts, numbers and negations need direct checks.
5. **Reference quality limits every metric.** A wrong or narrow reference punishes correct outputs (Group 2’s “It was okay.” label; Group 1’s `Four` vs `4`).
6. **Language matters.** Khmer needs word segmentation for word-based metrics, a stated character unit for CER, and a model that actually tokenizes Khmer for BERTScore.
7. **Small experiments illustrate; they do not rank.** The four reported experiments (Groups 2, 7, 8, 9) each used one to five items; they demonstrate the methods but cannot establish general winners, and they cannot be compared with each other.
8. **Good evaluation is a combination:** clear quality criteria, a fixed test set, complementary automatic metrics, targeted human review, and honest reporting.

Appendix A: Formula Reference Sheet

Table A.1 — Formula reference sheet.

Method	Formula	Range; better
Exact Match	$\mathrm{EM} = \frac{1}{N}\sum_i \max_k \mathbb{1}[\mathrm{norm}(P_i) = \mathrm{norm}(R_{i,k})]$	0–1; higher
Accuracy	$\frac{\text{correct}}{N} = \frac{TP+TN}{TP+TN+FP+FN}$	0–1; higher
Precision	$P = \frac{TP}{TP+FP}$	0–1; higher
Recall	$R = \frac{TP}{TP+FN}$	0–1; higher
F1	$F_1 = \frac{2PR}{P+R} = \frac{2TP}{2TP+FP+FN}$	0–1; higher
F_β	$\frac{(1+\beta^2)PR}{\beta^2 P+R}$	0–1; higher
BLEU clipped precision	$p_n = \frac{\sum_g \min(\mathrm{Count}(g), \mathrm{MaxRef}(g))}{\sum_g \mathrm{Count}(g)}$	0–1
Brevity penalty	$\mathrm{BP} = 1$ if $c > r$, else $e^{1-r/c}$	0–1
BLEU	$\mathrm{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$, usually $N=4$, $w_n = 1/4$	0–1 (×100); higher
ROUGE-N recall	$\frac{\sum_{g \in \mathrm{ref}} \min(\mathrm{Count}_{\mathrm{cand}}(g), \mathrm{Count}_{\mathrm{ref}}(g))}{\sum_{g \in \mathrm{ref}} \mathrm{Count}_{\mathrm{ref}}(g)}$	0–1; higher
ROUGE-L	$R = \frac{\mathrm{LCS}}{m}$, $P = \frac{\mathrm{LCS}}{n}$, $F = \frac{(1+\beta^2)RP}{R+\beta^2 P}$	0–1; higher
ROUGE-W	$R = f^{-1}\left(\frac{\mathrm{WLCS}}{f(m)}\right)$, e.g. $f(k) = k^2$	0–1; higher
ROUGE-S	$R = \frac{\mathrm{SKIP2}}{\binom{m}{2}}$, $P = \frac{\mathrm{SKIP2}}{\binom{n}{2}}$	0–1; higher
METEOR (2005)	$F_{\mathrm{mean}} = \frac{10PR}{R+9P}$; $\mathrm{Pen} = 0.5\left(\frac{ch}{m}\right)^3$; score $= F_{\mathrm{mean}}(1-\mathrm{Pen})$	0–1; higher
BERTScore	$R = \frac{1}{k}\sum_i \max_j \boldsymbol{x}_i^\top \widehat{\boldsymbol{x}}_j$, $P = \frac{1}{l}\sum_j \max_i \boldsymbol{x}_i^\top \widehat{\boldsymbol{x}}_j$, $F = \frac{2PR}{P+R}$	raw ≤ 1; higher
BERTScore rescaled	$\widehat{s} = \frac{s-b}{1-b}$	typically 0–1; higher
WER	$\frac{S+D+I}{N_{\mathrm{words}}}$	≥ 0; lower
CER	$\frac{S+D+I}{N_{\mathrm{chars}}}$	≥ 0; lower
Edit-distance recurrence	$D[i][j] = \min(D[i-1][j]+1,\ D[i][j-1]+1,\ D[i-1][j-1] + \mathbb{1}[a_i \neq b_j])$	—
Corpus error rate	$\frac{\sum(S+D+I)}{\sum N}$	≥ 0; lower

Appendix B: Glossary

Table B.1 — Glossary.

Term	Basic explanation
Alignment	A pairing of the words or characters of two texts that shows which match and which differ
Baseline (classification)	A simple reference model, such as always predicting the most common class
Baseline rescaling (BERTScore)	Linearly stretching scores using the typical score of unrelated sentence pairs
Brevity penalty	BLEU's factor that lowers the score of candidates shorter than the reference
Candidate / hypothesis / prediction	The model output being evaluated
Chunk (METEOR)	A group of matched words that are adjacent and in the same order in both texts
Class imbalance	Some classes are much more frequent than others
Clipping	Limiting n-gram credit to the number of times the n-gram appears in a reference
Code point	The number Unicode assigns to one encoded character, such as U+179F
Confusion matrix	A table counting predictions for each pair of actual and predicted classes
Contextual embedding	A token vector that depends on the surrounding words
Corpus-level score	A score computed by pooling counts over the whole test set
Cosine similarity	A measure of how closely two vectors point in the same direction, from −1 to 1
Deletion	An edit where a reference unit is missing from the output
Dynamic programming	Solving a problem by filling a table of answers to smaller sub-problems and reusing them
Edit distance (Levenshtein distance)	The minimum number of insertions, deletions and substitutions between two sequences
Embedding	A vector of numbers representing a token or text
False negative (FN)	An actual positive item predicted as negative (a miss)
False positive (FP)	An actual negative item predicted as positive (a false alarm)
Fragmentation penalty	METEOR's penalty for matches that are split into many chunks
Geometric mean	The k-th root of the product of k numbers
Grapheme cluster	A group of code points treated as one user-perceived character by Unicode rules
Harmonic mean	An average of rates that stays close to the smallest value; used in F1
IDF (inverse document frequency)	A weight that is higher for rare words
Indicator function	A function that gives 1 when a condition is true and 0 otherwise
Insertion	An edit where an extra unit appears in the output
LCS (longest common subsequence)	The longest sequence of units that appears in both texts in the same order, gaps allowed
Logarithm	The inverse of exponentiation; turns products into sums
Macro / micro / weighted average	Ways to combine per-class scores: equal per class, equal per item, or weighted by class size
n-gram	A run of n consecutive tokens

Term	Basic explanation
Normalization (text)	Cleanup applied to both texts before comparison (lowercasing, removing punctuation, etc.)
Normalization (Unicode)	Converting text to a standard encoded form such as NFC
OCR	Optical character recognition: reading text from images
Paraphrase	A different wording with the same meaning
Positive class	The class that Precision, Recall and F1 are computed for
Precision	Share of produced or predicted items that are correct
Recall	Share of the items that should be found that were found
Reference / ground truth	The accepted correct output
Skip-bigram	An ordered pair of words from a sentence with any gap between them
Smoothing (BLEU)	Adding small values to zero counts so sentence-level BLEU is not zero
Stemming	Reducing words to a rough root form, e.g. `running` → `run`
Substitution	An edit where a reference unit is replaced by a different unit
Subword token	A piece of a word used by neural-model tokenizers
Support	The number of items that truly belong to a class
Token / tokenization	A unit of text counted by a metric / the process of cutting text into tokens
True negative (TN) / true positive (TP)	Correctly predicted negative / positive items
WordNet	A large English lexical database of word senses and synonyms

Appendix C: Answers to Review Questions

All answers appear directly below the questions in each chapter's recap section: 1.K, 2.K, 3.K, 4.K, 5.K, 6.K, 7.K, 8.K, 9.K and 10.11.

Appendix D: Source-to-Chapter Credits

Each chapter is based on one group's presentation. Member names are taken from the official list `Activity_12A-Nov-Group_member_Friday.pdf` (71 students in 10 groups). Where a presentation's own member list differs from the official list, the difference is noted.

Table D.1 — Group credits.

Chapter	Group	Topic	Source file(s) in `Course/12A-Nov26-Activity/`	Members (official list)
1	1	Exact Match	`Group1_Exact_Match_EM_Evaluation_for_Large_Language_Models_no_gamma.pptx`	LENG DEVID, Doeun Bunheng, Tieng Sopanha, SOPHON RACHANA, CHHENG Sothean, Phan Wayotep, DUK PANGARITH
2	2	Accuracy	`Group2_Presentation (1).pdf`	DIN Reaksa, Ngen Tina, KHAY LINA, Monsreylin, Yong Lyhor, PHAL MENGHAK, KEM Sreyoun
3	3	Precision, Recall, F1	`Group3_Precision_Recall_F1_Research_Presentation.pptx.pdf`	Vandeth Sothy, CHHEANG PHANY, LENG Vicheka, Yory Sreykhuoch, ORN VANDA, CHEA Sophaktra, Sroy Liza
4	4	BLEU	`Group4_BLEU_Score_10_Slides (1).pptx`	LOT SOKLANG, NANG CHETTRA, Yos Nisiy, HEM BELLYDAY, HEANG PISETH, SOM Chann Reaksmey, HOK LYHOUR
5	5	ROUGE	`Group5_ROUGE SCORE Group5.pdf`	HANG MUYKHORNG, NY Chantharith, KE MANUT, CHOU KOSOL, YENG Kheang, SOU PICHCHOMRONG, KUONG Sivhoung
6	6	METEOR	`Group6_METEOR Score.pdf`	SIENG Lyheng, SROEUN BUNNARITH, MIN Sivee, NOV Panhavath, SOR RAKSMEY, LUN Rathana, Ry Chhorly
7	7	BERTScore	`Group7_BERTScore - Group 07.pptx`	TAING KORNGMENG, HEANG Denis, SOPHAT Vitou, CHHOUK PHALTHUNIN, Heng Menghong, SAMBATH Seakty, HOEURN SREYKA
8	8	WER	`Group8_Word_Error_Rate_WER_13_Slides.pdf`	Ly Chungheang, LY Chhaythean, HUN SOPHEAK, Mony Sothyra, SEOUK Bondol, Kong Sattha, NEANG Vanna
9	9	CER	`Group9_CER_Presentation.pptx.pdf`, `CER_Evaluation.ipynb`, `cer-demo/`	CHHO Sengmeng, SOUR Alya, CHOUB Botumraksa, Phyrun Pichchhorda, HEAM Visal, Nob Sreynich, HUON SOPANHA
10	10	Comparative evaluation	`Group10_Comparative_LLM_Evaluation (2).pdf`	Nhen Theary, CHIV Mengchou, SONG Vesodth, CHHEANG Udomveasna, NGO Seakyarith, HEANG Sotheara, Prim Veasna, NOB Sreyka

Notes on member lists. Group 1's title slides list five members (CHHENG Sothean, Duk Pagnarith, DOEUN Bunheng, SOPHON Rachana, PHAN WAYOTEP); LENG DEVID and Tieng Sopanha appear only in the official list. Group 5's slides show the names of four members on the team slide. Group 10's title slide lists SUM Devith, who is not in the official Group 10 list, and does not list NOB Sreyka. Spellings differ slightly between files (for example "HUON"/"HOUN" Sopanha, "Duk Pagnarith"/"DUK PANGARITH"). **[PLACEHOLDER: confirm final member lists and spellings with the lecturer.]**

Other sources. The teacher's assignment `Course/Activity_12A-Nov-Ch3-ADV_Programming_Friday.pdf` defines the ten topics and requirements. No separate written CER report file (other than the notebook and slides) was found in the activity folder; the notebook serves as Group 9's written report.

Appendix E: Experiments, Missing Information and Corrections

E.1 Evidence categories used in this guide

Table E.1 — Evidence categories.

Category	Where it appears	Status
Teaching examples	every chapter (labelled "teaching example")	invented for explanation; not model outputs
Group slide examples	Chapters 1–10 (labelled with the group)	illustrative unless stated otherwise by the group
Code verification	all "Executed output" blocks; Tables 1.2, 3.4, 4.3, 5.1, 7.3, 9.3, 10.3	run for this guide on 13 November 2026 (environment in "How to Use This Report")
Reported experiments	Chapters 2, 7, 8, 9 (and Table 10.4)	collected by the groups; not re-collected for this guide

E.2 Reported experiments: what is documented and what is missing

Table E.2 — Reported experiments.

Group	Documented	Missing	Re-checked in this guide
2 (Accuracy)	Model names "Gemini 1.5", "Llama 3"; 5 reviews; gold labels; predictions; 80% each	Model variants/sizes, prompt, date, decoding settings, label-mapping rule; one label ("It was okay." = Positive) is debatable	Totals recounted (4/5 each)
7 (BERTScore)	Model IDs; outputs; reference sentences; scoring configuration; results table	Run date; versions; which generation method (llama.cpp Q4_K_M vs Transformers pipeline — slides disagree); full Example 2 prompt; `compare_metrics.py`	Reported outputs **re-scored**: all BERTScore, BLEU and ROUGE-L values reproduced exactly; outputs not regenerated
8 (WER)	Systems; 5 reference sentences (32 words); per-sentence error types; totals	Hypothesis transcripts; audio source; Whisper version and settings; normalization rules; date	Word counts and percentages recomputed (3/32, 6/32); cannot re-score without transcripts
9 (CER)	Services and reported settings; prompt; inputs; references; answers; rules; checks; results	Collection date; full chat settings; chat records; exact Gemini model	Stored answers **re-scored**: 0 errors / 44 characters each; notebook details verified

Not reported. Groups 1, 3, 4, 5, 6 and 10 present no experiments with collected model outputs. Group 1's experiment slide states "Illustrative predictions, not results from actual API calls." Group 6's states "Teaching demo: these are example answers, not verified live LLM outputs." Group 3's slides mention evaluating "both LLMs" but contain no results. Groups 4, 5 and 10 have no experiment slides. This guide does not invent any missing results.

Unverified material. Group 9's slide 9 shows Khmer OCR results (Tesseract 5, Netra-OCR, Chakkhu) without a source, dataset or settings; it is not used as evidence (Chapter 9).

E.3 Corrections and clarifications to the slides

Table E.3 — Corrections log.

Group, slide	Slide statement	Correction or clarification	Chapter
2, Python	sklearn snippet uses `y_true`, `y_pred` but defines `refs`, `preds`; comment "66.6%"	`NameError` as written; correct output prints 66.67% (2/3)	2.F
2, Introduction	"how many were perfectly correct?" / "strict classification"	Accuracy uses the task's correctness rule; identical wording is EM's rule	2.D
3, slide 6	"High Precision means fewer false positive predictions"	True only as a proportion; higher Precision can coexist with more FPs	3.E (Ex. 3.3)
3	Experiment "both LLMs"	No results in the supplied file	3.K
6, alignment slide	4/4 words matched, including `glad≈happy`	NLTK 3.10.4 matches 3/4 (`glad–happy` fails after stemming); score 0.7361	6.E (Ex. 6.4)
6, experiment	Model B = 0.536	Reproduced only with space-split tokens; 0.8406 with punctuation split	6.E (Ex. 6.3)
6, slide 4	"METEOR = matching quality − word order penalty"	The penalty multiplies: $F_{\text{mean}}(1 - \text{Pen})$ (the next slide is correct)	6.J
7, slides 9 vs 11	llama.cpp Q4_K_M vs Transformers pipeline	Generation method unclear; scores re-checked from saved outputs only	7.K
7, slide 6	Toy matrix F1 ≈ 0.86	Correct for the invented values; real `bert-base-uncased` F1 for that pair is 0.7491	7.E
8, results	"accuracy 90.6%"	This is $1 - \text{WER}$, not a true accuracy; it can be negative	8.D
9, slide 9	OCR table, "Chakku has the lowest CER"	Source not documented; not the notebook's experiment; name spelled two ways	9.K
10, title	Member list includes SUM Devith	Differs from the official Group 10 list	App. D

Appendix F: References

Course materials

- Teacher's assignment: *Group Discussion Assignment: Evaluation Methods for Large Language Models (LLMs)*, `Course/Activity_12A-Nov-Ch3-ADV_Programming_Friday.pdf`.
- Group list: `Course/Activity_12A-Nov-Group_member_Friday.pdf`.
- Group presentations, notebook and demo: see Appendix D.

Papers

- Banerjee, S., & Lavie, A. (2005). METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, 65–72. https://aclanthology.org/W05-0909/
- Callison-Burch, C., Osborne, M., & Koehn, P. (2006). Re-evaluating the Role of BLEU in Machine Translation Research. *EACL*. https://aclanthology.org/E06-1032/
- Chen, B., & Cherry, C. (2014). A Systematic Comparison of Smoothing Techniques for Sentence-Level BLEU. *WMT*. https://aclanthology.org/W14-3346/
- Conneau, A., et al. (2020). Unsupervised Cross-lingual Representation Learning at Scale. *ACL*. https://aclanthology.org/2020.acl-main.747/
- Denkowski, M., & Lavie, A. (2014). Meteor Universal: Language Specific Translation Evaluation for Any Target Language. *WMT*. https://aclanthology.org/W14-3348/
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL*. https://aclanthology.org/N19-1423/
- Lavie, A., & Agarwal, A. (2007). METEOR: An Automatic Metric for MT Evaluation with High Levels of Correlation with Human Judgments. *WMT*. https://aclanthology.org/W07-0734/
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707–710.
- Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. *Text Summarization Branches Out*. https://aclanthology.org/W04-1013/
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*, Section 8.3. Cambridge University Press. https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-of-unranked-retrieval-sets-1.html
- Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On Faithfulness and Factuality in Abstractive Summarization. *ACL*. https://aclanthology.org/2020.acl-main.173/
- Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. *ACL*. https://aclanthology.org/P02-1040/
- Post, M. (2018). A Call for Clarity in Reporting BLEU Scores. *WMT*. https://aclanthology.org/W18-6319/
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust Speech Recognition via Large-Scale Weak Supervision. *ICML*. https://arxiv.org/abs/2212.04356
- Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. *EMNLP*. https://aclanthology.org/D16-1264/
- Reiter, E. (2018). A Structured Review of the Validity of BLEU. *Computational Linguistics*, 44(3), 393–401. https://aclanthology.org/J18-3002/

- Wagner, R. A., & Fischer, M. J. (1974). The String-to-String Correction Problem. *Journal of the ACM*, 21(1), 168–173. https://doi.org/10.1145/321796.321811
- Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating Text Generation with BERT. *ICLR*. https://arxiv.org/abs/1904.09675
- Zheng, L., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks Track*. https://arxiv.org/abs/2306.05685

Software documentation and source code

- bert-score: https://github.com/Tiiiger/bert_score; baseline rescaling: https://github.com/Tiiiger/bert_score/blob/master/journal/rescale_baseline.md
- Hugging Face *evaluate*, SQuAD scoring code: https://github.com/huggingface/evaluate/blob/main/metrics/squad/compute_score.py
- jiwer documentation: https://jitsi.github.io/jiwer/; process API: https://jitsi.github.io/jiwer/reference/process/; transformations: https://jitsi.github.io/jiwer/reference/transformations/; source and README: https://github.com/jitsi/jiwer
- NLTK `bleu_score`: https://www.nltk.org/api/nltk.translate.bleu_score.html; `meteor_score`: https://www.nltk.org/api/nltk.translate.meteor_score.html; source: https://github.com/nltk/nltk/blob/develop/nltk/translate/meteor_score.py
- Python `string` module: https://docs.python.org/3/library/string.html; Unicode HOWTO: https://docs.python.org/3/howto/unicode.html
- `regex` module: https://pypi.org/project/regex/
- rouge-score: https://github.com/google-research/google-research/tree/master/rouge; https://pypi.org/project/rouge-score/
- sacreBLEU: https://github.com/mjpost/sacrebleu
- scikit-learn model evaluation guide: https://scikit-learn.org/stable/modules/model_evaluation.html; `accuracy_score`: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html; `precision_recall_fscore_support`: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_recall_fscore_support.html
- Unicode Standard Annex #15 (Normalization Forms): https://www.unicode.org/reports/tr15/; Annex #29 (Text Segmentation): https://www.unicode.org/reports/tr29/
- WordNet: https://wordnet.princeton.edu/